



Inteligencia artificial aplicada a los mercados financieros: Optimización de carteras mediante aprendizaje profundo por refuerzo

Autor: Don Martín Herrera Roncero.

Máster en Finanzas

Instituto de Estudios Cajasol

Curso académico 2024/2025

"Por la presente certifico que este informe y la presentación que lo acompaña es mi propio trabajo, original en su totalidad, salvo donde expresamente se indique lo contrario y se cite convenientemente la fuente."

Resumen ejecutivo

Introducción y objetivo del trabajo

El trabajo busca construir un modelo de optimización de carteras empleando técnicas avanzadas de inteligencia artificial, concretamente las subramas deep learning y deep reinforcement learning haciendo uso de una arquitectura innovadora. Esto se plantea tras probar las deficiencias de los métodos de optimización tradicionales, el supuesto de linealidad y la carencia de adaptación en entornos dinámicos. Por otro lado, se explica como este nuevo enfoque basado en inteligencia artificial resuelve estos problemas.

Metodología

El deep reinforcement learning o el aprendizaje profundo por refuerzo, es un método de inteligencia artificial en el que el modelo aprende al tomar decisiones y evaluarlas mediante premios y penalizaciones definidos en una función de recompensa. En este caso, explicado de manera sencilla, la función de recompensa se encuentra definida para maximizar la rentabilidad por riesgo. De modo que el algoritmo tratará de maximizar su recompensa, maximizando la rentabilidad por riesgo.

Las decisiones que toma el agente en este caso es la de asignar el peso de los diferentes activos en la cartera.

Para enriquecer al modelo y ayudarlo en la toma de decisiones se añaden variables explicativas, indicadores de análisis técnico, variables macroeconómicas, términos de corrección de error derivados de análisis de cointegración y otras variables que describen el comportamiento estadístico de los activos.

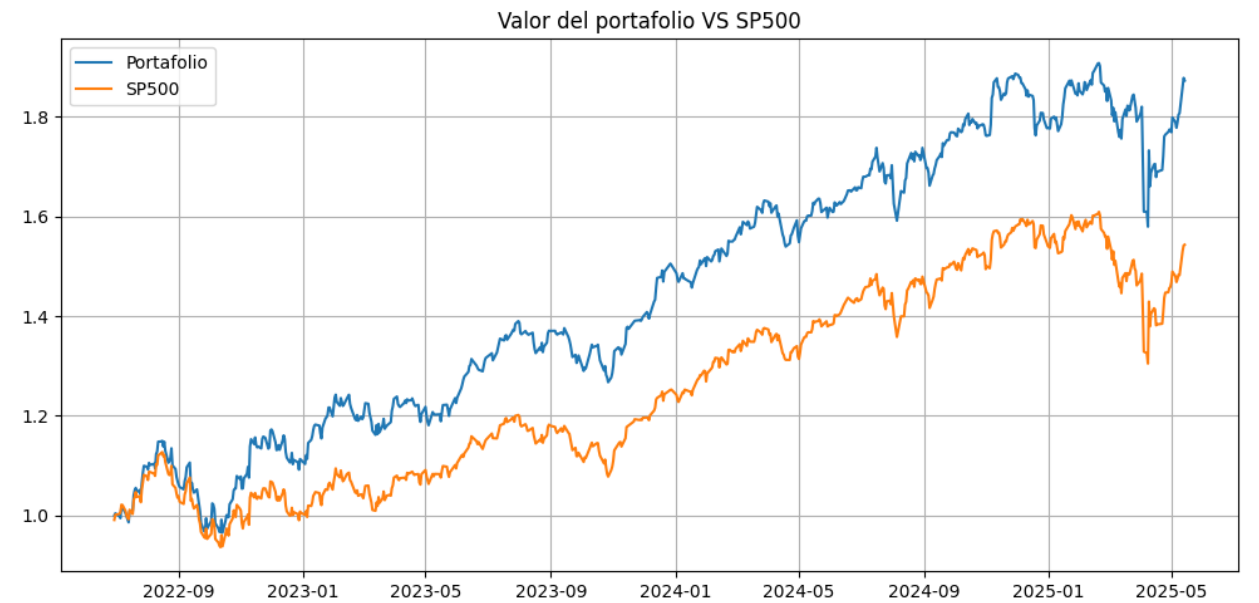
El universo de acción del modelo es el mercado estadounidense, en concreto trabaja con las acciones incluidas en el índice S&P500, adicionalmente se permite comprar ETFs de bonos del tesoro y mantener hasta un 20% de efectivo en la cartera.

A la hora de la implementación práctica del modelo se han empleado diferentes arquitecturas y diferentes algoritmos de deep reinforcement learning, en concreto PPO, DDPG y SAC. Tras realizar pruebas y validaciones en busca del mejor modelo se ha determinado que la combinación de una red neuronal transformer junto con una red neuronal LSTM y el algoritmo PPO es la que ofrece mejor rendimiento. Por otro lado, merece la pena mencionar que esta combinación conforma un enfoque innovador del que existe menor literatura sobre su aplicación a gestión de carteras.

Resultados y conclusiones

Este desarrollo ha dado como resultado una cartera que supera significativamente y en todos los aspectos a la optimización por Markowitz y al desempeño de fondos de inversión con benchmark y objetivos similares. Esto se consigue gracias a la gran capacidad de adaptación al entorno y modelaje y captura de relaciones no lineales entre activos y variables explicativas.

En concreto se han obtenido los siguientes resultados en el intervalo temporal 30 de junio de 2022 a 16 de mayo de 2025



	Valor final	Sharpe	TE	IR	Máximo drawdown	Rentabilidad anual	Volatilidad anual
Cartera	1,8690	1,2020	5,86%	1,295	32,73%	27,45%	19,13%
SP500	1,5430	0,7586	-	-	18,90%	18,32%	18,32%

Sin duda este enfoque ha demostrado una eficacia abrumadora frente a los modelos tradicionales y los fondos de inversión.

Sin embargo, como desventaja, el desarrollo y ejecución de estos modelos supone un coste computacional elevado, esta limitación se ha hecho evidente a lo largo del trabajo obligando a recortar ciertos aspectos del desarrollo del modelo. A la hora de emplear estos modelos se ha de tener en cuenta la disponibilidad hardware capaz.

Dadas las limitaciones sufridas en el desarrollo se sugieren mejoras a implementar, como añadir mayor complejidad al entorno, añadir más variables explicativas o utilizar redes neuronales para crear nuevas variables que aporten señales útiles al modelo

Otra limitación independiente del modelo sufrida en el trabajo, ha sido la indisponibilidad de datos con frecuencia de actualización inferior a la diaria ya que son ofrecidos por plataformas de pago a inversores institucionales. El incluir estos datos seguramente potencie el desempeño del modelo pues las redes neuronales complejas destacan en estos entornos.

Por último, el modelo presenta una clara aplicabilidad inmediata en el ámbito profesional, multitud de entidades financieras en posesión de carteras de inversión pueden beneficiarse de esta tecnología. Es más, las aplicaciones no solo se limitan a la gestión de carteras de valores, también se presentan oportunidades claras en tareas de control de gestión empresarial.

Inteligencia artificial aplicada a los mercados financieros: Optimización de carteras mediante aprendizaje profundo por refuerzo1

1.	Introducción	6
1.1.	Contexto general	6
1.2.	Justificación del tema	7
1.3.	Objetivo del trabajo.....	7
1.4.	Metodología breve	7
1.5.	Alcance y limitaciones.....	8
1.6.	Estructura del trabajo.....	8
2.	Estado del arte y marco teórico	8
2.1.	Conceptos fundamentales sobre optimización de carteras.....	8
2.1.1.	Teoría de selección de carteras de Harry Markowitz (1952)	8
2.1.2.	Extensiones y mejoras a la teoría clásica.....	11
2.1.3.	Supuesto de linealidad	12
2.2.	Inteligencia artificial y aprendizaje automático en optimización de carteras	13
2.2.1.	Machine learning	13
2.2.2.	Deep learning	15
2.2.3.	Reinforcement learning	21
2.2.4.	Revisión de literatura de reinforcement learning aplicado a la optimización de carteras	26
2.2.5.	Desafíos del deep reinforcement learning.....	26
3.	Metodología.....	27
3.1.	Objetivo del modelo	27
3.2.	Datos utilizados	27
3.3.	Variables explicativas	28
3.3.1.	Indicadores técnicos	28
3.3.2.	Variables macroeconómicas	30
3.3.3.	Otras variables	31
3.3.4.	Análisis correlación entre variables	32
3.3.5.	Análisis cointegración	34
3.4.	Entorno de simulación	37
3.4.1.	Espacio de observación	37
3.4.2.	Espacio de acción	37
3.4.3.	Lógica interna.....	37
3.4.4.	Función de recompensa.....	38
3.5.	Algoritmos.....	38
3.6.	Estrategia de validación del modelo	39
3.7.	Métricas de evaluación	39

3.8.	Herramientas para la implementación	40
4.	Implementación práctica.....	41
4.1.	Limitaciones en la implementación.....	41
4.2.	Selección de acciones.....	41
4.3.	Función de recompensa.....	43
4.4.	Resultados de los diferentes extractores de características	44
4.4.1.	Sin extractor de características.....	45
4.4.2.	Transformer+LSTM tamaño de red pequeña	46
4.4.3.	Transformer+LSTM tamaño de red grande.....	47
4.4.4.	CNN+LSTM	48
4.4.5.	Selección de arquitectura.....	50
4.5.	Selección del algoritmo de deep reinforcement learning a emplear en el modelo final	50
4.5.1.	Optimización de hiperparámetros de los diferentes algoritmos de Deep Reinforcement Learning.....	50
4.5.2.	Pruebas de rendimiento finales de cada uno de los algoritmos.....	53
5.	Resultados	54
5.1.	Resultados del modelo final	54
5.2.	Comparación contra optimización por Markowitz.....	56
5.3.	Comparación contra fondos de inversión con objetivo y benchmark similar	59
6.	Conclusiones y posibles vías de mejora del modelo	59
7.	Referencias	62
8.	Anexo	64
8.1.	Distribuciones rentabilidades diarias valores S&P500	64
8.2.	VECM tipos de interés, tipos de cambio, S&P500, oro y barril WTI.....	66
8.3.	VECM tipos de interés tipos de cambios	74
8.4.	Objetivos y estrategias de fondos de inversión referenciados.	82
8.5.	Código íntegro de python	84

1. Introducción

En esta introducción al trabajo se contextualizará el tema a tratar mediante una breve exposición de la evolución histórica de la optimización de carteras y tendencias actuales en la industria financiera. Se presentarán los objetivos del proyecto y se describirá brevemente la metodología que se utilizará.

1.1. Contexto general

Para comenzar definimos cartera como el conjunto de activos que un inversor tiene en posesión con el objetivo de que le reporte utilidad en el sentido económico. Considerando a los agentes económicos racionales estos desearán tomar las decisiones más eficientes según sus preferencias. Es aquí donde entra en juego la optimización de carteras.

A lo largo del último siglo se ha desarrollado una extensa literatura económica sobre este menester. La teoría moderna de la optimización de carteras sienta sus bases gracias a Harry Markowitz que introduce el concepto de eficiencia de portafolios. Esto es, para un conjunto de activos se evalúan las diferentes composiciones de cartera que maximizan la rentabilidad para un nivel de riesgo dado, o, minimizan el riesgo para una rentabilidad dada. Todo esto parte de la idea de que los inversores son seres racionales y por lo tanto tienen aversión al riesgo y buscan maximizar la rentabilidad esperada. (Markowitz, 1952)

A partir del modelo de Markowitz ha surgido un flujo incesante de trabajos académicos modificando y añadiendo mejoras a la teoría de selección de carteras. Entre las principales extensiones y diferentes propuestas al modelo de Markowitz destacan:

El modelo de Black-Litterman modifica al modelo de Markowitz mediante el uso del Capital Asset Pricing Model (CAPM) (Sharpe, 1964) y añadiendo opiniones del inversor mediante un enfoque bayesiano, solucionando de este modo problemas como la alta sensibilidad que presentaba el modelo de Markowitz. (Black & Litterman, 1992)

Por otro lado, se han desarrollado modelos que utilizan medidas de riesgo diferente a la varianza como MAD (Mean Absolute Deviation), este modelo permite obtener resultados similares a Markowitz utilizando optimización lineal en vez de optimización cuadrática lo que facilita enormemente el cálculo, de este modo permite añadir una variedad más amplia de activos a la cartera (Konno & Hiroaki, 1991). Otra propuesta es utilizar medidas de riesgo como el Value-at-Risk, Smoothed VaR o Conditional CVaR, de manera que el problema de optimización gire en torno a estas medidas. (Gaivoronski & Pflug, 2005) (Krokhmal, et al., 2002).

Más recientemente y con el desarrollo de la industria de la ciencia de datos han surgido enfoques basados en machine learning y deep learning. Sin entrar en detalle, se puede decir que estos modelos aprenden estructuras complejas en grandes conjuntos de datos permitiendo en ocasiones obtener resultados mejores que los que se obtendrían empleando métodos tradicionales, entendiendo por métodos tradicionales aquellos que son diferentes a machine learning y deep learning. En esta línea han surgido enfoques puramente basados en deep learning como el trabajo de (Zhang, et al., 2020) donde se maximiza la ratio de Sharpe empleando una red LSTM (Long Short Term Memory) obteniendo resultados positivos frente a enfoques tradicionales. También se han adoptado otros enfoques híbridos donde se combinan deep learning junto métodos tradicionales, Obeidat, et al. (2018) combinan la optimización de Markowitz con predicciones generadas por una red neuronal LSTM. Por último, otra corriente adoptada en este campo es la del aprendizaje por refuerzo, esto consiste en entrenar al modelo mediante recompensas y penalizaciones con el fin de maximizar alguna variable, artículos como el de Soleymani & Paquet (2020) utilizan este enfoque.

1.2. Justificación del tema

Muchos de los métodos tradicionales anteriormente expuestos presentan limitaciones y se basan en supuestos que en ocasiones no se cumplen y otros que se pueden tachar de cuestionables. En el apartado dos de este trabajo se tratará en detalle este tema. De manera breve se pueden mencionar ciertas controversias del modelo de Markowitz (1952) como que se asume que los retornos de los activos siguen una distribución normal futura conocida, mercados eficientes y es un modelo estático. El modelo de Black-Litterman (1992) soluciona algunos de estos aspectos sin embargo también presenta objeciones como la calibración de las opiniones subjetivas o basarse en el modelo CAPM (Sharpe, 1964) que presenta limitaciones importantes. Modelos basados en VaR y CVaR como los de Gaivoronski & Pflug (2005) y Krokmal, et al. (2002) también presentan limitaciones como una distribución de retornos conocida, no convexidad para los basados en VaR, sensibilidad a los escenarios en CVaR o la imposibilidad de distinguir eventos dentro de la cola de distribución.

Los modelos de machine learning y deep learning no requieren de estos supuestos, parten de datos en bruto y se autoajustan hasta que encuentran el mínimo de su función de pérdida. Un supuesto limitante común en los modelos tradicionales es el de la linealidad, este nuevo enfoque no requiere de este supuesto por lo que se pueden capturar relaciones más complejas. Además se trata de modelos dinámicos que se autoajustan ante variaciones en los datos, esto les permite reaccionar ante el dinamismo del mercado. En la sección dos y tres se detallarán estos aspectos.

Es por esto que es relevante estudiar e implementar estas técnicas en la construcción de carteras óptimas.

El enfoque base a tomar será el de aprendizaje profundo por refuerzo debido a que no se está resolviendo ningún problema de clasificación ni de regresión. El objetivo es que la tarea de optimización la lleve a cabo un algoritmo. Enfoques como redes neuronales se podrán utilizar como complementarios con el objetivo de modelar distintas variables para potenciar la interpretación de datos del modelo.

1.3. Objetivo del trabajo

El objetivo de este trabajo es implementar las ideas anteriormente expuestas en el desarrollo de un modelo basado en aprendizaje profundo por refuerzo. El modelo optimizará los pesos asignados a cada activo en la cartera de inversión mediante el entrenamiento basado en una serie de indicadores técnicos, macroeconómicos y otras variables explicativas propias de cada activo.

Se implementarán varios modelos de aprendizaje profundo por refuerzo y se compararán sus desempeños contra un benchmark, el índice Standard and Poor's 500 (S&P500), el objetivo de la cartera será el de superar al rendimiento del índice mientras se introduce cierta aversión al riesgo. Además, se evaluarán los resultados entre los diferentes modelos y se compararán contra fondos de inversión con el mismo benchmark y objetivo de la cartera y optimización por Markowitz.

1.4. Metodología breve

Para llevar a cabo el objeto principal del trabajo se aplicarán algoritmos de aprendizaje profundo por refuerzo. Estos algoritmos se basan en maximizar su función de recompensa, o bien, minimizar su función de pérdida. Esta función tiene origen en las recompensas y penalizaciones que el modelo obtiene al realizar una acción. Con la experiencia, es decir, tras entrenar al modelo en reiteradas ocasiones este consigue aprender cuáles son las acciones óptimas a ejecutar para maximizar sus recompensas.

En nuestro caso particular las recompensas y penalizaciones son definidas, a grandes rasgos, como los rendimientos positivos y negativos que obtiene el portafolio y la volatilidad. De este modo el modelo buscará maximizar retornos positivos minimizando volatilidad. Complementariamente se añadirán penalizaciones para introducir más aversión al riesgo.

El modelo se construirá utilizando el lenguaje de programación Python utilizando las librerías: yfinance, pandas, numpy, ta, scikit-learn, torch, gym, matplotlib, stablebaselines3 y optuna.

1.5. Alcance y limitaciones

El modelo se encuentra limitado por motivos de simplicidad al mercado estadounidense, en concreto se han escogido las 500 acciones del índice S&P500 a abril de 2025.

El horizonte temporal del trabajo abarca desde enero de 2012 a mayo de 2025. Por motivos de accesibilidad a los datos se trabaja con un intervalo temporal diario.

La fuente de datos utilizada es Yahoo Finance ya que está integrada en Python a través de la librería yfinance.

El trabajo sufre de una importante limitación de hardware ya que en ocasiones se tiene que limitar el modelo para no consumir la totalidad de la memoria VRAM de la tarjeta gráfica durante el entrenamiento. Del mismo modo se sufre una limitación en cuanto al tiempo de cómputo, entrenar ciertos modelos requiere de horas de cómputo. Por lo que a la hora de evaluar distintos modelos es requerido bajar el número de iteraciones en el entrenamiento para realizar esta tarea en un tiempo razonable. Esto puede tener impacto en algunos modelos pues puede que necesiten un número grande de iteraciones para converger hacia el mínimo de la función de pérdida.

Durante el desarrollo del trabajo se irán evidenciando estas limitaciones que serán explicadas detalladamente más adelante.

1.6. Estructura del trabajo

El trabajo se va a desarrollar de la siguiente forma:

- Estado del arte y marco teórico: se explicarán conceptos básicos sobre la optimización de carteras e inteligencia artificial y se expondrán los modelos más relevantes.
- Metodología: se detallarán los datos empleados, los algoritmos que se van a implementar y una breve descripción de los recursos utilizados.
- Implementación práctica: De manera práctica se explicará la construcción y validación de los modelos.
- Resultados: visualización de los datos de los diferentes modelos, comparación de indicadores clave y comparación respecto a fondos de inversión y optimización por Markowitz.
- Conclusión y posibles vías de mejora del modelo: resumen de los resultados obtenidos e indicación de posibles mejoras a añadir en el modelo.
- Referencias bibliográficas
- Anexo

2. Estado del arte y marco teórico

2.1. Conceptos fundamentales sobre optimización de carteras

2.1.1. Teoría de selección de carteras de Harry Markowitz (1952)

Markowitz mediante un enfoque cuantitativo introduce la teoría moderna de optimización de carteras. Propone que un inversor racional buscará maximizar el retorno dado un nivel de riesgo o buscará minimizar el riesgo dado un nivel de retorno.

El modelo supone que los retornos futuros de los activos siguen distribuciones normales conocidas, los inversores son aversos al riesgo y no se permiten ventas en corto.

Siendo N el número de activos, X_i la proporción asignada al activo i , por lo tanto $\sum_i^N X_i = 1$, μ_i el rendimiento esperado del activo i y σ_{ij} la covarianza entre los activos i y j .

Rendimiento esperado de la cartera:

$$E(R) = \sum_{i=1}^N X_i \mu_i$$

Varianza de la cartera:

$$\sigma^2 = \sum_{i=1}^N \sum_{j=1}^N X_i X_j \sigma_{ij}$$

El inversor elegirá carteras que maximicen el retorno para una varianza dada o minimicen la varianza para un retorno dado. Entonces se define un problema de optimización tal que

$$\begin{aligned} \text{Max} \quad & E(R) = \sum_{i=1}^N X_i \mu_i \\ \text{s.a.} \quad & \sum_{i=1}^N X_i = 1 \\ & X_i \geq 0 \\ & \sigma^2 = (\text{varianza dada}) \end{aligned}$$

O bien,

$$\begin{aligned} \text{Min} \quad & \sigma^2 = \sum_{i=1}^N \sum_{j=1}^N X_i X_j \sigma_{ij} \\ \text{s.a.} \quad & \sum_{i=1}^N X_i = 1 \\ & X_i \geq 0 \\ & E(R) = (\text{rendimiento dado}) \end{aligned}$$

Tomemos por ejemplo los valores Microsoft, Nvidia y Philip Morris para construir una cartera óptima utilizando Markowitz.

Tabla 1 Rentabilidad y varianza anual media de Microsoft, Philip Morris y Nvidia 2021-2025

	Rentabilidad media anual	Varianza anual
Microsoft	20.83%	0.0679
Nvidia	72.29%	0.2795
Philip Morris	16.97%	0.398

Fuente: Yahoo finance, elaboración propia

Tabla 2 Matriz covarianzas Microsoft, Nvidia y Philip Morris 2021-2025

	Microsoft	Nvidia	Philip Morris
Microsoft	0.0679	0.0867	0.0079
Nvidia	0.0867	0.2795	0.0045
Philip Morris	0.0079	0.0045	0.0398

Fuente: Yahoo finance, elaboración propia

El resultado de resolver el problema de optimización para, por ejemplo, una rentabilidad dada del 22% anual es:

Tabla 3 Pesos óptimos de cartera por optimización de Markowitz para MSFT, NVDA y PM

	Pesos de los activos en la cartera
Microsoft	27.11%
Nvidia	7.19%
Philip Morris	65.70%

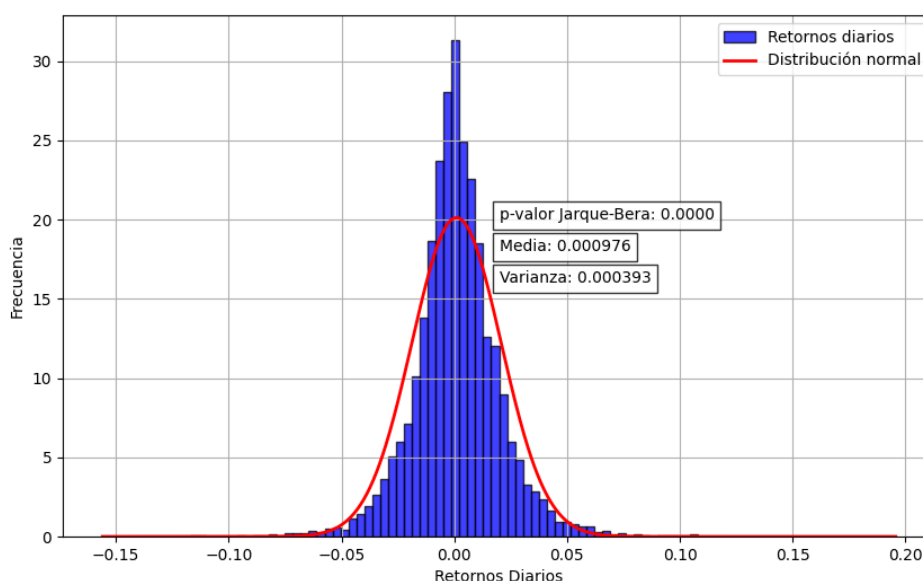
Fuente: elaboración propia

Con una desviación típica de la cartera de 17.40%

Este modelo presenta limitaciones:

1. Se presume que los retornos futuros son una variable aleatoria que sigue una distribución normal con media y desviación típica igual a la histórica. Esta simplificación en ocasiones resulta no ajustarse a la realidad, existen multitud de activos que no ajustan sus retornos a una distribución normal, además el valor en el futuro no tiene porqué comportarse como se ha comportado históricamente. Sin realizar una profunda selección de valores tomemos Microsoft como ejemplo ya que lo habíamos usado con anterioridad. Comprobaremos si este supuesto se cumple:

Gráfico 1 Distribución de los retornos diarios de Microsoft 1990-2025



La distribución de rentabilidades diarias de Microsoft en los últimos 35 años no se ajusta a una distribución normal como indica el contraste de Jarque-Bera. Además, si dividimos el intervalo temporal en dos la media y varianza observada para el periodo 1990 a 2018 (0.000959 y 0.000405 respectivamente) distan de ser iguales a las observadas de 2018 a 2025 (0.001034 y 0.000334). Estas evidencias probablemente también se cumplan para otros valores. En el anexo se muestra una tabla que indica si se los rendimientos diarios de las 100 acciones de mayor capitalización del S&P500 a marzo de 2025 se ajustan a una distribución normal, ninguno de ellos se ajusta.

2. El modelo también presenta una alta sensibilidad a las estimaciones de las rentabilidades y varianzas futuras, problema que se recrudece si tenemos en cuenta que la afirmación de que la distribución histórica es igual a la distribución futura es cuestionable. Volviendo a nuestro ejemplo veamos como varía la asignación de pesos ante cambios mínimos en μ e σ .

Tabla 4 Nuevos valores de rentabilidad y varianza MSFT, NVDA y PM

	Rentabilidad media anual	Varianza anual
Microsoft	21%	0.07
Nvidia	72 %	0.28
Philip Morris	17%	0.04

Fuente: elaboración propia

Los pesos nuevos óptimos serían los siguientes

Tabla 5 Reasignación de pesos tras variar parámetros en Markowitz

	Pesos originales	Nuevos pesos
Microsoft	27.11%	33.60%
Nvidia	7.19%	6.65%
Philip Morris	65.70%	59.75%

Fuente: Elaboración propia

Y una nueva volatilidad de la cartera del 15.30%

- Por otro lado, el modelo original no concibe reequilibrios de la cartera, se podrían implementar, pero dada la naturaleza inestable del modelo probablemente se llegue a cambios extremos en la asignación de pesos. Esto empeora si tenemos en cuenta que el modelo tampoco contempla costes de transacción e impuestos.
- Por último, otra limitación importante de este modelo es que se basa en relaciones lineales entre activos por lo que es incapaz de captar relaciones más complejas que rigen el comportamiento del mercado. Además, la matriz de covarianzas, que juega un importante papel, es cambiante, es decir existe heterocedasticidad, esto puede hacer que se desvirtúe la asignación de pesos para un momento t .

Podemos concluir que el modelo de Markowitz sienta las bases de la optimización de carteras, a pesar de ello el modelo es limitado lo que abre la puerta a posibles mejoras.

2.1.2. Extensiones y mejoras a la teoría clásica

2.1.2.1. Modelo Black-Litterman (Black & Litterman, 1992)

Fruto de las debilidades del modelo de Markowitz (1952) han surgido multitud de variaciones a su teoría entre ellas destaca el modelo de Black-Litterman (Black & Litterman, 1992).

Este modelo ataja el problema de estimación de los retornos futuros combinando equilibrio de mercado calculado en base al modelo CAPM global y opiniones subjetivas del inversor.

Los retornos esperados se calculan partiendo del equilibrio del modelo CAPM global. Se calculan los rendimientos implícitos del mercado en base a la aversión al riesgo del inversor, la matriz de covarianza entre activos y los pesos de los activos en el total del mercado.

$$\Pi = \lambda \sum \omega_m$$

Π =vector de retornos implícitos del mercado

λ = aversión al riesgo del inversor

Σ = matriz de covarianzas entre activos

ω_m = vector de pesos de los activos en el mercado

Una vez se obtiene el vector de rendimientos se continúa incorporando las opiniones del inversor sobre el rendimiento de los activos. Por ejemplo, una opinión relativa, el activo A rendirá un 2% más que el activo B.

Las opiniones se estructuran en una matriz P que se cuantifica mediante un vector Q que recoge la información sobre el rendimiento que se espera de los activos en base a las opiniones subjetivas del inversor. Volviendo al ejemplo anterior, este se podría expresar como $P = [1, -1]$ $Q = [0.02]$. Cada una de estas opiniones cuenta con un grado de confianza del inversor que viene denotado por la matriz Ω . Toda esta información se combina con los rendimientos esperados del mercado para obtener los rendimientos esperados ajustados por las opiniones del inversor(μ).

$$\mu = \left[(\tau \sum)^{-1} + P' \Omega^{-1} \right]^{-1} \left[(\tau \sum)^{-1} \Pi + P' \Omega^{-1} Q \right]$$

Esto es una media condicional bayesiana, balancea los rendimientos esperados del mercado con las opiniones del inversor. Una vez se obtiene μ se procede a resolver el problema de optimización media-varianza semejante al del modelo de Markowitz (1952).

Este modelo mejora la sensibilidad ante cambios en las variables que sufría el modelo de Markowitz, sin embargo, sigue presentando debilidades. Entre otras objeciones se pueden nombrar que, de nuevo establecer como eje central del modelo la matriz de covarianzas supone que solo se captan relaciones lineales. Además, en presencia de heterocedasticidad la matriz cambia a lo largo del tiempo lo que puede llevar a asignaciones erróneas de activos, aunque esto es algo que se puede paliar empleando otros métodos estadísticos. Por otro lado, la matriz Ω es difícil de cuantificar y también se asume normalidad en la distribución de los rendimientos esperados de los activos.

2.1.2.2. Otros modelos

Junto a las dos principales obras expuestas anteriormente se han desarrollado otros modelos.

1. Modelos minimax: se basan en un problema de programación lineal que tiene el objetivo de maximizar el retorno mínimo garantizado. Se obtienen resultados similares al modelo de Markowitz. (Qiang Cai, et al., 2004)
2. Modelos MAD: utilizan la media de la desviación absoluta como parámetro a minimizar en un problema de programación lineal. Computacionalmente requiere menos recursos que el modelo de Markowitz lo que permite construir carteras más diversificadas. Se ha observado que estos modelos son más efectivos que el modelo de Markowitz. (Gunjan & Bhattacharyya, 2023)
3. Modelos basados en VaR y CVaR: otros enfoques han adoptado como medida de riesgo el Value-at-risk (VaR) y el Conditional VaR. El VaR representa la pérdida máxima esperada a un nivel de confianza, por ejemplo 95%, aplicado a la optimización de carteras se tratará de minimizar el VaR. Sin embargo, presenta problemas como no convexidad que dificulta la minimización, en ocasiones no promueve diversificación y no informa sobre las pérdidas más allá del umbral. Para solucionar estos problemas se ha propuesto el enfoque del CVaR que resuelve el problema de no convexidad y promueve la diversificación, además cuantifica las pérdidas como una media de los valores por debajo de un percentil dado, informa sobre la pérdida promedio esperada en la cola de la distribución. (Gaivoronski & Pflug, 2005) (Krokhmal, et al., 2002)

2.1.3. Supuesto de linealidad

Los modelos "tradicionales" se desarrollan asumiendo supuestos que normalmente no se asemejan al entorno real, por ejemplo, distribución normal de los retornos diarios, ausencia de impuestos y costes de transacción. Mas allá de estos supuestos existe otro factor especialmente restrictivo y limitante en estos modelos, la linealidad. Los modelos asumen que la interdependencia entre activos se describe a través de relaciones lineales, se basan en matrices de covarianzas.

Esto tiene varias desventajas, es posible que las varianzas de los activos no sean constantes en el tiempo por lo que las covarianzas tampoco lo serán. Consecuentemente pueden darse casos en los que un activo A solo presente correlación respecto a un activo B en periodos de crisis.

Por otro lado, se ignoran relaciones no lineales. Una relación lineal entre dos activos puede ser $r_a = 2r_b$ sin embargo pueden darse casos que esa relación no sea lineal, por ejemplo $r_a = (r_b)^2$.

Cuando existe una relación lineal la covarianza sí resulta ser un valor representativo ya que:

Suponiendo que X e Y son variables aleatorias de media cero y distribución simétrica, Si $Y = aX + b + \varepsilon$

$$\text{cov}(X, Y) = \text{cov}(X, aX + b + \varepsilon) = a * \text{var}(X) + \text{cov}(X, \varepsilon)$$

Como ε es un término aleatorio $\text{cov}(X, \varepsilon) = 0$

Entonces $\text{cov}(X, Y) = a * \text{var}(X)$, la covarianza refleja la dependencia lineal.

Sin embargo, cuando existe una relación no lineal:

Si $Y = X^2$

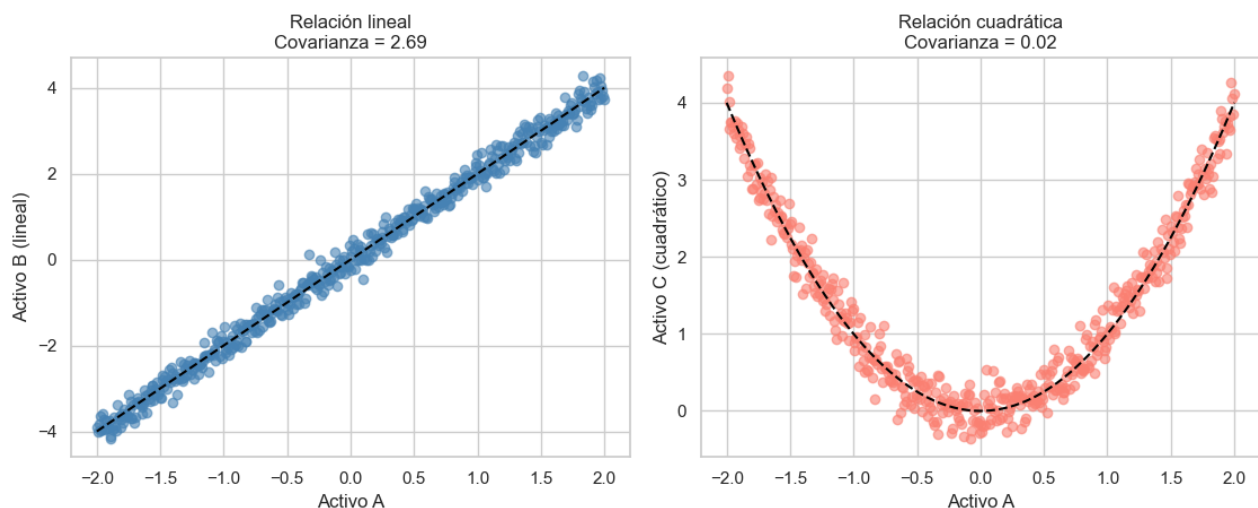
$$\text{cov}(X, Y) = E(XY) - E(X)E(Y)$$

$$\text{cov}(X, X^2) = E(X^3) - E(X)E(X^2) = 0$$

A pesar de que existe una dependencia entre las variables la covarianza es cero. Ejemplificándolo:

Suponiendo que el activo A es una variable que se distribuye uniformemente en el espacio $[-2, 2]$, que $\text{ActivoB} = \text{ActivoA} * 2 + \varepsilon$ y $\text{ActivoC} = \text{ActivoA}^2 + \varepsilon$

Gráfico 2 Comparación covarianzas relación lineal y cuadrática



Fuente: elaboración propia.

Por lo tanto, los modelos tradicionales pueden dar lugar a asignaciones subóptimas de activos como resultado de asumir el supuesto de linealidad. Esta es una de las razones por las que se ha recurrido a modelos de inteligencia artificial que son capaces de optimizar carteras sin necesidad de asumir este supuesto.

2.2. Inteligencia artificial y aprendizaje automático en optimización de carteras

Entendemos inteligencia artificial como el conjunto de tecnologías que tratan de replicar el proceso de aprendizaje humano en un ordenador, esto engloba machine learning, deep learning y reinforcement learning, en español aprendizaje automático, aprendizaje profundo y aprendizaje por refuerzo.

2.2.1. Machine learning

A diferencia de como se ha hecho tradicionalmente el machine learning trata de programar al ordenador para que aprenda a realizar tareas a partir de datos en lugar de usar una programación basada en reglas. Existe un modelo matemático a optimizar, el machine learning consiste en ejecutar un programa informático que optimice los parámetros del modelo usando datos y/o la experiencia pasada. De esta manera el modelo también consigue adaptarse a cambios cuando se

introducen datos nuevos, tiene la habilidad de aprender. Los modelos pueden ser predictivos o descriptivos. Ejemplos de modelos usados en machine learning son regresiones lineales, clasificaciones lineales, arboles de decisión, bosques aleatorios o clustering entre otros. (Alpaydin, 2014)

El tipo de aprendizaje se puede clasificar en supervisado y no supervisado:

El aprendizaje supervisado consiste en entrenar al ordenador con datos etiquetados, esto quiere decir datos para los cuales se conoce la respuesta correcta. De manera que por cada resultado que dé el modelo se le aporte la información de si su salida ha sido correcta o no. Por ejemplo, para la clasificación de nombres de personas según género el ordenador dará una salida hombre o mujer y seguidamente se le aportará información de si ha acertado o no.

El aprendizaje no supervisado consiste en entrenar al ordenador con datos no etiquetados, de esta manera el modelo aprenderá patrones presentes en los datos sin que se le aporte información desde el exterior. Por ejemplo, para otro problema de clasificación (esta vez como es aprendizaje no supervisado se denomina clustering) donde se quiere separar tenedores de cucharas presentando imágenes, el algoritmo aprenderá que hay características comunes en los objetos, hay objetos con puntas y objetos sin puntas, el algoritmo los separará sin saber lo que son y sin saber si lo está haciendo correctamente o no.

2.2.1.1. Clustering

En nuestro modelo usaremos clustering por eso se expondrá una breve explicación.

El clustering es un modelo de clasificación de aprendizaje no supervisado que trata de encontrar grupos de características similares en el conjunto de datos. Los datos dentro de un grupo deberán de ser lo mas parecidos entre sí y lo mas diferentes posibles respecto al resto de los grupos. El método más utilizado es k-means que consiste en dividir el conjunto de datos en k grupos mediante k centroides $\{m_i\}_{i=1}^k$. Este método intenta minimizar el error de construcción

$$E(\{m_i\}|X) = \sum_{t=1}^N \sum_{i=1}^k b_{ti} \|x_t - m_i\|^2$$

Donde

$$b_{ti} = \begin{cases} 1, & \text{si } \|x_t - m_i\| = \min_j \|x_t - m_j\| \\ 0, & \text{en otro caso.} \end{cases}$$

(Alpaydin, 2014)

El término $\|x_t - m_i\|^2$ mide el error de aproximar x_t al centroide m_i . La suma de estos errores al cuadrado indica como de preciso han sido los centroides. El objetivo es encontrar los centroides que minimicen la suma de estos errores. Esta minimización se consigue mediante el siguiente proceso. Primero se escogen unos centroides aleatorios $\{m_i\}$ fijos y se asigna cada x_t a su centroide más cercano mediante el parámetro b_{ti} . Una vez se tienen estos parámetros se puede comenzar a minimizar. Se deriva $E(\{m_i\}|X)$ y se iguala a cero, de aquí se obtiene:

$$m_i = \frac{\sum_{t=1}^N b_{ti} x_t}{\sum_{t=1}^N b_{ti}}$$

Que es la media de todos los puntos asignados al cluster i . Con estos nuevos centroides se vuelve al principio y se repite el proceso hasta converger a un minimo.

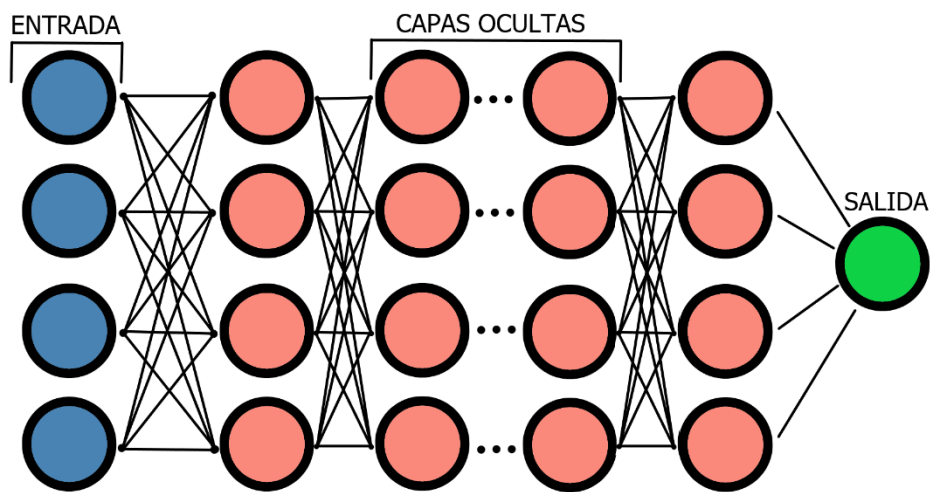
(Alpaydin, 2014)

2.2.2. Deep learning

Inspirándose en el funcionamiento del cerebro humano surge el aprendizaje profundo, en inglés deep learning. Se trata de modelos que interpretan la entrada de información a través de capas paralelas de neuronas, lo que se identifica como una red neuronal. Este enfoque es distinto al machine learning en cuanto a que no requiere de un diseño exhaustivo de las variables y características, la propia red neuronal se encarga de realizar esta tarea. Por ejemplo, para un programa de reconocimiento facial, usando un enfoque de machine learning habría que ofrecerle explícitamente al modelo variables clave como distancia entre ojos. Sin embargo, una red neuronal aprendería estas características directamente desde los datos brutos, las imágenes en este caso. (Alpaydin, 2014)

Para una mejor interpretación una red neuronal básica o Multi-Layer Preception (MLP) se puede representar visualmente:

Ilustración 1 Representación de una red neuronal básica



Fuente: elaboración propia

Se recibe una capa input que se procesa en las capas de neuronas y se devuelve una capa de salida. Matemáticamente el proceso se describe:

La capa de entrada tiene un tamaño n y la capa de salida un tamaño m , suponiendo que se está en la capa l :

$x^{(l)}$: vector de activaciones (salidas) de la capa l

$W^{(l)}$: matriz de pesos de tamaño $n_{l+1} * n_l$

$b^{(l)}$: vector de sesgos de la capa $l + 1$

Con este input la neurona calcularía su output

$$z^{(l+1)} = W^{(l)} * x^{(l)} + b^{(l)}$$

$$x^{(l+1)} = f(z^{(l+1)})$$

Donde $f(\cdot)$ es una función de activación, introduce no linealidad transformando la combinación lineal z , existen varios tipos como ReLU, GeLU, softmax, sigmoide o tanh. (Alpaydin, 2014)

Este proceso se repite hasta la capa de salida. El objetivo de la red neuronal es que su salida sea precisa, por ejemplo, si está prediciendo que su predicción se acerque lo máximo posible al valor real. Para lograr esto se ajustan los pesos y los sesgos. Para realizar este ajuste se define una

función de pérdida, la especificación de la función dependerá del tipo de modelo. El valor de la función de pérdida será mayor cuanto más imprecisa sea la salida de la red neuronal y viceversa. De esta manera el objetivo será minimizar la función de pérdida. Para esto se sigue el método de descenso de gradiente. (Alpaydin, 2014)

$$W^{(l)} \leftarrow W^{(l)} - \eta * \frac{\partial \mathcal{L}}{\partial W^{(l)}}$$

$$b^{(l)} \leftarrow b^{(l)} - \eta * \frac{\partial \mathcal{L}}{\partial b^{(l)}}$$

Donde $\mathcal{L}$ es la función de pérdida y η la tasa de aprendizaje. Los vectores gradientes miden como cambia la función de pérdida ante cambios mínimos en los parámetros de las neuronas. Los vectores de los parámetros de la red cambiarán entonces en la dirección opuesta que indican los gradientes en una cuantía determinada por la tasa de aprendizaje, de este modo nos moveríamos hacia el mínimo de la función de pérdida. Elegir correctamente el valor de la tasa de aprendizaje es crítico en las redes neuronales. Si la tasa es muy pequeña el modelo aprende muy lento e incluso puede quedarse atrapado en secciones planas o en mínimos locales de la función de pérdida, si es muy grande puede que se mueva demasiado rápido a través de la función de pérdida imposibilitando alcanzar el mínimo absoluto y haciendo el modelo inestable. (Alpaydin, 2014)

Existen multitud de tipos de redes neuronales, a la hora de analizar series temporales como cotizaciones de activos destacan RNN (Recurrent Neural Network), en concreto LSTM, y CNN (Convolutional Neural Networks) (Olorunnimbe & Viktor, 2023). También destacan las Graph Neural Network (GNN) (Song, et al., 2020). Recientemente se han desarrollado modelos basados en Transformers (Yañez, et al., 2024). Para el desarrollo de nuestro modelo se usarán estas redes como extractoras de características de las series temporales.

2.2.2.1. Redes Long-Short Term Memory

Al modelar series temporales se trabaja con más de un vector de inputs, uno por cada instante de tiempo. Entonces el descenso de gradiente no solo se produce a través de las capas de neuronas sino también a través de los instantes de tiempo.

Las redes LSTM son redes neuronales creadas con el objetivo de modelar relaciones temporales a largo plazo. Estas redes surgen de la necesidad de solucionar dos problemas al trabajar con series temporales, el desvanecimiento y explosión de gradiente. Esto es, cuando el producto de la derivada de la función de activación y los pesos es menor a 1 y mayor a 1 respectivamente.

$$f'(z^{(l+1,t+1)}) * W < 1 \quad y \quad f'(z^{(l+1,t+1)}) * W > 1$$

Esto provoca, en el primer caso, que los primeros instantes de tiempo apenas reciban señal correctora, y en el segundo caso que reciban señales demasiado grandes desestabilizando el entrenamiento. Por lo tanto, las redes quedarían limitadas temporalmente ya que a cierta cantidad de pasos temporales se pierde la señal.

Para solucionar esto las redes LSTM introducen el Constant Error Carousel (CEC), donde $f'(z^{(l+1,t+1)}) * W = 1$, esto hace que el error se propague a lo largo de todos los pasos de tiempo.

La arquitectura de una célula de una red LSTM es la siguiente:

El sumador inicial recibe el vector de inputs del momento τ , salidas de neuronas de la misma capa y la salida de la capa anterior. Tras pasar por una función de activación $g(x)$ se multiplica por la puerta de entrada y_{in} que es un valor entre 0 y 1 que determina cuanta información se incorpora.

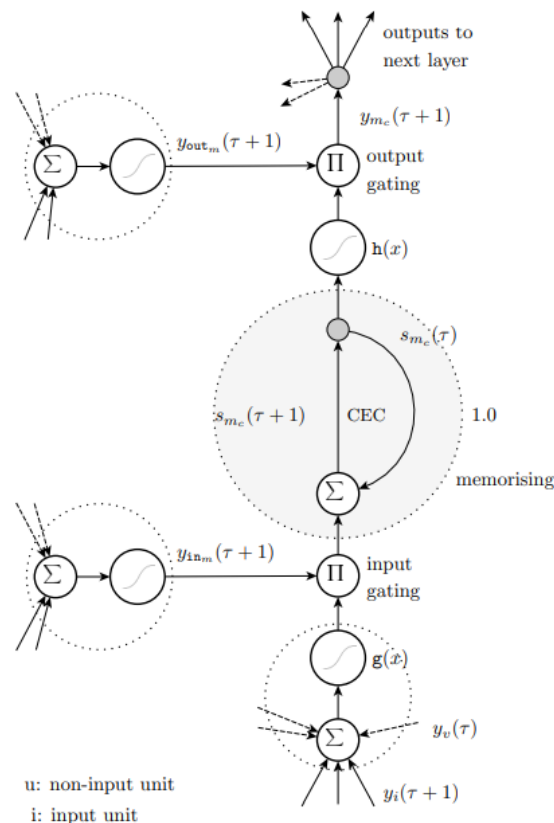
El siguiente paso es el CEC, cada neurona contiene un estado $s_{mc}(\tau)$ que actúa como memoria a largo plazo, es la información que se había almacenado hasta el momento τ . El nuevo estado se

calcula como $s_{mc}(\tau + 1) = s_{mc}(\tau) \times 1 + \text{nuevo input}$. 1 es el termino del CEC que impide que la informacion anterior se diluya en el tiempo.

La salida del CEC se transforma por una función de activación $h(x)$ para combinarse en la puerta de salida de nuevo con inputs del momento τ y la salida de otras neuronas de la misma capa. Esta puerta decide la información que se transmite a la capa siguiente.

(Staudemeyer & Rothstein Morris, 2019)

Ilustración 2 Arquitectura de una neurona LSTM

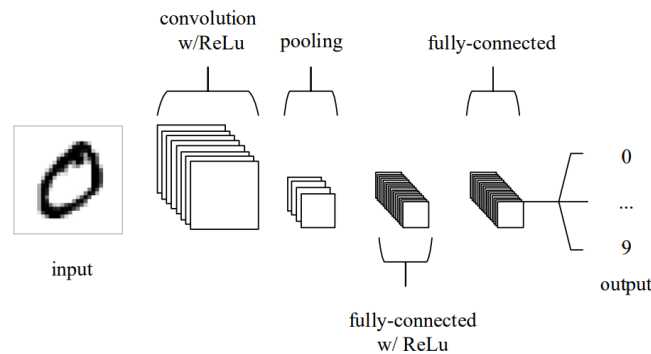


Fuente: (Staudemeyer & Rothstein Morris, 2019)

2.2.2.2. Redes neuronales convolucionales

Una variante de las redes neuronales con las redes neuronales convolucionales (CNN). Estas fueron diseñadas con el objetivo de reconocer imágenes. Sin embargo, también tienen un buen desempeño a la hora de extraer características de series temporales.

Ilustración 3 Arquitectura de una red CNN



Fuente: (O'Shea & Nash, 2015)

Una CNN básica se compone de las siguientes capas:

Capa convolucional: recibe la dimensión espacial de una región del input(imagen) y determina una salida, que se puede interpretar como las características de esa región de la imagen. Aplica filtros llamados "kernels" que detectan rasgos del input generando un mapa de activación 2D. Esta salida se dirige a la capa de agrupación. Al centrarse solo en regiones del input, se reduce el número de parámetros de la red lo que la hace menos pesada en términos computacionales y reduce el overfitting.

Capa de agrupación: reduce la dimensión de la salida de las capas convolucionales, de modo que hace más sencilla la red neuronal. Vuelve a aplicar kernels reduciendo el mapa de activación a un 25% del tamaño original. Con esto se consigue reducir aún más los requerimientos de potencia de cómputo y el overfitting.

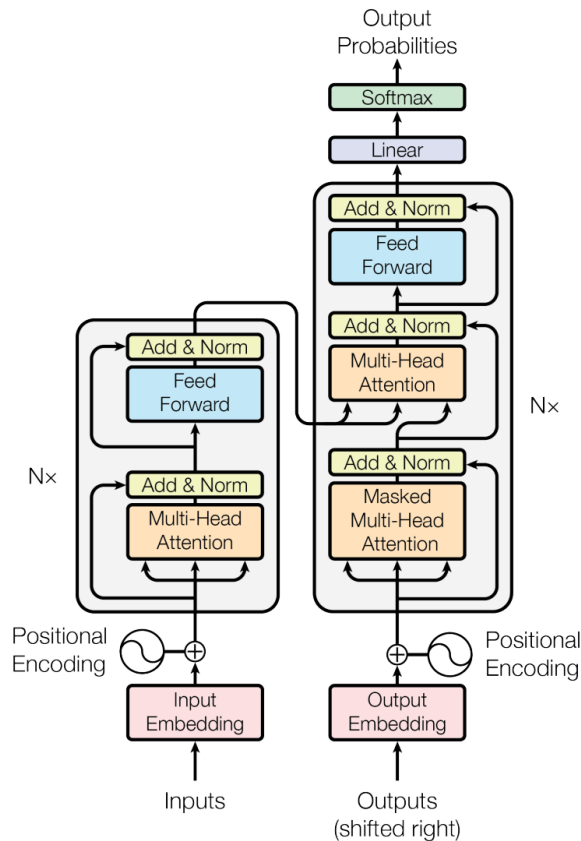
Capas totalmente conectadas: reciben como input la salida de la capa de agrupación y funcionan como una red neuronal básica. Como salida dan la solución a un problema de clasificación.

(O'Shea & Nash, 2015)

2.2.2.3. Redes neuronales Transformers

Estas redes aportan un enfoque distinto a las LSTM y CNN, se basan exclusivamente en "mecanismos de atención" para capturar relaciones entre las entradas y las salidas de la red. Los autores afirman que de esta manera se consiguen mejores resultados empleando menos recursos computacionales. Una ventaja sustancial de estas redes respecto a las CNN consiste en que tratan al conjunto de datos íntegramente mientras que las CNN se centran en espacios locales del conjunto de datos.

Ilustración 4 Arquitectura red transformer



Fuente: (Vaswani, et al., 2023)

La atención se define como:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Donde $Q = XW^O$ representa la información que se quiere extraer, $K = XW^O$ representa un conjunto de $V = XW^O$ valores. d_k es la dimensión de cada cabeza que es definida por $\frac{d_{model}}{h}$ siendo d_{model} la dimensión de la red y h el número de cabezas.

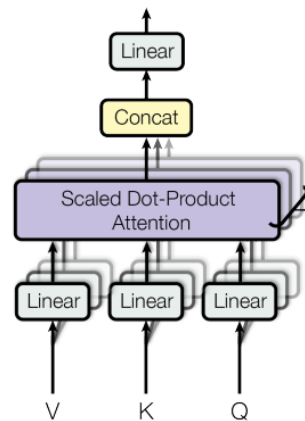
El modelo calcula paralelamente h proyecciones de attention de modo que se enriquece la capacidad de aprendizaje

$$MultiHead(Q, K, V) = Concat(head_1, \dots, head_h)W^O$$

$$head_i = (QW_i^Q, KW_i^K, VW_i^V)$$

(Vaswani, et al., 2023)

Ilustración 5 Multi head attention



Fuente: (Vaswani, et al., 2023)

Sigue una arquitectura encoder-decoder.

En estos modelos se hace necesaria añadir una codificación posicional "embedding", de modo que el modelo interprete un orden en los datos.

El encoder codifica el vector de entrada input mediante una red de 6 capas idénticas. Donde cada capa contiene dos subcapas multi-head self-attention y feed forward position wise. Cada subcapa da una salida que se normaliza $LayerNorm(x + Sublayer(x))$ para estabilizar el entrenamiento. Con esto el encoder transforma $v_{input} = (x_1, \dots, x_n)$ en $v_{codificado} = (z_1, \dots, z_n)$.

El decoder produce una salida en base a $v_{codificado} = (z_1, \dots, z_n)$. El decoder se configura de manera similar al encoder. Presenta 3 subcapas, multi-head self-attention y feed forward position wise y de manera añadida otra multi-head attention que recibe la salida del encoder, $v_{codificado} = (z_1, \dots, z_n)$ y su anterior salida $v_{salida-1} = (y_1, \dots, y_{i-1})$. De nuevo se normaliza la salida de cada subcapa $LayerNorm(x + Sublayer(x))$. El decoder termina dando una salida y_i .

El feed forward es una red neuronal de dos capas para combinar las características extraídas por la capa multihead con el objetivo de capturar patrones complejos.

(Vaswani, et al., 2023)

Aplicado a series temporales, los Transformers pueden modelar relaciones entre variables lejanas en el tiempo, no sufren del problema de desvanecimiento ni explosión de gradiente. El encoder convierte el input en $v_{codificado} = (z_1, \dots, z_n)$ que es una representación de los datos en el momento t . En base a este vector se podría usar el decoder del transformer para capturar las dependencias dinámicas entre variables o usar otra red distinta por ejemplo LSTM.

2.2.2.4. Autoencoders

Otro enfoque ampliamente utilizado en el ámbito de los mercados financieros son los autoencoders. (Olorunnimbe & Viktor, 2023).

"Un autocodificador es un tipo específico de red neuronal, diseñado principalmente para codificar la entrada en una representación comprimida y significativa, y luego decodificarla de nuevo para que la entrada reconstruida sea lo más similar posible a la original." (Bank, et al., 2020, p. 1)

La arquitectura se basa en:

Encoder: el modelo recibe la entrada $x \in \mathbb{R}^n$ y la transforma en una versión comprimida $h \in \mathbb{R}^q$ donde en principio $n > q$.

h sería una representación comprimida de x

Decoder: el algoritmo reconstruiría la representación h en $\hat{x}$

El objetivo de la red neuronal será que su predicción $\hat{x}$ se acerque lo máximo posible a x .

(Michelucci, 2022)

Aplicaciones de los autoencoders son la detección de anomalías y la reducción de ruido de los datos de entrada mediante reducción de dimensionalidad (Michelucci, 2022). Algo importante y útil en los mercados financieros.

Es fundamental elegir correctamente la dimensión a la que se quiere reducir los datos de entrada. Para ello es de especial utilidad el análisis de componentes principales (PCA), en pocas palabras es la versión lineal de un autoencoder que sirve de guía para seleccionar las dimensiones correctas en este.

El análisis PCA es una técnica que tiene como objetivo extraer la información más relevante de una matriz de datos X con I observaciones y J variables correlacionadas para construir una representación de esta con un número menor de componentes a la vez que se capta la mayor parte de la varianza. Este análisis devuelve una lista de componentes con su respectiva aportación marginal a la varianza. (Abdi & Williams, 2010)

Tabla 6 Ejemplo salida PCA

Nº componentes	Varianza explicada
1	30%
2	55%
3	75%
4	88%
5	97%

Fuente: elaboración propia

De esta manera a la hora de aplicar un autoencoder se puede establecer, por ejemplo, un umbral del 85% de modo que se seleccionaran cuatro dimensiones latentes a la hora de aplicar el autoencoder.

2.2.3. Reinforcement learning

El aprendizaje por refuerzo es un tipo de aprendizaje automático en el que el modelo aprende a base de prueba y error, para ello recibe recompensas y castigos según interactúa con el entorno. Tiene como objetivo maximizar su función de recompensa, o bien, minimizar su función de pérdida. El reinforcement learning consta de los siguientes elementos:

Agente: el sistema que aprende y toma las decisiones.

Entorno: el medio externo con el que interactúa el agente.

Estado (s): situación del entorno en el momento t .

Acción (a): decisión que toma el agente, puede ser determinista o estocástica según el tipo de política.

Recompensa (r): valor que indica si la acción fue buena o no. Las recompensas pueden recibirse al final del proceso, lo que sería refuerzo terminal, o de manera continua tras cada acción, lo que sería refuerzo denso.

Política (π): estrategia que el agente sigue para elegir las acciones, puede ser determinista $\pi(s)$ o estocástica $\pi(a|s)$. Las políticas deterministas siempre toman la misma acción dado un estado mientras que las estocásticas asignan una probabilidad a cada acción dado un estado. Este último

tipo de políticas es útil en contextos de incertidumbre permitiendo al modelo realizar labores de exploración diversificando decisiones.

Enlazando con lo anterior, un concepto básico del reinforcement learning es el de exploración y explotación. Con exploración nos referimos a tomar nuevas decisiones con el objetivo de obtener mejores resultados y explotación hace referencia a tomar las acciones que el modelo ya ha probado que funcionan. Encontrar un equilibrio entre estos dos conceptos es clave.

(Sutton & Barto, 2018)

A modo de ejemplo en un contexto financiero, el agente sería el inversor, el entorno el mercado de valores y el estado sería la situación del mercado de valores para un día t . La acción podría ser variar la asignación de pesos de los activos en la cartera. La recompensa se puede contemplar como el retorno del portafolio. Y la política puede interpretarse como la estrategia seguida por el inversor, por ejemplo, "si el RSI se mantiene por encima de la banda de 70 ejecutar orden de venta esperando una bajada de la cotización."

El aprendizaje del modelo ocurre siguiendo este proceso:

1. El agente observa un estado del entorno.
2. Elige una acción en función de la política actual.
3. Recibe recompensa y observa un nuevo estado.
4. Ajusta la política para maximizar la recompensa acumulada a largo plazo.

(Alpaydin, 2014)

Alpaydin (2014) presenta el reinforcement learning como un proceso de decisión de Markov configurado por:

1. S conjunto de estados
2. A conjunto de acciones
3. $P(s'|s, a)$ función de transición
4. $R(s, a)$ función de recompensa

El objetivo del agente es maximizar la recompensa

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

r es la recompensa para cada momento y $\gamma \in [0,1)$ es un factor de descuento de modo que se valoran las recompensas más cercanas en el tiempo (Alpaydin, 2014). Esta definición asume que el modelo tiene un horizonte temporal infinito que es lo que se asemeja a una optimización de cartera.

Otro concepto fundamental del reinforcement learning es distinguir entre enfoques on-policy y off-policy. Los métodos on-policy el agente aprende sobre la política que está empleando mientras que en los métodos off-policy el agente aprende sobre una política diferente a la que está siguiendo. Este último enfoque permite explorar a la vez que optimiza una política óptima en paralelo. (Sutton & Barto, 2018)

2.2.3.1. Deep reinforcement learning

El reinforcement learning presenta limitaciones a la hora de trabajar con entornos complejos como imágenes o series temporales de gran tamaño. Como extensión al reinforcement learning surge el deep reinforcement learning que incorpora deep learning para modelar funciones de política y valores. Esto permite trabajar con datos de mayor tamaño y dimensión (Sutton & Barto, 2018).

En un ámbito como los mercados financieros donde existen multitud de activos y relaciones no lineales el deep reinforcement learning es especialmente útil.

A partir de la definición general de aprendizaje por refuerzo expuesta anteriormente se han desarrollado distintos enfoques a la hora de abordar el problema, se pueden clasificar en tres grandes grupos:

2.2.3.1.1. Métodos basados en valor

Estos métodos aprenden una función de valor. Una función de valor es una cuantificación de como de bueno es un estado o una acción en ese estado. Toma decisiones sin aprender una política

Función de valor de estado para una política π :

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | S_t = s \right]$$

El valor de un estado s bajo una política π . Si se sigue una política estocástica $\pi(a|s)$ esto es el valor esperado de una variable aleatoria dado un momento t . (Sutton & Barto, 2018)

Función de valor de una acción a :

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a] = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} | S_t = s, A_t = a \right]$$

Esto es el valor esperado de tomar una acción a en un estado s bajo una política π . De nuevo si se sigue una política estocástica sería el valor de una variable aleatoria. (Sutton & Barto, 2018)

Los métodos basados en valor se tienen fundamento en la ecuación de Bellman que define el valor de un estado o acción en función de las recompensas futuras esperadas.

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[r_{t+1} + \gamma Q^{\pi}(s_{t+1}, a_{t+1}) | s_t = s, a_t = a]$$

La ecuación de Bellman define el valor de una acción como la recompensa inmediata mas el valor esperado de la siguiente acción. (Sutton & Barto, 2018).

Los métodos basados en valor intentan estimar el valor de cada acción en cada estado de manera que conozcan que decisiones tomar para maximizar la recompensa acumulada a largo plazo. Lo hacen aproximando la ecuación de Bellman interactuando con el entorno para obtener el valor óptimo.

$$Q^*(s, a) = \mathbb{E}[r_{t+1} + \gamma \max_{a'} Q^*(s_{t+1}, a') | s_t = s, a_t = a]$$

Una vez han estimado la función de valor conocen que acciones maximizan el valor por lo tanto saben que acciones tomar para maximizar la función de valor, de esta manera se consigue la política óptima. (Sutton & Barto, 2018)

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$$

Por matizar, los métodos basados en valor no buscan maximizar $Q(s, a)$, lo estiman iterativamente a través de la ecuación de Bellman interactuando con el entorno y una vez que cuentan con esta información el modelo toma la acción con mayor valor para cada estado. (Sutton & Barto, 2018)

La explicación se ha basado en la función de valor de la acción $Q^{\pi}(s, a)$, del mismo modo estos modelos se pueden basar en $V^{\pi}(s)$, sin embargo, según Sutton & Barto (2018) la función de valor de la acción es más útil pues refleja las consecuencias de acciones individuales.

Dos algoritmos representativos de métodos basados en valor son Q-learning (Watkins & Dayan, 1992) y SARSA (Rummery & Niranjan, 1994).

Cuando se trabaja con cantidades grandes de datos estimar directamente $Q(s, a)$ o $V(s)$ no es viable pues existe un gran número de estados, para solucionar esto se han desarrollado DQN (Deep Q-Network) (Mnih, et al., 2013).

Estos métodos suelen ser más simples y tienen menor varianza, pero como inconveniente suelen ser efectivos cuando se trata con variables discretas, además la exploración suele ser limitada. (Sutton & Barto, 2018)

2.2.3.1.2. Métodos basados en políticas

Los métodos basados en valor estimaban las funciones de valor de forma que podían inferir cuales eran las acciones óptimas a tomar, de esta manera construían una política que como fundamento tenía a las funciones de valor. En contraste con este enfoque los métodos basados en políticas aprenden directamente una política parametrizada $\pi = (a|s, \theta)$ (estocástica) o $\pi = (s|\theta)$ (determinista) que puede tomar acciones sin consultar funciones de valor. $\theta \in \mathbb{R}^{d'}$ es el vector de parámetros. (Sutton & Barto, 2018).

La política parametrizada se define como:

$$\pi = (a|s, \theta) = \Pr\{A_t = a | S_t = s, \theta_t = \theta\}$$

La probabilidad de tomar la acción a en un momento t dado un entorno en un estado s en el momento t con unos parámetros θ . (Sutton & Barto, 2018)

El objetivo de estos métodos es encontrar los parámetros que maximizan la función de recompensa $J(\theta)$.

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

Esto se consigue mediante el método de ascenso de gradiente estocástico. A diferencia de las redes neuronales "tradicionales" donde se minimizaba la función de pérdida en el reinforcement learning se trata de maximizar la función de recompensa. (Sutton & Barto, 2018)

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t)$$

Donde α es la tasa de aprendizaje y $\nabla J(\theta_t) \in \mathbb{R}^d$ un estimador estocástico del gradiente respecto a la función de recompensa o medida de rendimiento, esto es el gradiente de la política. En la práctica no es posible calcular el gradiente exacto por eso se usa una estimación hecha con muestras de las trayectorias del agente en el entorno. (Sutton & Barto, 2018)

Los algoritmos basados en política se construyen partiendo de la base del teorema del gradiente

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a|s, \theta)$$

$\mu(s)$ es la distribución estacionaria de estados bajo la política π

$q_\pi(s, a)$ es la función de valor de una acción

$\pi(a|s, \theta)$ es una política parametrizada

Sin entrar en más detalle teórico, el teorema del gradiente de la política facilita el cálculo del gradiente, de otra manera el cálculo dependería de factores desconocidos. (Sutton & Barto, 2018).

Este enfoque presenta ventajas entre la que destaca que puede representar políticas estocásticas de manera natural, hecho que mejora la exploración y convergencia (Sutton & Barto, 2018).

Algoritmos representativos de los métodos basados en políticas son Reinforce (Williams, 1992)

De nuevo para grandes cantidades de datos y alta dimensión se recurre a redes neuronales.

Estos métodos trabajan bien con variables continuas y políticas estocásticas, por el contrario, necesitan una cantidad elevada de datos para converger al óptimo y presentan alta varianza en la estimación del gradiente, es decir hay ruido. Esto puede hacer lento e inestable el aprendizaje. (Sutton & Barto, 2018)

2.2.3.1.3. Métodos basados en actor-critic

Los algoritmos basados en actor-critic combinan los enfoques de los métodos basados en políticas y los de basados en valor. Existen dos agentes, el actor que toma decisiones en base a una política y el crítico estima una función de valor que sirve de aprendizaje al actor. (Sutton & Barto, 2018)

El actor ejecuta una acción a en base a su política $\pi = (a|s, \theta)$ (estocástica) que es evaluada por el critic de la siguiente forma:

$$\delta_t = R_{t+1} + \gamma \hat{V}(S_{t+1}, w) - \hat{V}(S_t, w)^1$$

Calcula un error TD (temporal difference) en base a la recompensa de $t+1$ y la diferencia entre el valor de estado de $t+1$ y el valor del estado en t . Si $\delta_t > 0$ quiere decir que se ha mejorado la recompensa esperada para el nuevo estado, si $\delta_t < 0$ se ha empeorado. Por lo que el algoritmo, el actor, se actualizará y favorecerá acciones con TD positivo. Además, el critic ajustará sus estimaciones de $V(s, w)$ para hacerlas más precisas (Sutton & Barto, 2018)

El proceso de actualización se describe matemáticamente como:

1. $z_t^w = \lambda^w z_{t-1}^w + \nabla \hat{V}(S_t, w)$
2. $z_t^\theta = \lambda^\theta z_{t-1}^\theta + \nabla \ln \pi(A_t | S_t, \theta)$
3. $w \leftarrow w + \alpha^w \delta_t z_t^w$
4. $\theta \leftarrow \theta + \alpha^\theta \delta_t z_t^\theta$

z_t^w y z_t^θ son los vectores de trazas de elegibilidad, las trazas de elegibilidad son parámetros que regulan la memoria del modelo. Esto quiere decir que el valor de un estado o una política depende del valor pasado en una proporción $\lambda \in [0,1]$. Este concepto permite repartir las recompensas entre los estados y políticas anteriores a la actual asignándoles cierto grado de responsabilidad. De este modo existirán acciones que se ven recompensadas a largo plazo. El paso 3 es la actualización de parámetros del crítico, ajusta w para estimar mejor la función de valor, cuanto mayor error y mayor traza mayor corrección. Del mismo modo el paso 4 es la actualización del actor, ajusta θ para ajustar la política. La velocidad de ajuste de estos dos últimos pasos viene determinada por la tasa de aprendizaje α . (Sutton & Barto, 2018)

Modelos basados en actor critic son PPO (Proximal Policy Optimization) (Schulman, et al., 2017) DDPG (Deep deterministic Policy Gradient) (Silver, et al., 2014) (Silver, et al., 2016), TD3 (Fujimoto, et al., 2018) o SAC (Soft Actor Critic) (Haarnoja, et al., 2018)

Los métodos basados en actor-critic trabaja bien en entornos continuos, además reduce la varianza respecto a los métodos basados en políticas al introducir las funciones de valor y el aprendizaje por temporal difference. En contraparte se hace necesario que el agente critic sea preciso, adicionalmente el modelo se hace más pesado al tener que entrenar dos agentes actor y crítico. (Sutton & Barto, 2018)

¹ La notación de la función de valor del estado cambia de $V(s)$ a $\hat{V}(S, w)$ esto debido a que en entornos complejos y continuos que manejan los métodos actor-critic no es posible calcular directamente $V(s)$ por lo tanto se recurre a redes neuronales, el parámetro w se corresponde a los parámetros de la red neuronal

2.2.4. Revisión de literatura de reinforcement learning aplicado a la optimización de carteras

Esta tecnología ha despertado el interés de investigadores y académicos, en los últimos años se han desarrollado distintos enfoques en la aplicación del reinforcement learning a la optimización de carteras. Entre otros destacan:

Soleymani y Paquet (2020) con su modelo "DeepBreath" combinan deep reinforcement learning con una red neuronal convolucional para aprender la política de inversión. Además, aplican técnicas de denoising con autoencoder a la entrada de datos. Como resultado obtienen mejor desempeño respecto a los modelos tradicionales en cuanto a rentabilidad y riesgo.

Liang, *et al.* (2018) compara el rendimiento de tres algoritmos de aprendizaje profundo por refuerzo PG (Williams, 1992) DDPG (Silver, et al., 2016) y PPO (Schulman, et al., 2017). Para eliminar ruido emplean la técnica de adversarial deep reinforcement learning que consiste en introducir ruido artificial para que el modelo aprenda a lidiar con entornos ruidosos. Como resultados obtienen que el adversarial training es robusto en entornos con perturbaciones y que el algoritmo que presentó un mejor comportamiento fue PG en contraposición a DDPG y PPO que obtuvieron peores resultados debido al ruido.

Hieu (2020) compara el desempeño de diferentes algoritmos de deep reinforcement learning obteniendo como resultado que los algoritmos DDPG (Silver, et al., 2016) y GDPG (Cai, et al., 2018) superan significativamente a PPO (Schulman, et al., 2017), en especial en entornos volátiles.

Huang, *et al.* (2021) propone un enfoque basado en deep reinforcement learning al que se le añade adversarial learning, autoencoders para reducir ruido y una estrategia de muestreo que prioriza los entornos más relevantes para el aprendizaje. Como resultado obtienen que mediante esta estrategia de muestreo mejorar la ratio de sharpe y el drawdown. Adicionalmente concluyen que los modelos basados en actor-critic consiguen mejores resultados que los basados en políticas.

2.2.5. Desafíos del deep reinforcement learning

Por otro lado, hay autores que advierten de ciertos problemas que podrían darse en los modelos de deep reinforcement learning:

1. Sobreajuste u overfitting: esto es que el modelo se ajuste muy bien a los datos de entrenamiento aprendiendo incluso ruido y que después no pueda generalizar cuando se presenten datos nuevos.
2. Disponibilidad de datos: muchos modelos requieren de gran cantidad de datos que en ocasiones su disponibilidad se encuentra limitada
3. Sesgos: es posible que se puedan producir errores en la construcción del modelo y se introduzcan datos futuros en el entrenamiento generando un sesgo hacia delante. El modelo de manera accidental dispondría de información sobre cómo se comportará el futuro.
4. Capacidad de cómputo: para ciertos modelos es posible que se requiera gran capacidad de cómputo lo que hace inviable en la práctica ejecutar modelos en intervalos de tiempo cortos (Palomar, 2025)

De estos problemas hay dos, el overfitting y los sesgos en los que se tiene poder de acción. Existen técnicas para detectar overfitting, la primera de ellas y fundamental es separar datos de entrenamiento y datos de test. De este modo el modelo se entrena con un set de datos y aplica lo aprendido a un conjunto de datos test con los que no se ha entrenado. Aun así esto puede no ser suficiente ya que es posible que la estructura de los datos de test sea similar a los de entrenamiento. Por ejemplo, en un contexto financiero, el overfitting podría darse en una situación en la que los datos de entrenamiento hayan sido en su mayoría de un periodo alcista por lo que el modelo se ajustará muy bien a un set de datos de test que pertenezca a un periodo de mercado alcista. Sin embargo, cuando se presenta un set de datos de test de un periodo bajista el modelo

tendrá un desempeño deficiente. Para evitar esto debemos asegurarnos de que el set de entrenamiento sea lo suficientemente amplio como para tener suficiente variedad de contextos económicos. Adicionalmente la validación debe darse no solo con un set de datos de test, el test debe darse en ventanas a lo largo de la cronología de la serie temporal, lo que se llama validación walk-forward.

Por otra parte, modelos demasiado complejos con sets de entrenamiento insuficientes pueden comenzar a memorizar ruido lo que dificultará la generalización en datos nuevos. Este problema en parte se puede mitigar con técnicas de denoising con autoencoders o con redes extractoras de características

Otras técnicas implementadas son Dropout (Srivastava, et al., 2014) que consiste en inhabilitar neuronas de manera temporal en una red neuronal de manera que no exista dependencia excesiva de ningún patrón. Weight Decay (Goodfellow, et al., 2016) técnica que consiste en penalizar en la función de pérdida la asignación excesiva de pesos a una neurona en una red neuronal. Y por último early stopping (Caruana, et al., 2000) que se trata de detener el entrenamiento del modelo antes de que empiece a memorizar ruido una vez que la función de pérdida deja de disminuir.

Considerando otra dimensión del problema, la elección del algoritmo de reinforcement learning juega un gran papel en el problema del overfitting. Modelos con políticas deterministas son por naturaleza menos exploradores que los modelos con políticas estocásticas. No generan distintas trayectorias para cada estado lo que favorece el overfitting.

3. Metodología

3.1. Objetivo del modelo

El objetivo del modelo será el de maximizar el retorno del portafolio. No se tendrá en cuenta el riesgo. El benchmark será el índice S&P500. El algoritmo tratará de superar en rendimiento al S&P500.

3.2. Datos utilizados

La fuente de datos empleada para los activos será Yahoo Finance (<https://es.finance.yahoo.com/>). En concreto como se usarán activos con origen en Estados Unidos, Yahoo Finance indica que los datos son proporcionados por Commodity Systems Inc. Se ha elegido esta fuente puesto que cuenta con una librería integrada en Python lo que facilita el trabajo significativamente.

Los activos seleccionados para diseñar el modelo son, las quinientas acciones del índice S&P500 a marzo de 2025, tres fondos cotizados (ETF) de bonos del tesoro estadounidense, en concreto, TLT (bonos largo plazo, más de 20 años), IEF (bonos medio plazo, 7-10 años) y SHY (bonos corto plazo, 1-3años). De manera adicional se permite mantener hasta un 20% del valor de la cartera en efectivo.

Por simplicidad se han escogido activos exclusivamente del mercado estadounidense, sin embargo, es posible aumentar el abanico de posibilidades. Incluir activos de otras regiones requeriría normalizar los datos por cambios en los días de apertura de la bolsa y horarios de cierre y apertura.

El modelo utiliza datos de cotización diarios cuyo horizonte temporal abarca desde el 3 de enero de 2012 al 25 de abril de 2025. Para comenzar a trabajar con los datos se hace necesario eliminar aquellas empresas que presentan datos vacíos ya sea por comenzar a cotizar después de 2012 o por presentar datos faltantes en la serie. Se descartan acciones que empezaron después de 2012: ['ABBV', 'ABNB', 'ALLE', 'AMCR', 'ANET', 'CARR', 'CDW', 'CFG', 'CTVA', 'CZR', 'DOW', 'ENPH', 'EPAM', 'ETSY', 'FANG', 'FOX', 'FOXA', 'FTV', 'GEHC', 'HLT', 'HPE', 'HWM', 'IQV', 'IR', 'KEYS', 'KHC', 'LW', 'META', 'MRNA', 'NCLH', 'NOW', 'NWS', 'NWSA', 'OGN', 'OTIS', 'PAYC', 'PSX', 'PYPL', 'QRVO', 'SEDG', 'SYF', 'UA']

Se descartan por datos vacíos ['BRK.B', 'CDAY', 'FISV', 'XLNX', 'FLT', 'CTLT', 'CERN', 'DISCA', 'RE', 'WLTW', 'VIAC', 'WRK', 'PXD', 'DRE', 'XEC', 'TPX', 'VAR', 'SGEN', 'FRC', 'FLIR', 'PKI', 'PEAK', 'DISH', 'TSS', 'TIF', 'DISCK', 'MRO', 'CTXS', 'SIVB', 'TWTR']

En ciertos modelos nos encontramos con problemas de dimensionalidad. El modelo se comporta de manera pobre en entornos de alta dimensión. El principal componente que agrega un exceso de dimensión al modelo es el número de acciones. Trabajar con alrededor de 400-500 acciones se hace inviable según el algoritmo implicado. Por ello cuando nos encontremos ante esta situación se aplicarán filtros. Si esto no es suficiente es posible reducir dimensionalidad aplicando autoencoders.

El primer filtrado de acciones será descartar por capitalización de mercado. Los valores con menor volumen de negociación tienden a ser series con alto ruido, mucha volatilidad y gaps en la cotización.

El segundo será agrupar acciones de similares características mediante un clustering. Se escogerán las acciones más representativas de cada clúster para construir la cartera. De este modo se consigue prescindir de un alto número de valores sin rechazar la diversidad entre ellos. Por ejemplo, si se aplicara un filtro donde solo se toman las 50 acciones de mayor capitalización, nos arriesgamos a que compartan características muy parecidas comprometiendo la diversificación.

3.3. Variables explicativas

Sería especialmente útil evaluar el rendimiento marginal de incluir cada variable en el modelo, además aportaría un mayor fundamento y robustez en la elección de variables. Sin embargo, como se comentó al principio, se hace evidente la limitación de hardware, evaluar la inclusión de cada variable requiere la ejecución de decenas de veces el programa, lo que consume días de cómputo.

3.3.1. Indicadores técnicos

Se han elegido indicadores empleados frecuentemente en el análisis técnico y se han implementado mediante la librería de Python TaLib. Estos indicadores amplían la información de precios permitiendo obtener información sobre el estado de la serie temporal en cada momento. Se han elegido dos indicadores de momentum y dos indicadores de tendencia con el objetivo de que el modelo combine las señales de cada uno de ellos. Por otra parte no se ha incluido ningún indicador de volatilidad pues como se muestra más adelante, se incluye la desviación típica calculada en ventanas de 50 días y el índice VIX.

3.3.1.1. Indicadores de momentum:

Relative Strenght Index: Mide la relación entre las ganancias y las pérdidas de un periodo de la siguiente forma

$$RS = \frac{\text{Media ganancias en } n}{\text{Media pérdidas en } n}$$

$$RSI = 100 - \frac{100}{1 + RS}$$

(Ghobadi, 2014)

Oscilador estocástico: Compara el precio de cierre de una vela C_t con el máximo H_n y mínimo L_n de un periodo n . Se calcula como

$$\%K_t = \frac{C_t - L_n}{H_n - L_n} \times 100$$

$$\%D_t = SMA_m(\%K_t)$$

(Ghobadi, 2014)

3.3.1.2. Indicadores de tendencia

Moving Average Convergence Divergence (MACD): mide la convergencia o divergencia entre dos medias móviles exponenciales (EMA). Cálculo:

$$MACD_t = EMA_{rápida}(P_t, n_{rápida}) - EMA_{lenta}(P_t, n_{lenta})$$

$$SEÑAL_t = EMA(MACD_t, n_{señal})$$

(Anghel, 2015)

Average Directional Index: mide la fuerza de la tendencia

$$+DM = Máximo_t - Máximo_{t-1}$$

$$-DM = Mínimo_t - Mínimo_{t-1}$$

Estas son las definiciones de la dirección del movimiento. Si ambos valores son positivos prevalece el mayor valor mientras que al menor se le asigna el valor cero.

$$TR = MAX(Máximo_t - Mínimo_t, Máximo_t - Cierre_{t-1}, Mínimo_t - Cierre_{t-1})$$

True Range (TR) se usa para normalizar la dirección del movimiento

A continuación, se calcula una media móvil suavizada de TR. Primero se calcula una media aritmética de los primeros n periodos

$$TR_{SMA,1} = \sum_{k=1}^N -DM_k$$

$$TR_{SMA,i} = TR_{SMA,i-1} - \frac{TR_{SMA,i-1}}{N} + TR_i$$

Del mismo modo se calcula la media móvil suavizada $+DM_{SMA,i}$ y $-DM_{SMA,i}$.

$$+DI = 100 * \frac{+DM_{SMA,i}}{TR_{SMA,i}}$$

$$-DI = 100 * \frac{-DM_{SMA,i}}{TR_{SMA,i}}$$

Con estos valores se calcula el índice direccional

$$DX_i = 100 * \frac{|+DI_i - -DI_i|}{|+DI_i + -DI_i|}$$

Esto mide la diferencia entre la presión alcista y bajista.

Por último, se construye el indicador final ADX, que es una media de DX_i

$$\text{Primer ADX} = \frac{1}{N} \sum_{k=1}^N DX_k$$

$$ADX_i = \frac{(ADX_{i-1}(N-1)) + DX_i}{N}$$

(Potters, 2024)

3.3.2. Variables macroeconómicas

Incluir variables macroeconómicas supone un reto ya que estamos limitados a variables que se publiquen con frecuencia diaria. Las variables macroeconómicas típicas se publican mensual o trimestralmente. Por ello hay que recurrir a variables proxy que reflejen en cierta medida el estado macroeconómico.

Tipos de interés de los bonos del tesoro de Estados Unidos.

Tabla 7 Variables tipos de interés de bonos del tesoro de EE.UU.

DGS3MO	Rendimiento del bono a 3 meses
DGS1	Rendimiento del bono a 1 año
DGS2	Rendimiento del bono a 2 años
DGS5	Rendimiento del bono a 5 años
DGS7	Rendimiento del bono a 7 años
DGS10	Rendimiento del bono a 10 años

Estos bonos incluyen información sobre el riesgo sistémico del país y las actuaciones y expectativas sobre política monetaria e inflación.

Además, se añadirán los rendimientos de los bonos corporativos y su spread respecto a los bonos del tesoro a 10 años

Tabla 8 Variables tipos de interés bonos corporativos

Moody's BAA	Rendimiento de bonos a más de 10 años de vencimiento con calificación BAA
Moody's AAA	Rendimiento de bonos a más de 10 años de vencimiento con calificación AAA
Spread BAA	Diferencial de rendimiento respecto al bono del tesoro a 10 años
Spread AAA	Diferencial de rendimiento respecto al bono del tesoro a 10 años

El rendimiento de estos bonos capta el riesgo de invertir en empresas de estas categorías de calificación crediticia.

Índice VIX, se construye mediante el promedio de precios ponderados de opciones sobre el S&P500. Es un indicador de volatilidad, aporta información sobre el sentimiento del mercado.

CBOE SKEW Index, mide la asimetría en el precio de las opciones sobre el S&P500. En concreto mide la diferencia de precio entre las puts OTM y las call OTM. Se interpreta como la prima que se paga como protección ante caídas severas del índice. Este índice sirve de complemento al VIX.

Tipo de cambio, se empleará el tipo de cambio EUR/USD, CNY/USD, MXN/USD y CAD/USD. Los principales socios comerciales de EE.UU., Canadá, U.E., México y China (Fuente OEC). Esta variable aporta información sobre los flujos de capitales además de que es altamente sensible para las empresas exportadoras e importadoras. Del mismo modo el tipo de cambio tiene una fuerte relación macroeconómica con los tipos de interés por lo que se abre la posibilidad a modelar estas relaciones.

Por último, se añade la cotización de dos commodities, se incorpora la cotización del oro que es utilizado como activo refugio y puede dar señales macroeconómicas útiles (Baur & McDermott, 2010). Y, además, la cotización del barril de petróleo West Texas Intermediate (WTI) ya que algunos estudios sugieren que guarda relación con índices bursátiles. (Allen & McAleer, 2020)

3.3.3. Otras variables

Por otro lado, con el fin de revelar las características particulares de cada activo y aportar información útil para lograr el fin de superar al S&P500, se han añadidos, calculadas en ventanas de 50 días, las siguientes variables:

Retorno medio, medido como la diferencia logarítmica entre t y $t-1$.

Volatilidad medida como la desviación típica

CVaR, como la media de los peores 5% de los resultados en esa ventana.

Asimetría en las colas de distribución de rendimientos logarítmicos

Correlación respecto al S&P500

Coeficiente beta respecto al S&P500, medido como $\beta = \frac{COV(X, SP500)}{VAR(SP500)}$

Ratio de Sharpe $S = \frac{\text{Retorno medio}}{\text{volatilidad media}}$ suponiendo tasa libre de riesgo 0.

Tracking Error $TE = \sqrt{VAR(\text{retorno medio} - \text{retorno medio SP500})}$

Ratio de información $IR = \frac{(\text{Rentabilidad media} - \text{Rentabilidad media SP500})}{TE}$

Adicionalmente, mediante un clustering basado en las variables anteriores se etiqueta a las acciones según el régimen de mercado en el que se encuentre de modo que se complementa la información anterior.

Del cálculo de las métricas anteriores se obtienen para cada día los valores de las variables. Sobre esos días y esas variables se aplica el clustering.

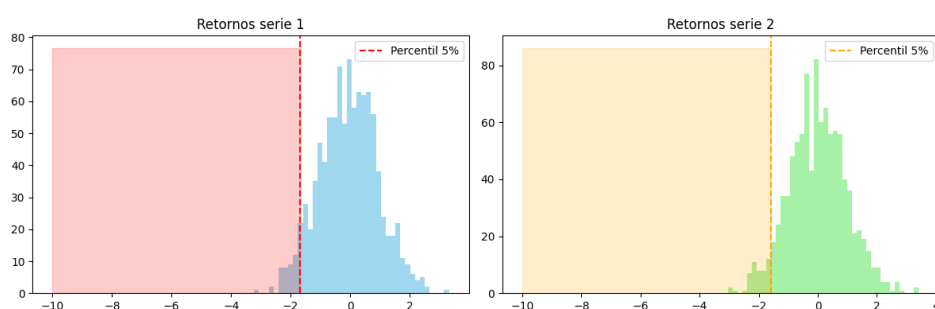
Cada categoría resultante del clustering se puede interpretar como un régimen de mercado. Entonces se crean 5 variables dummies para cada activo y según en el régimen de mercado en el que se encuentren la variable tomara el valor 0 o 1.

Sumado a esto, se ha añadido otra variable dummy para los días de presentación de resultados de cada empresa correspondiente a cada acción en la cartera.

Por último, se ha incluido como variable el comportamiento de cada activo en la cola de distribución de rendimientos con p-valor 5%. Esta variable se define como el número de veces que el activo n se encuentra en la cola de distribución a la vez que el S&P500 a lo largo de un año.

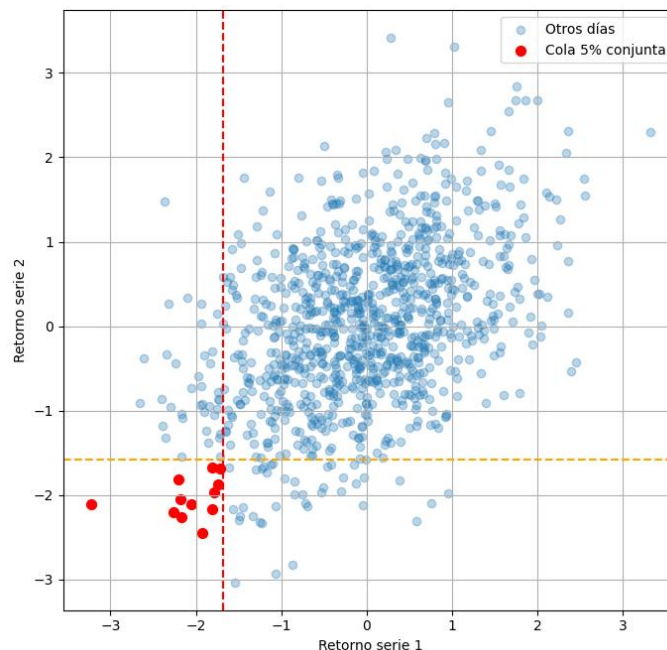
Esta última variable aporta información sobre comovimientos extremos entre el activo y el S&P500. Cuantas más veces se dé la condición, más propenso es el activo de replicar los movimientos extremos del índice. De manera ilustrativa:

Gráfico 3 Ejemplo distribución de retornos



Fuente: Elaboración propia

Gráfico 4 Ejemplo cola conjunta



Fuente: Elaboración propia

Cuantas más veces los rendimientos tanto del activo como del S&P500 se encuentren al mismo tiempo por debajo de sus respectivas colas al 5%, más propenso es el activo a comovimientos extremos a la baja. Lo mismo aplica para el caso alcista.

Se calcula la proporción de días en los que el activo y el índice han estado conjuntamente a la vez en sus respectivas colas en la ventana de 252 días. Esta es la variable que se añade.

3.3.4. Análisis correlación entre variables

Nuestro modelo no resuelve ningún problema de predicción, por lo que el análisis de las variables explicativas no se debe centrar en la capacidad de predicción sobre una variable concreta.

Para lograr maximizar la función de recompensa el modelo deberá maximizar el retorno del portafolio, superar al retorno del S&P500 y no sufrir pérdidas significativas ni drawdowns fuertes. Se entiende entonces que deberá modelar correctamente el comportamiento de los retornos de las acciones, el S&P500, el CVaR y los drawdowns. En el escenario más ideal se construiría la matriz de correlaciones de cada acción, sin embargo, se hace imposible representar una matriz de tales dimensiones. Por sencillez y practicidad la matriz de correlaciones se construirá tomando como base el S&P500. Se asume entonces que estas variables se comportaran de manera similar para cada uno de los activos de la cartera.

Tabla 9 Matriz de correlación Pearson variables explicativas

ret	1.00	0.13	0.03	0.02	0.06	0.14	0.35	0.45	0.18	-0.03	-0.02	0.01	-0.01	0.01	-0.00	-0.06	-0.00	-0.00	-0.01	-0.01	-0.02	-0.02	-0.01	-0.02	0.01	0.00	-0.00	0.02
ret50	-0.13	1.00	0.01	0.31	0.21	0.96	0.65	0.35	0.07	-0.10	-0.06	0.10	-0.07	0.09	0.08	-0.18	-0.00	-0.02	-0.05	-0.10	-0.12	-0.12	-0.22	-0.19	-0.10	-0.08	0.03	0.13
vol50	-0.03	0.01	1.00	-0.86	0.14	0.01	0.10	0.10	0.11	-0.22	0.13	-0.18	-0.20	-0.21	-0.21	0.27	-0.29	-0.24	-0.23	-0.24	-0.26	-0.29	-0.10	-0.22	0.40	0.30	-0.39	-0.23
CVaR95_50	-0.02	0.31	-0.86	1.00	0.23	0.29	0.11	0.02	-0.07	0.16	-0.12	0.15	0.17	0.21	0.15	-0.33	0.31	0.28	0.27	0.26	0.26	0.28	0.10	0.19	-0.39	-0.34	0.39	0.23
skew50	-0.06	0.21	0.14	0.23	1.00	0.22	0.17	0.10	0.07	-0.08	-0.12	-0.14	-0.13	0.08	0.01	-0.13	0.16	0.16	0.16	0.14	0.12	0.11	0.06	0.04	-0.12	-0.19	-0.09	0.07
sharpe50	0.14	0.96	0.01	0.29	0.22	1.00	0.69	0.38	0.10	-0.07	-0.04	0.11	-0.07	0.07	0.07	-0.16	-0.02	-0.04	-0.07	-0.13	-0.14	-0.15	-0.24	-0.22	-0.09	-0.07	0.03	0.12
RSI	0.35	0.65	0.10	0.11	0.17	0.69	1.00	0.83	0.66	-0.15	-0.03	0.09	-0.03	0.05	0.03	-0.15	-0.03	-0.04	-0.06	-0.11	-0.11	-0.12	-0.15	-0.16	-0.02	-0.02	0.03	0.10
Stoch_%K	0.45	0.35	0.10	0.02	0.10	0.38	0.83	1.00	0.77	-0.12	-0.03	0.04	-0.00	0.03	0.01	-0.10	-0.01	-0.02	-0.03	-0.06	-0.06	-0.06	-0.06	-0.07	0.03	0.01	0.03	0.08
MACD	0.18	0.07	0.11	-0.07	0.07	0.10	0.66	0.77	1.00	-0.12	-0.02	0.00	0.01	0.01	-0.03	-0.08	0.01	0.00	-0.00	-0.01	-0.01	-0.00	0.03	0.00	0.05	0.02	0.00	0.03
ADX	-0.03	-0.10	-0.22	0.16	-0.08	-0.07	-0.15	-0.12	-0.12	1.00	-0.17	-0.10	-0.07	0.09	-0.03	0.19	0.15	0.15	0.15	0.15	0.14	0.14	0.14	0.14	-0.05	-0.06	-0.03	0.08
VIX	-0.02	-0.06	0.13	-0.12	-0.12	-0.04	-0.03	-0.03	-0.02	-0.17	1.00	0.74	0.67	-0.66	-0.30	-0.19	-0.75	-0.74	-0.71	-0.59	-0.52	-0.43	-0.26	-0.25	0.43	0.60	0.30	-0.56
SKEW	0.01	0.10	-0.18	0.15	-0.14	0.11	0.09	0.04	-0.00	-0.10	0.74	1.00	0.74	0.55	-0.19	-0.21	-0.53	-0.58	-0.59	-0.50	-0.43	-0.31	-0.22	-0.17	0.26	0.45	0.44	-0.30
EURUSD	-0.01	-0.07	-0.20	0.17	-0.13	-0.07	-0.03	-0.00	0.01	-0.07	0.67	0.74	1.00	-0.60	-0.34	-0.39	-0.29	-0.33	-0.33	-0.21	-0.12	0.02	0.25	0.25	0.35	0.47	0.56	-0.31
CNYUSD	0.01	0.09	-0.21	0.21	0.08	0.07	0.05	0.03	0.01	0.09	-0.66	-0.55	-0.60	1.00	0.63	0.17	0.70	0.69	0.68	0.63	0.58	0.51	0.14	0.25	-0.76	-0.80	0.06	0.84
MXNUSD	-0.00	0.08	-0.21	0.15	0.01	0.07	0.03	0.01	-0.03	-0.03	-0.30	-0.19	-0.34	0.63	1.00	-0.14	0.30	0.26	0.27	0.28	0.27	0.25	-0.05	0.08	-0.56	-0.47	-0.07	0.50
CADUSD	-0.06	-0.18	0.27	-0.33	-0.13	-0.16	-0.15	-0.10	-0.08	0.19	-0.19	-0.21	-0.39	0.17	-0.14	1.00	-0.05	-0.03	-0.03	-0.12	-0.17	-0.23	-0.13	-0.24	0.25	0.10	-0.19	0.24
DG53MO	-0.00	-0.00	-0.29	0.31	0.16	-0.02	-0.03	-0.01	0.01	0.15	-0.75	-0.53	-0.29	0.70	0.30	-0.05	1.00	0.99	0.97	0.92	0.89	0.85	0.65	0.68	-0.64	-0.80	0.14	0.62
DG51	-0.00	-0.02	-0.24	0.28	0.16	-0.04	-0.04	-0.02	0.00	0.15	-0.74	-0.58	-0.33	0.69	0.26	-0.03	0.99	1.00	0.99	0.95	0.91	0.86	0.67	0.69	-0.63	-0.82	0.16	0.57
DG52	-0.01	-0.05	-0.23	0.27	0.16	-0.07	-0.06	-0.03	-0.00	0.15	-0.71	-0.59	-0.33	0.68	0.27	-0.03	0.97	0.99	1.00	0.97	0.94	0.89	0.69	0.72	-0.65	-0.82	0.18	0.54
DG55	-0.01	-0.10	-0.24	0.26	0.14	-0.13	-0.11	-0.06	-0.01	0.15	-0.59	-0.50	-0.21	0.63	0.28	-0.12	0.92	0.95	0.97	1.00	0.99	0.96	0.77	0.83	-0.67	-0.79	0.26	0.47
DG57	-0.02	-0.12	-0.26	0.26	0.12	-0.14	-0.11	-0.06	-0.01	0.14	-0.52	-0.43	-0.12	0.58	0.27	-0.17	0.89	0.91	0.94	0.99	1.00	0.99	0.81	0.88	-0.66	-0.75	0.32	0.45
DG510	-0.02	-0.12	-0.29	0.28	0.11	-0.15	-0.12	-0.06	-0.00	0.14	-0.43	-0.31	0.02	0.51	0.25	-0.23	0.85	0.86	0.89	0.96	0.99	1.00	0.86	0.93	-0.61	-0.67	0.40	0.43
DBAA	-0.01	-0.22	-0.10	0.10	0.06	-0.24	-0.15	-0.06	0.03	0.14	-0.26	-0.22	0.25	0.14	-0.05	-0.13	0.65	0.67	0.69	0.77	0.81	0.86	1.00	0.96	-0.11	-0.26	0.34	0.21
DAAA	-0.02	-0.19	-0.22	0.19	0.04	-0.22	-0.16	-0.07	0.00	0.14	-0.25	-0.17	0.25	0.25	0.08	-0.24	0.68	0.69	0.72	0.83	0.88	0.93	0.96	1.00	-0.33	-0.36	0.42	0.28
BAA10Y	0.01	-0.10	0.40	-0.39	-0.12	-0.09	-0.02	0.03	0.05	-0.05	0.43	0.26	0.35	-0.76	-0.56	0.25	-0.64	-0.63	-0.65	-0.67	-0.66	-0.61	-0.11	-0.33	1.00	0.90	-0.25	-0.51
AAA10Y	0.00	-0.08	0.30	-0.34	-0.19	-0.07	-0.02	0.01	0.02	-0.06	0.60	0.45	0.47	-0.80	-0.47	0.10	-0.80	-0.82	-0.82	-0.79	-0.75	-0.67	-0.26	-0.36	0.90	1.00	-0.17	-0.55
gold	-0.00	0.03	-0.39	0.39	-0.09	0.03	0.03	0.03	0.00	-0.03	0.30	0.44	0.56	0.06	-0.07	-0.19	0.14	0.16	0.18	0.26	0.32	0.40	0.34	0.42	-0.25	-0.17	1.00	0.20
wti	0.02	0.13	-0.23	0.23	0.07	0.12	0.10	0.08	0.03	0.08	-0.56	-0.30	-0.31	0.84	0.50	0.24	0.62	0.57	0.54	0.47	0.45	0.43	0.21	0.28	-0.51	-0.55	0.20	1.00
ret	ret50	vol50	CVaR95_50	skew50	sharpe50	RSI	Stoch_%K	MACD	ADX	VIX	SKEW	EURUSD	CNYUSD	MXNUSD	CADUSD	DG53MO	DG51	DG52	DG55	DG57	DG510	DBAA	DAAA	BAA10Y	AAA10Y	gold	wti	

Fuente: Yahoo Finance, FED, elaboración propia

Tabla 10 Matriz de correlación de Spearman variables explicativas

ret	1.00	0.13	0.02	0.01	0.01	0.13	0.35	0.50	0.18	-0.02	-0.02	0.02	-0.00	0.01	0.00	-0.04	-0.01	-0.02	-0.03	-0.03	-0.03	-0.03	-0.02	-0.03	0.01	0.01	0.01	0.03
ret50	0.13	1.00	-0.04	0.27	0.21	0.99	0.63	0.35	0.05	-0.13	-0.06	0.15	-0.08	0.12	0.14	-0.09	0.01	-0.08	-0.12	-0.15	-0.15	-0.16	-0.22	-0.21	-0.09	-0.06	-0.02	0.20
vol50	-0.02	-0.04	1.00	-0.87	0.15	-0.02	0.08	0.09	0.10	-0.21	0.15	-0.10	-0.18	-0.19	-0.22	0.27	-0.22	-0.14	-0.19	-0.22	-0.21	-0.22	-0.07	-0.19	0.38	0.28	-0.37	-0.30
CVaR95_50	0.01	0.27	-0.87	1.00	0.21	0.26	0.09	0.02	-0.08	0.17	-0.14	0.12	0.17	0.21	0.18	-0.24	0.25	0.19	0.21	0.21	0.18	0.20	0.04	0.12	-0.39	-0.32	0.36	0.32
skew50	0.01	0.21	0.15	0.21	1.00	0.22	0.17	0.10	0.04	-0.10	-0.09	-0.01	-0.03	0.05	-0.04	-0.01	0.15	0.18	0.15	0.10	0.07	0.07	0.04	-0.01	-0.11	-0.14	-0.09	-0.03
sharpe50	0.13	0.99	-0.02	0.26	0.22	1.00	0.65	0.36	0.06	-0.11	-0.05	0.15	-0.08	0.11	0.13	-0.08	-0.01	-0.09	-0.13	-0.16	-0.16	-0.17	-0.22	-0.22	-0.07	-0.05	-0.02	0.19
RSI	0.35	0.63	0.08	0.09	0.17	0.65	1.00	0.85	0.66	-0.20	-0.03	0.13	-0.03	0.06	0.05	-0.10	-0.02	-0.09	-0.12	-0.14	-0.13	-0.12	-0.13	-0.15	-0.00	0.00	0.01	0.15
Stoch_%K	0.50	0.35	0.09	0.02	0.10	0.36	0.85	1.00	0.80	-0.13	-0.03	0.07	-0.00	0.03	0.02	-0.10	-0.02	-0.06	-0.08	-0.08	-0.08	-0.07	-0.06	-0.08	0.02	0.02	0.02	0.09
MACD	0.18	0.05	0.10	-0.08	0.04	0.06	0.66	0.80	1.00	-0.13	-0.02	-0.01	0.02	-0.00	-0.03	-0.07	-0.02	-0.02	-0.03	-0.03	-0.02	-0.01	0.02	-0.01	0.06	0.04	0.00	0.01
ADX	-0.02	-0.13	-0.21	0.17	-0.10	-0.11	-0.20	-0.13	-0.13	1.00	-0.13	-0.10	-0.04	0.07	-0.03	0.09	0.16	0.18	0.18	0.15	0.14	0.13	0.14	0.14	-0.07	-0.06	0.01	0.05
VIX	-0.02	-0.06	0.15	-0.14	-0.09	-0.05	-0.03	-0.03	-0.02	-0.13	1.00	0.73	0.56	-0.71	-0.29	-0.14	-0.81	-0.72	-0.63	-0.49	-0.42	-0.33	-0.26	-0.22	0.47	0.61	0.27	-0.50
SKREW	0.02	0.15	-0.10	0.12	-0.01	0.15	0.13	0.07	-0.01	-0.10	0.73	1.00	0.45	-0.57	-0.12	-0.13	-0.64	-0.72	-0.68	-0.56	-0.48	-0.38	-0.37	-0.32	0.29	0.45	0.26	-0.24
EURUSD	-0.00	-0.08	-0.18	0.17	-0.03	-0.08	-0.03	-0.00	0.02	-0.04	0.56	0.45	1.00	-0.66	-0.29	-0.46	-0.30	-0.18	-0.12	-0.04	0.03	0.16	0.32	0.32	0.33	0.39	0.40	-0.34
CNYUSD	0.01	0.12	-0.19	0.21	0.05	0.11	0.06	0.03	-0.00	0.07	-0.71	-0.57	-0.66	1.00	0.56	0.25	0.60	0.53	0.53	0.49	0.42	0.32	0.09	0.13	-0.79	-0.83	-0.02	0.72
MXNUSD	0.00	-0.04	-0.22	0.18	-0.04	0.13	0.05	0.02	-0.03	-0.03	-0.29	-0.12	-0.29	0.56	1.00	-0.12	0.18	0.10	0.15	0.20	0.18	0.13	-0.07	0.02	-0.57	-0.46	-0.10	0.34
CADUSD	-0.04	-0.09	0.27	-0.24	-0.01	-0.08	-0.10	-0.10	-0.07	0.09	-0.14	-0.13	-0.46	0.25	-0.12	1.00	0.02	-0.03	-0.11	-0.18	-0.19	-0.20	-0.05	-0.19	0.11	-0.03	-0.08	0.37
DG53MO	-0.01	0.01	-0.22	0.25	0.15	-0.01	-0.02	-0.02	-0.02	0.16	-0.81	-0.64	-0.30	0.60	0.18	0.02	1.00	0.93	0.86	0.74	0.69	0.62	0.48	0.45	-0.56	-0.73	0.01	0.44
DG51	-0.02	-0.08	-0.14	0.19	0.18	-0.09	-0.09	-0.06	-0.02	0.18	-0.72	-0.72	-0.18	0.53	0.10	-0.03	0.93	1.00	0.96	0.85	0.79	0.72	0.60	0.56	-0.52	-0.69	0.01	0.28
DG52	-0.03	-0.12	-0.19	0.21	0.15	-0.13	-0.12	-0.08	-0.03	0.18	-0.63	-0.66	-0.12	0.53	0.15	-0.11	0.86	0.96	1.00	0.94	0.89	0.82	0.64	0.66	-0.61	-0.74	0.12	0.22
DG55	-0.03	-0.15	-0.22	0.21	0.10	-0.16	-0.14	-0.08	-0.03	0.15	-0.49	-0.56	-0.04	0.49	0.20	-0.18	0.74	0.85	0.94	1.00	0.99	0.94	0.71	0.79	-0.67	-0.75	0.28	0.19
DG57	-0.03	-0.15	-0.21	0.18	0.07	-0.16	-0.13	-0.08	-0.02	0.14	-0.42	-0.46	0.03	0.42	0.18	-0.19	0.69	0.79	0.89	0.99	1.00	0.98	0.77	0.86	-0.63	-0.71	0.34	0.17
DG510	-0.03	-0.16	-0.22	0.20	0.07	-0.17	-0.12	-0.07	-0.01	0.13	-0.33	-0.38	0.16	0.32	0.13	-0.20	0.62	0.72	0.82	0.94	0.98	1.00	0.84	0.92	-0.56	-0.63	0.43	0.17
DBAA	-0.02	-0.22	-0.07	0.04	0.04	-0.22	-0.13	-0.06	0.02	0.14	-0.26	-0.37	0.32	0.09	-0.07	-0.05	0.48	0.60	0.64	0.71	0.77	0.84	1.00	0.94	-0.15	-0.28	0.37	0.13
DAAA	-0.03	-0.21	-0.19	0.12	-0.01	-0.22	-0.15	-0.08	-0.01	0.14	-0.22	-0.32	0.32	0.13	0.02	-0.19	0.45	0.56	0.66	0.79	0.86	0.92	0.94	1.00	-0.32	-0.36	0.46	0.12
BAA10Y	0.01	-0.09	0.38	-0.39	-0.11	-0.07	-0.00	0.02	0.06	-0.07	0.47	0.29	0.33	-0.79	-0.57	0.11	-0.56	-0.52	-0.61	-0.67	-0.63	-0.56	-0.15	-0.32	1.00	0.92	-0.25	-0.51
AAA10Y	0.01	-0.06	0.28	-0.32	-0.14	-0.05	0.00	0.02	0.04	-0.06	0.61	0.45	0.39	-0.83	-0.46	-0.03	-0.73	-0.69	-0.74	-0.75	-0.71	-0.63	-0.28	-0.36	0.92	1.00	-0.19	-0.54
gold	0.01	-0.02	-0.37	0.36	-0.09	-0.02	0.01	0.02	0.00	0.01	0.27	0.26	0.40	-0.02	-0.10	-0.08	0.01	0.01	0.12	0.28	0.34	0.43	0.37	0.46	-0.25	-0.19	1.00	0.31
wti	0.03	0.20	-0.30	0.32	-0.03	0.19	0.15	0.09	0.01	0.05	-0.50	-0.24	-0.34	0.72	0.34	0.37	0.44	0.28	0.22	0.19	0.17	0.17	0.13	0.12	-0.51	-0.54	0.31	1.00
ret	ret50	vol50	CVaR95_50	skew50	sharpe50	RSI	Stoch_%K	MACD	ADX	VIX	SKREW	EURUSD	CNYUSD	MXNUSD	CADUSD	DG53MO	DG51	DG52	DG55	DG57	DG510	DBAA	DAAA	BAA10Y	AAA10Y	gold	wti	

En ambas matrices de correlaciones, tanto de Pearson como de Spearman, las variables tienen poder explicativo sobre las variables a explicar, en principio deben añadir información útil. Por otro lado, al incluir tantas variables explicativas se da el fenómeno de la multicolinealidad, existen variables explicativas muy correladas entre sí. Esto quiere decir que no aportan un valor añadido significativo al poder explicativo. Para solucionar esto tradicionalmente se eliminan las variables que presentan multicolinealidad. Sin embargo, se puede tomar otro enfoque, en el contexto de este trabajo, deep learning, la tarea de tratar con multicolinealidad se delegará a los extractores de características y las redes actor y critic. Estas redes neuronales presentan la capacidad de extraer la información útil del set de datos al combinar los inputs en sus neuronas.

3.3.5. Análisis cointegración

La cointegración indica si dos o más variables mantienen una relación de equilibrio a largo plazo. Es útil porque nos aporta información de cuál es el comportamiento esperado de las variables a largo plazo. Si se encuentran en desequilibrio se esperará que las variables se muevan hacia el equilibrio en el futuro. Para ello es fundamental que las series sean no estacionarias.

El análisis de cointegración se realizará entre las variables macroeconómicas. Sería especialmente enriquecedor para el modelo realizar el análisis sobre las variables explicativas de cada activo, sin embargo, en un universo de acción tan grande requeriría hacer el análisis decenas de veces, una para cada activo. Este aspecto es una posible mejora a añadir en el futuro.

Tabla 11 Constraste Aumented Dickey Fuller

	DGS3MO	DGS1	DGS2	DGS5	DGS7	DGS10	DBAA	DAAA	VIX	SKE W	ORO	WTI	S&P50 0	EUR USD	CNY USD	MXN USD	CAD USD
Estacionariada	No	No	No	No	No	No	No	No	Si	SI	No	No	No	No	No	No	No

Fuente Yahoo finance, FED, elaboración propia.

El contraste de Dickey Fuller indica cuales series son no estacionarias (Gujarati & Porter, 2013). Una vez sabemos cuáles series cumplen esta condición se analiza cuales conforman vectores cointegrantes. Para ello se emplea el contraste de Johansen. (Dwyer, 2015)

Este contraste está limitado a 12 variables, por ello primero se realizará con las variables DGS3MO, DGS1, DGS2, DGS5, DGS10, DBAA, DAAA, S&P500, ORO, WTI.

Primero hay que seleccionar el orden de retardos del modelo, esto se hace con los criterios de información AIC, BIC y HQC. (Gujarati & Porter, 2013)

Sistema VAR, máximo orden de retardos 16

Los asteriscos de abajo indican los mejores (es decir, los mínimos) valores de cada criterio de información, AIC = criterio de Akaike, BIC = criterio bayesiano de Schwarz y HQC = criterio de Hannan-Quinn.
retardos log.veros p(RV) AIC BIC HQC

1	44627.80795		-32.675832	-32.337176	-32.553415
2	45495.26671	0.00000	-33.207397	-32.556134*	-32.971978*
3	45753.25663	0.00000	-33.291151	-32.327283	-32.942731
4	45986.31202	0.00000	-33.356585	-32.080110	-32.895164
5	46218.69030	0.00000	-33.421521	-31.832441	-32.847099
6	46410.23683	0.00000	-33.456456	-31.554770	-32.769033
7	46559.68160	0.00000	-33.460457	-31.246164	-32.660033
8	46729.39033	0.00000	-33.479346	-30.952448	-32.565921
9	46886.95996	0.00000	-33.489317	-30.649812	-32.462891
10	47067.41930	0.00000	-33.516105	-30.363995	-32.376678
11	47267.44833	0.00000	-33.557273*	-30.092556	-32.304845
12	47384.72896	0.00000	-33.537641	-29.760318	-32.172211
13	47498.67662	0.00001	-33.515560	-29.425631	-32.037129

Máster en Finanzas

Curso 2024-25

Herrera Roncero, Martín

14	47644.72330	0.00000	-33.517063	-29.114529	-31.925632
15	47749.30485	0.00032	-33.488101	-28.772960	-31.783668
16	47895.61487	0.00000	-33.489798	-28.462051	-31.672364

El orden VAR indica que los retardos óptimos son dos

Se realiza el contraste de Johansen en orden dos.

Contraste de Johansen:

Número de ecuaciones = 12

Orden del retardo = 2

Periodo de estimación: 2012-01-05 - 2022-06-30 (T = 2736)

Caso 3: Constante no restringida

Log-verosimilitud = 53481.4 (Incluyendo un término constante: 45717)

Rango	Valor propio	Estad. traza	valor p	Estad. Lmáx	valor p
0	0.075901	622.97	[0.0000]	215.97	[0.0000]
1	0.043640	407.00	[0.0000]	122.08	[0.0000]
2	0.029593	284.92	[0.0001]	82.189	[0.0001]
3	0.020072	202.73	[0.0255]	55.475	[0.0936]
4	0.017316	147.25	[0.1912]	47.791	[0.1378]
5	0.013629	99.460	[0.6208]	37.545	[0.3241]
6	0.0085454	61.915	[0.9210]	23.481	[0.8481]
7	0.0050322	38.434	[0.9639]	13.803	[0.9839]
8	0.0036672	24.631	[0.9240]	10.052	[0.9759]
9	0.0028662	14.579	[0.8084]	7.8531	[0.9039]
10	0.0022635	6.7263	[0.6152]	6.1999	[0.5944]
11	0.00019239	0.52643	[0.4681]	0.52643	[0.4681]

Corregido por el tamaño muestral (gl = 2711)

Rango Estad. traza valor p

0	622.97	[0.0000]
1	407.00	[0.0000]
2	284.92	[0.0001]
3	202.73	[0.0265]
4	147.25	[0.1949]
5	99.460	[0.6246]
6	61.915	[0.9221]
7	38.434	[0.9643]
8	24.631	[0.9244]
9	14.579	[0.8062]
10	6.7263	[0.6155]
11	0.52643	[0.4683]

Fuente Yahoo finance, FED, elaboración propia.

El contraste indica que existen cuatro relaciones de largo plazo en el conjunto de las variables. Se abre la posibilidad de modelar estas relaciones mediante un modelo VECM y añadir los términos de corrección de error a las variables explicativas del modelo. (Gujarati & Porter, 2013)

A continuación, se realiza el mismo análisis con las variables DGS3MO, DGS1, DGS2, DGS5, DGS7, DGS10, DBAA, DAAA, EURUSD, CNYUSD, MXNUSD, CADUSD.

Sistema VAR, máximo orden de retardos 16

Los asteriscos de abajo indican los mejores (es decir, los mínimos)

valores de cada criterio de información, AIC = criterio de Akaike,

BIC = criterio bayesiano de Schwarz y HQC = criterio de Hannan-Quinn.

retardos	log.veros	p(RV)	AIC	BIC	HQC
1	113005.24174		-82.946888	-82.608129	-82.824431
2	113887.36410	0.00000	-83.489426	-82.837965*	-83.253931*
3	114105.82674	0.00000	-83.544158	-82.579995	-83.195625
4	114281.41419	0.00000	-83.567375	-82.290511	-83.105805
5	114434.28217	0.00000	-83.573894	-81.984328	-82.999285
6	114594.55705	0.00000	-83.585856*	-81.683589	-82.898210
7	114702.17235	0.00011	-83.559112	-81.344144	-82.758429
8	114826.75092	0.00000	-83.544837	-81.017167	-82.631116
9	114941.33774	0.00001	-83.523218	-80.682846	-82.496459
10	115102.58096	0.00000	-83.535892	-80.382819	-82.396096
11	115287.01890	0.00000	-83.565615	-80.099840	-82.312781
12	115392.92831	0.00020	-83.537617	-79.759141	-82.171746
13	115490.66568	0.00280	-83.503613	-79.412436	-82.024704
14	115604.37671	0.00001	-83.481350	-79.077471	-81.889403
15	115712.85019	0.00008	-83.455237	-78.738657	-81.750253
16	115819.19924	0.00017	-83.427563	-78.398281	-81.609541

Contraste Johansen

Número de ecuaciones = 12
Orden del retardo = 2
Periodo de estimación: 2012-01-05 - 2022-06-30 (T = 2735)
Caso 3: Constante no restringida

Log-verosimilitud = 122209 (Incluyendo un término constante: 114447)

Rango	Valor propio	Estad. traza	valor p	Estad. Lmáx	valor p
0	0.075380	687.06	[0.0000]	214.35	[0.0000]
1	0.050613	472.71	[0.0000]	142.05	[0.0000]
2	0.039477	330.66	[0.0000]	110.16	[0.0000]
3	0.026575	220.51	[0.0018]	73.667	[0.0003]
4	0.015586	146.84	[0.1987]	42.965	[0.3395]
5	0.013051	103.87	[0.4815]	35.928	[0.4180]
6	0.0082984	67.946	[0.7861]	22.791	[0.8782]
7	0.0064555	45.155	[0.8236]	17.713	[0.8827]
8	0.0037715	27.442	[0.8350]	10.334	[0.9707]
9	0.0033617	17.107	[0.6405]	9.2097	[0.8129]
10	0.0018171	7.8977	[0.4836]	4.9742	[0.7452]
11	0.0010683	2.9235	[0.0873]	2.9235	[0.0873]

Corregido por el tamaño muestral (gl = 2710)

Rango	Estad. traza	valor p
0	687.06	[0.0000]
1	472.71	[0.0000]
2	330.66	[0.0000]
3	220.51	[0.0019]
4	146.84	[0.2025]
5	103.87	[0.4855]
6	67.946	[0.7882]
7	45.155	[0.8249]
8	27.442	[0.8358]
9	17.107	[0.6374]
10	7.8977	[0.4840]
11	2.9235	[0.0871]

Fuente Yahoo finance, FED, elaboración propia.

El contraste indica que existen cuatro relaciones de largo plazo en el conjunto de las variables. Se abre la posibilidad de modelar estas relaciones mediante un modelo VECM y añadir los términos de corrección de error a las variables explicativas del modelo. (Gujarati & Porter, 2013)

Se han realizado dos modelos VECM y se han añadido los términos de corrección de error a las variables explicativas de forma que los modelos puedan interpretar las relaciones entre estos. Los modelos VECM calculados presentan significatividad en los términos de error y se han incluido en el [anexo](#). Como aspecto importante a destacar, aunque no se ha llegado al apartado de implementación práctica, es conveniente comentar que esta variable ha tenido un impacto significativo. Se ha añadido en último lugar al modelo y de manera individual (no se han añadido más variables la misma vez) por lo que evaluar su impacto individualmente ha sido factible, el efecto de añadir esta variable ha sido muy positivo elevando el valor final de la cartera en 15 puntos porcentuales en el modelo que se expondrá al final del trabajo.

3.4. Entorno de simulación

3.4.1. Espacio de observación

El entorno con el que trabaja e interactúa el algoritmo está compuesto por una matriz $n \text{ activos} \times n \text{ variables}$ y por un vector de pesos en cartera de los activos. Este último vector se recalcula en cada paso fruto del aumento o disminución del valor de cada activo.

$$\begin{pmatrix} a_1 & \cdots & a_n \\ \vdots & \ddots & \vdots \\ f_{1j} & \cdots & f_{nj} \end{pmatrix} \quad v_p = (p_1, \dots, p_n)$$

Además se incluye la matriz de correlaciones entre activos en una ventana móvil de 60 días.

3.4.2. Espacio de acción

El agente decide en cada paso, en cada día, la asignación de activos en la cartera. Por ejemplo, pasar de un 8.5% de TSLA a un 8.7%. Además, se incorpora la opción de poseer hasta un 20% de la cartera en efectivo, el efectivo se ha definido como un activo con 0% de rentabilidad.

3.4.3. Lógica interna

- 1) El agente comienza con una riqueza inicial con valor 1.
- 2) Decide una distribución de pesos inicial
- 3) Calcula como cambiarán los pesos para el día siguiente en función del cambio de valor de los activos en el mercado
- 4) Decide la distribución de pesos de los activos para el día siguiente teniendo en cuenta un coste de transacción del 0,1%.*
- 5) Cálculo del retorno bruto del portafolio en ese día, cálculo del retorno neto que es el retorno bruto menos el coste de transacción y cálculo del retorno del S&P500
- 6) Cálculo de la recompensa
- 7) Actualización del patrimonio acumulado
- 8) Guardado de datos de pesos, retorno y patrimonio acumulado

* Los costes de transacción normalmente están definidos por el spread entre precio de venta y precio de compra. Cada bróker aplica spreads distintos, además dentro del mismo bróker pueden aplicarse spreads distintos para cada activo. La web [Brokerchooser.com](https://www.brokerchooser.com) ofrece una comparativa sobre los spreads de los CFDs sobre el S&P500 entre 10 brokers. Como nuestra cartera estará compuesta por acciones del S&P500 y ante la indisponibilidad de obtener el spread de cada acción se tomará como referencia el spread de este CFD.

Los spreads están indicados en puntos de cotización, para traducirlos a porcentaje se dividirá entre la cotización del índice. Como cotización del índice se ha tomado la media aritmética de la cotización en el horizonte temporal del trabajo, 2012-2025, 3039.64 puntos.

Tabla 12 Spreads CFDs del S&P500 en distintos brokers

Broker	Spread S&P500 (puntos)	Coste como %
Hantec Markets	0,3	0,0099%
IC Markets	0,2	0,0066%
MultiBank	0,3	0,0099%
AvaTrade	0,3	0,0099%
Oanda	0,7	0,0230%
FxPro	0,7	0,0230%
Eightcap	0,5	0,0164%
FP Markets	0,5	0,0164%
FXCM	0,4	0,0132%
Trade Nation	0,3	0,0099%

Fuente: [Brokerchooser.com](https://www.brokerchooser.com/), Yahoo Finance, elaboración propia.

Con el objetivo de mantener un perfil conservador ya que puede que existan acciones con spreads mayores se tomará un coste de transacción diez veces más elevado que el de la tabla, 0,1%.

3.4.4. Función de recompensa

La función de recompensa será el determinante del comportamiento del modelo pues es la que da señales al agente para tomar las acciones. En el caso de este trabajo, como se ha comentado anteriormente, se intentará superar por el mayor diferencial posible los rendimientos del S&P500 incluyendo aversión al riesgo.

La función de recompensa decide que se va a optimizar, del mismo modo que se ha decidido maximizar la diferencia entre rendimientos de la cartera y el S&P500 teniendo en cuenta el riesgo, se podría haber decidido, por ejemplo, optimizar el ratio de información entre otras métricas.

La función de se definirá conforme a los retornos del portafolio en función de la volatilidad, la lógica que esconde esto es normalizar las señales que se le da al modelo puesto que valores extremos en los retornos pueden provocar inestabilidad en el entrenamiento. Además de este modo aparte de maximizar rendimientos el modelo buscará reducir la volatilidad.

Por otro lado, se penalizarán rendimientos inferiores a los rendimientos del S&P500, para hacer más estable el modelo, esta condición se calculará con una ventana de 30 días, es decir se penalizarán rendimientos menores al S&P500 en los últimos 30 días.

Por último, con el fin de penalizar fuertes caídas se han implementado dos penalizaciones. La primera se basa en el enfoque Conditional VaR que se expuso al principio del trabajo, se toma la media del 5% de los peores retornos en los últimos 50 días, un CVaR negativo penalizará. La segunda es una penalización por Drawdown, es decir, máxima pérdida encadenada, se computará como penalización.

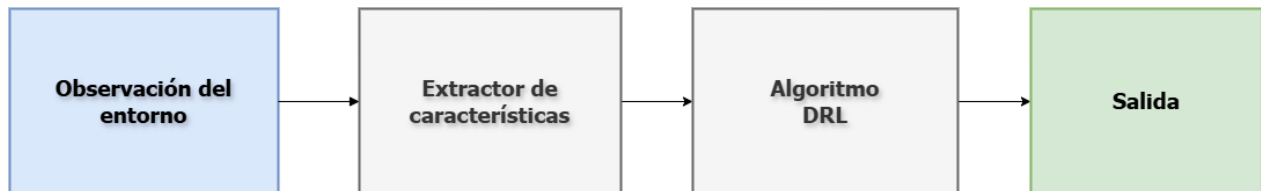
3.5. Algoritmos

Una vez desarrollado el entorno de simulación se probarán distintas arquitecturas y diseños del código. Esta tarea se realizará íntegramente con el algoritmo PPO. Una vez se obtenga la arquitectura óptima esta se exportará a los demás algoritmos de deep reinforcement learning. Realmente este proceso debería hacerse con cada algoritmo de manera independiente, sin embargo, requeriría de un número exagerado de horas de cómputo

Extractor de características: para modelizar la entrada de datos a los diferentes algoritmos de DRL se pueden aplicar redes neuronales que modele patrones relevantes en los datos. Esto se puede hacer mediante CNN, LSTM, Transformers o combinaciones de estos últimos. Se probarán de manera uniforme, con los mismos hiperparámetros en el modelo de deep reinforcement learning. Una vez se encuentre la arquitectura óptima se procederá a la optimización de hiperparámetros.

Algoritmos de deep reinforcement learning: tras decidir una arquitectura adecuada de extractor de características se continuará implementando diferentes algoritmos de deep reinforcement learning, PPO, DDPG y SAC (incluidos en la librería de Python Stable Baselines 3). Tras revisar la literatura, todos ellos serán algoritmos basados en actor critic, pues son los mas recientes y los mas usados y efectivos en optimizacion de carteras.

Ilustración 6 Arquitectura general del modelo



Para encontrar los hiperparámetros óptimos para los diferentes modelos se hará uso de la librería de Python Optuna (Akiba, et al., 2019). Los hiperparámetros a optimizar serán:

Para PPO: Tasa de aprendizaje, coeficiente de entropía, tamaño de batch, n_steps, gamma, clip_range, gae_lambda, max_grad_norm, vf_coef, número de epochs.

Para DDPG: Tasa de aprendizaje, tamaño de buffer, inicio de aprendizaje, tamaño de batch, tau, gamma, frecuencia de entrenamiento, gradient_steps, ruido de acción, ruido de acción theta.

Para SAC: Tasa de aprendizaje, coeficiente de entropía, tamaño de buffer, tamaño de batch, tau, gamma, frecuencia de entrenamiento, gradient_steps, target update Interval, use sde, sde sample freq.

La definición de cada uno de estos parámetros se encuentra en la documentación de la librería Stable Baselines 3 <https://stable-baselines3.readthedocs.io/en/master/index.html#>

3.6. Estrategia de validación del modelo

Los datos del modelo se dividirán en dos, 80% datos para entrenamiento y 20% datos de validación. El modelo aprenderá una función de valor y política óptima que se pondrá a prueba en datos que no ha visto antes, los datos de validación.

Esto no es suficiente ya que puede que los datos de validación conformen un conjunto muy homogéneo que se ajuste muy bien, incluso por mera casualidad, a los parámetros aprendidos por el modelo. Por ello adicionalmente se implementará una validación walk forward al modelo. Esta validación se aplicará a las diferentes arquitecturas extractoras de características y tras sus resultados se escogerá la mejor para proceder con la optimización de hiperparámetros y construcción del modelo final.

Por otra parte, se debe tener en cuenta la posibilidad de overfitting en las redes neuronales extractoras de características. Por ello se implementará la técnica dropout en estas redes.

3.7. Métricas de evaluación

El objetivo del modelo es superar al máximo posible al S&P500 en rentabilidad teniendo en cuenta el riesgo, las métricas de evaluación serán:

Valor final de la cartera: rentabilidad acumulada a lo largo de todo el periodo de test.

Ratio de Sharpe: $S = \frac{\text{rentabilidad} - \text{rentabilidad libre de riesgo}}{\text{volatilidad}}$

para la rentabilidad libre de riesgo se ha tomado la rentabilidad del bono estadounidense a 10 años. Directamente no se optimiza esta métrica pues la rentabilidad libre de riesgo no se incluye

en la función de recompensa, aunque si la rentabilidad entre volatilidad. Además, en la función de recompensa se tienen en cuenta otras métricas. El ratio de Sharpe idealmente será superior a 1.

Tracking Error: $TE = \sqrt{VAR(rentabilidad - rentabilidad SP500)}$

Mide la desviación típica de la rentabilidad de la cartera respecto al S&P500. Cuanto mayor sea mayor desviación, tanto al alza como a la baja.

Ratio de información $IR = \frac{(Rentabilidad - Rentabilidad SP500)}{TE}$

Esta métrica compara la diferencia de rentabilidad de la cartera respecto al S&P500 con el tracking error. Es decir, compara la rentabilidad que añade la cartera por cada unidad de riesgo que se asume por encima del S&P500. No se optimiza directamente en la función de recompensa sin embargo es de gran utilidad para medir la eficiencia en la gestión de a cartera

Máxima pérdida seguida(drawdown), recuperarse de caídas severas en el valor de la cartera es difícil. Si el valor de la cartera cae un 50% se necesita un incremento del 100% para volver a la situación inicial. Controlar la perdida máxima seguida es fundamental puesto que tendrá un impacto relevante en la rentabilidad final de la cartera.

3.8. Herramientas para la implementación

Para llevar a la práctica el modelo se ha usado el lenguaje de programación Python, versión 3.11. Como editor de código se ha usado Visual Studio Code. Las librerías de Python empleadas en la construcción del código son:

Pandas: manipulación de series temporales

Numpy: operaciones matemáticas, calculo matricial y vectorial

Matplotlib: recursos gráficos

Yfinance: obtención de datos vía conexión a Yahoo Finance

Talib: indicadores de análisis técnico

Scikit-learn: modelos de machine learning y manipulación de datos.

Gymnasium: creación de entorno para reinforcement learning

Stable Baselines3: implementación de algoritmos de reinforcement learning

Tensorflow: modelos de machine learning

Pytorch: modelos de machine learning

Statsmodels: modelos econométricos

La exigencia de hardware para ejecutar los distintos modelos es alta. En concreto el modelo se ha desarrollado sobre una tarjeta gráfica NVIDIA RTX 5070 y procesador ryzen 9600X y 32GB de memoria RAM. Cuyas especificaciones técnicas se pueden encontrar en:

<https://www.nvidia.com/es-es/geforce/graphics-cards/50-series/rtx-5070-family/>

<https://www.amd.com/es/products/processors/desktops/ryzen/9000-series/amd-ryzen-5-9600x.html>

El principal limitante es el uso de la memoria VRAM (memoria de la tarjeta gráfica, en este caso la RTX 5070 cuenta con 12GB) de la tarjeta gráfica.

4. Implementación práctica

Las características óptimas del modelo, tales como número de acciones con las que operar y extractor de características, se han determinado realizando múltiples pruebas con un modelo base con algoritmo PPO. Tales características podrían variar dependiendo del algoritmo empleado, pero debido a las limitaciones computacionales y de tiempo se hace imposible hacer un número tan elevado de pruebas.

Hiperparámetros del modelo base: Coeficiente de entropía 0.01, tasa de aprendizaje 0.0005, tamaño de batch 1024, nsteps 2048, gamma 0.96 y clip range 0.1. El número de timesteps (número de veces que el agente interactúa con el entorno en el entrenamiento) ha sido de 30.000 en cada ventana de validación walk forward. Este número de iteraciones es bajo teniendo en cuenta la complejidad de las redes neuronales que se evaluarán más adelante. La potencia de cómputo disponible hace que añadir más iteraciones suponga un consumo de tiempo exagerado e inviable para la realización de este trabajo.

4.1. Limitaciones en la implementación

A la hora de la implementación práctica se han manifestado limitaciones tanto de hardware como de software. La implementación de ciertas características hace que el consumo de recursos se multiplique, en concreto se ha sufrido una especial limitación con el uso de la memoria VRAM. Durante el desarrollo del modelo se implementarán medidas para aliviar el consumo de este recurso, algunas de ellas perjudiciales para el desempeño del modelo limitando su potencial.

En cuanto a software, se ha sufrido otra limitación, la librería de Python Stable Baselines 3 permite el uso de redes neuronales recurrentes (LSTM en nuestro caso) solo en el algoritmo PPO. Existe la librería Rlib que permite el uso de redes recurrentes en otros algoritmos actor-critic, sin embargo, su uso no ha sido posible por problemas de compatibilidad en Windows.

4.2. Selección de acciones

Se han descartado los valores con datos faltantes y los que empezaron a cotizar en una fecha posterior al 3 de enero de 2012.

Tras comenzar a entrenar el modelo con el resto de las acciones, 433, se obtienen resultados muy pobres en comparación con los resultados obtenidos con un número menor. La alta dimensionalidad tiene un efecto perjudicial en el desempeño del modelo PPO, además de que reducir la dimensión también se hace necesario para disminuir los requerimientos de memoria VRAM.

Tras ejecutar el modelo con diferentes números de acciones se ha determinado que el comportamiento óptimo se da en torno a 60 acciones.

Para reducir el número de acciones de 433 a 60 se aplica un primer filtro de capitalización de mercado. Se eliminan la mitad de las acciones de menor capitalización, que se presume que poseen datos más ruidosos.

Estas son 216 acciones, en concreto: ['A', 'AAL', 'AAP', 'ADM', 'AEE', 'AES', 'AIZ', 'AKAM', 'ALB', 'ALGN', 'ALK', 'ANSS', 'AOS', 'APA', 'ARE', 'ATO', 'AVB', 'AVY', 'AWK', 'BALL', 'BAX', 'BBWI', 'BBY', 'BEN', 'BG', 'BIIB', 'BIO', 'BR', 'BRO', 'BWA', 'BXP', 'CAG', 'CAH', 'CBOE', 'CCL', 'CE', 'CF', 'CHD', 'CHRW', 'CINF', 'CLX', 'CMA', 'CMS', 'CNC', 'CNP', 'COO', 'CPB', 'CPT', 'CRL', 'CSGP', 'CTRA', 'DAL', 'DD', 'DG', 'DGX', 'DLTR', 'DOV', 'DPZ', 'DRI', 'DTE', 'DVA', 'DVN', 'DXC', 'DXCM', 'EBAY', 'EFX', 'EIX', 'EL', 'EMN', 'EQR', 'EQT', 'ES', 'ESS', 'EVRG', 'EXPD', 'EXPE', 'EXR', 'FDS', 'FE', 'FFIV', 'FITB', 'FLR', 'FLS', 'FMC', 'FRT', 'GEN', 'GIS', 'GL', 'GNRC', 'GPC', 'GPN', 'HAL', 'HAS', 'HBAN', 'HII', 'HOLX', 'HPQ', 'HRL', 'HSIC', 'HST', 'HSY', 'HUM', 'IEX', 'IFF', 'ILMN', 'INCY', 'IP', 'IPG', 'IPGP', 'IRM', 'IT', 'IVZ', 'J', 'JBHT', 'JKHY', 'JNPR', 'K', 'KEY', 'KIM', 'KMX', 'L', 'LDOS', 'LEN', 'LH', 'LKQ', 'LNC', 'LNT', 'LUMN', 'LUV', 'LYB', 'M', 'MAA', 'MAS', 'MCHP', 'MGM', 'MHK', 'MKC', 'MKTX', 'MLM', 'MOS', 'MPWR', 'MTB',

'MTD', 'NDSN', 'NI', 'NRG', 'NTAP', 'NTRS', 'NUE', 'NVR', 'NWL', 'ODFL', 'OMC', 'ON', 'PARA', 'PFG', 'PHM', 'PKG', 'PNR', 'PNW', 'PPG', 'PPL', 'PRGO', 'PVH', 'R', 'REG', 'RF', 'RHI', 'RIG', 'RJF', 'RL', 'ROK', 'ROL', 'SBAC', 'SCI', 'SEE', 'SJM', 'SLM', 'SNA', 'STE', 'STT', 'STX', 'STZ', 'SWK', 'SWKS', 'SYY', 'TAP', 'TER', 'TFX', 'TRIP', 'TROW', 'TSCO', 'TXT', 'TYL', 'UAA', 'UAL', 'UDR', 'UHS', 'ULTA', 'UNM', 'VFC', 'VNO', 'VRSN', 'VTR', 'VTRS', 'WAB', 'WAT', 'WBA', 'WDC', 'WEC', 'WHR', 'WRB', 'WST', 'WU', 'WY', 'WYNN']

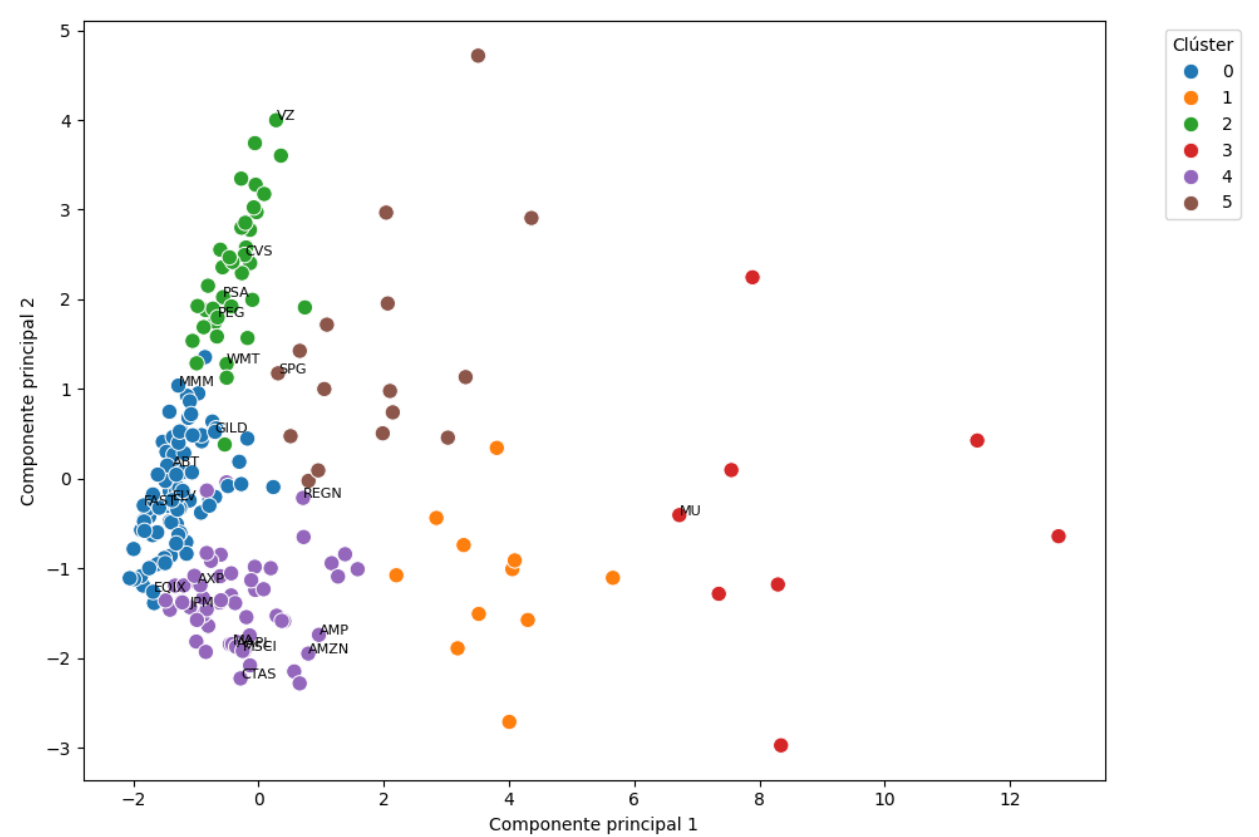
A continuación, con el objetivo de conservar la diversidad en el conjunto de acciones y reducir el número de acciones en cartera a 60 se aplica un clustering.

Como variables para crear los distintos clústeres se han escogido, el retorno medio diario, volatilidad medida como desviación típica, CVaR al 95%, coeficiente beta, correlación respecto al S&P500, ratio de Sharpe, Tracking Error y ratio de información.

Esta técnica permite agrupar las acciones según su comportamiento de manera que obtengamos grupos de acciones que se comportan de manera distinta entre sí, pero de manera similar dentro del mismo grupo.

Una vez hecho el clustering se escogerán las acciones más representativas de cada clúster de modo que se llegue un número óptimo de en torno a 60 acciones conservando diversidad en el comportamiento de estas.

Gráfico 5 Clustering selección de acciones



Fuente: Yahoo Finance, elaboración propia

Tabla 13 Características de cada clúster

Cluster	R. media	Volatilidad	CVaR95	Beta	Corr.	Sharpe	TE	IR
0	0.0007	0.0164	-0.0373	0.9328	0.6067	0.5583	0.0208	0.0981
1	0.0012	0.0247	-0.0543	1.3906	0.6035	0.6633	0.0319	0.5042
2	0.0004	0.0135	-0.0311	0.5997	0.4744	0.4449	0.0201	-0.1784

3	0.0016	0.0316	-0.0686	1.5241	0.5160	0.6572	0.0439	0.5786
4	0.0009	0.0179	-0.0404	1.0858	0.6487	0.7794	0.0217	0.4177
5	0.0005	0.0230	-0.0519	1.1245	0.5216	0.1854	0.0312	-0.0503

Fuente: Yahoo Finance, elaboración propia

Los activos más representativos de cada clúster serían:

Clúster 0: 'APD', 'UNP', 'AFL', 'CSCO', 'FAST', 'AMGN', 'PLD', 'PCAR', 'RTX', 'BK'

Clúster 1: 'FTNT', 'NXPI', 'MPC', 'AMAT', 'KLAC', 'LRCX', 'VLO', 'DFS', 'URI', 'AVGO'

Clúster 2: 'MRK', 'T', 'PSA', 'BDX', 'SO', 'PEG', 'XEL', 'PM', 'MDLZ', 'MO'

Clúster 3: 'RCL', 'AXON', 'MU', 'NFLX', 'AMD', 'NVDA', 'FCX', 'TSLA'

Clúster 4: 'AAPL', 'MSCI', 'INTU', 'SNPS', 'BAC', 'ODFL', 'PH', 'MCO', 'BKNG', 'ADBE'

Clúster 5: 'EOG', 'BKR', 'COP', 'BA', 'WMB', 'F', 'INTC', 'HES', 'OKE', 'SPG'

Adicionalmente se permite mantener en cartera efectivo, y los ETF de bonos del tesoro de EE.UU. "SHY", "TLT", "IEF". De este modo la cartera queda definida con 62 activos.

4.3. Función de recompensa

Antes de definir la función de recompensa se definirán las siguientes variables:

$$\text{Rendimiento de cada activo}(r) = (P_t - P_{t-1})/P_{t-1}$$

$$\text{Rendimiento bruto de la cartera} = \sum w_i r_i \text{ donde } w_i \text{ es el peso de cada activo en la cartera}$$

$$\text{Coste de transacción} = \left(\sum w_{i,t} - w_{i,t-1} \right) * 0,1\%$$

$$\text{Rendimiento neto de la cartera} = \text{Rendimiento bruto de la cartera} - \text{coste de transacción}$$

$$\text{Rendimiento neto normalizado de la cartera} = \frac{\text{Rendimiento neto de la cartera}}{\text{Desviación típica últimos 50 días}}$$

De aquí se extrae la recompensa base que se calcula como

$$\text{Recompensa base} = \ln(1 + \text{rendimiento neto normalizado de la cartera})$$

Para evitar errores el mínimo que puede estar dentro del logaritmo como rendimiento neto normalizado de la cartera es -0.999

A continuación, la penalización por rendimientos inferiores a los del S&P500

Se calcula el diferencial de retorno entre la cartera y el S&P500 y se normaliza usando la desviación típica de rendimientos de la cartera

$$\text{Diferencial normalizado} = \frac{\text{Retonos SP500}(30\text{días}) - \text{Retonos cartera}(30\text{días})}{\text{Desviación típica últimos 30 días}}$$

Se define un coeficiente de penalización, en nuestro caso 0,5

$$\text{Penalización diferencial respecto a SP500} = \text{Coeficiente de penalización} * \text{Diferencial norm.}$$

La penalización por drawdown mayor al 20% se define como:

$$\text{Penalización por drawdown} = \text{drawdown} * 100$$

Por último, se añade otra penalización de aversión al riesgo basada en el condicional VaR al 5% de los últimos 50 días que se especifica como:

$$CVaR = \frac{\sum n \text{ rendimientos en la cola inferior}}{n}$$

Se define otro coeficiente de penalización en este caso 1.

$$\text{Penalización CVaR} = \frac{\text{coeficiente} * (-CVaR)}{\text{Desviación típica últimos 50 días}}$$

Al final la recompensa queda como

$$\text{Recompensa} = \text{Recompensa base} - \text{penalización diferencial respecto a SP500} - \text{Penalización CVaR} + \text{Penalización por drawdown}$$

A modo ilustrativo se aporta la siguiente tabla que ejemplifica algunas situaciones de recompensa

Tabla 14 Variables para función de recompensa

Rendimiento neto	Volatilidad 50 días	Retornos S&P500 30 días	Retornos cartera 30 días	Volatilidad 30 días	CVaR	Drawdown
0,02	0,01	0,1	0,06	0,02	-0,02	0
0,05	0,005	0,07	0,11	0,02	0,01	0
-0,01	0,03	-0,01	0,005	0,01	-0,08	0
-0,04	0,005	-0,14	-0,08	0,07	-0,14	-0,21

Fuente: elaboración propia

Tabla 15 Resultados función de recompensa

Base	Diferencial	CVaR	Drawdown	Recompensa Total
1,098612	1,000	-0,2	0	0,299
2,3978	-1,000	0,1429	0	3,255
-0,40547	-0,750	8	0	-7,655
-6,9078	-0,429	1	-21	-28,479

Fuente: elaboración propia

La normalización mediante la desviación típica hace que la recompensa sea proporcionada a la volatilidad del mercado. Por ejemplo, se penalizan fuertemente malos resultados en circunstancias de baja volatilidad, la penalización por CVaR en el tercer caso. El caso opuesto ocurre en el caso segundo, se recompensa fuertemente la obtención de buenos resultados en baja volatilidad.

El drawdown se penaliza de manera significativa pues potencialmente es uno de los hechos que más daño puede hacer al desempeño de la cartera.

Es llamativo que la recompensa base se calcula como un logaritmo mientras que las demás recompensas no. Esto es simplemente porque a lo largo del desarrollo del modelo se han obtenido mejores resultados empleando esta estructura.

Por último, existe la posibilidad de que haya momentos en que la volatilidad tienda a cero por lo que la recompensa o penalización puede tender a infinito. Para evitar este problema la recompensas y penalizaciones se recortan a un mínimo de -30 y a un máximo de 30.

4.4. Resultados de los diferentes extractores de características

Para comparar los resultados de las diferentes arquitecturas se hace necesaria la ejecución del algoritmo múltiples veces ya que al incluir procesos estocásticos y estimaciones complejas mediante redes neuronales cada vez que se ejecuta el programa se obtienen salidas distintas. Debido a las limitaciones de cómputo se realizarán cinco validaciones walk forward por cada arquitectura y se tomará la media de los resultados como referencia. Idealmente se hubiesen

ejecutado un número más significativo de pruebas pero la restricción de tiempo de cómputo se hace presente.

Como limitación añadida cabe mencionar que las redes actor y critic se han mantenido como MLP, es posible usar otro tipo de redes neuronales que potencialmente den mejores resultados. Sin embargo, se abre un abanico de posibles combinaciones de arquitectura que requiere de un tiempo de cómputo significativo para obtener resultados robustos. Por ello ante la limitación de tiempo de cómputo disponible se ha simplificado utilizando solo redes MLP en este caso.

4.4.1. Sin extractor de características

En este caso no se utilizará ningún extractor de características, la observación del entorno se envía directamente a las redes actor y critic. Este enfoque servirá de base para evaluar a los demás.

Ilustración 7 Arquitectura sin extractor de características

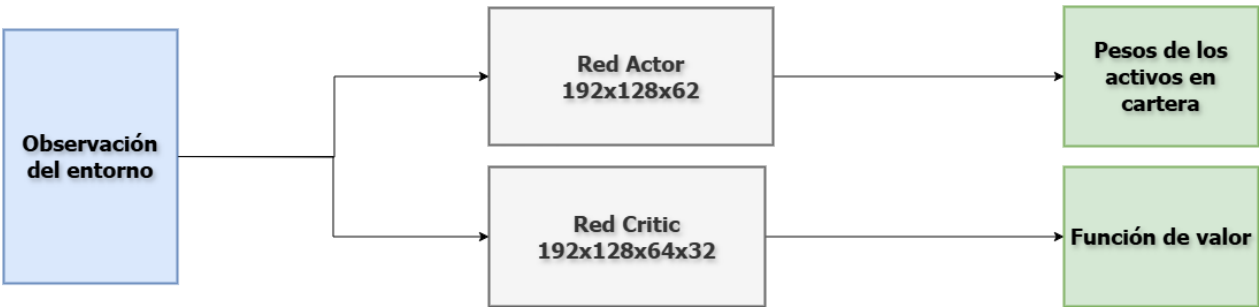


Tabla 16 Métricas ejecuciones sin extractor de características

	Valor final	Sharpe	TE	IR	Max DD	Rentabilidad anual	Volatilidad anual
Prueba 1							
WF1	1,436	0,6691	5,07%	1,3515	20,99%	15,06%	15,86%
WF2	1,5596	1,0028	4,23%	1,5806	29,03%	18,80%	14,31%
WF3	1,5441	0,4864	8,65%	1,0875	42,70%	18,34%	28,56%
WF4	1,8685	1,2013	5,85%	1,2947	32,70%	27,43%	19,12%
Prueba 2							
WF1	1,4379	0,6732	5,06%	1,3638	20,92%	15,12%	15,85%
WF2	1,5623	1,0047	4,28%	1,5806	29,15%	18,80%	14,37%
WF3	1,543	0,4855	8,64%	1,0863	42,66%	18,31%	28,56%
WF4	1,8728	1,2087	5,85%	1,3096	32,69%	27,54%	19,10%
Prueba 3							
WF1	1,4361	0,67	5,06%	1,3543	21,01%	15,06%	15,84%
WF2	1,5601	1,0022	4,26%	1,5766	29,07%	18,82%	14,34%
WF3	1,5422	0,4839	8,69%	1,0784	42,77%	18,29%	28,60%
WF4	1,8685	1,2019	5,84%	1,2974	32,67%	27,42%	19,12%
Prueba 4							
WF1	1,4362	0,6701	5,06%	1,353	20,97%	15,07%	15,84%
WF2	1,5596	0,9991	4,29%	1,5631	29,08%	18,80%	14,37%
WF3	1,5435	0,4854	8,67%	1,0842	42,72%	18,33%	28,59%
WF4	1,8682	1,202	5,84%	1,2966	32,66%	27,42%	19,11%
Prueba 5							
WF1	1,435	0,6675	5,06%	1,3475	21,03%	15,03%	15,85%
WF2	1,5603	1,0021	4,26%	1,5765	29,07%	18,82%	14,34%
WF3	1,5412	0,4833	8,65%	1,0795	42,70%	18,26%	28,57%
WF4	1,8676	1,2007	5,87%	1,2879	32,70%	27,40%	19,11%
MEDIA							

WF1	1,4362	0,6700	5,06%	1,3540	20,98%	15,07%	15,85%
WF2	1,5604	1,0022	4,26%	1,5755	29,08%	18,81%	14,35%
WF3	1,5428	0,4849	8,66%	1,0832	42,71%	18,31%	28,58%
WF4	1,8691	1,2029	5,85%	1,2972	32,68%	27,44%	19,11%

Fuente: Yahoo Finance, elaboración propia

4.4.2. Transformer+LSTM tamaño de red pequeña

Esta arquitectura consta de un extractor de características de la serie temporal basado en un modelo transformer. El modelo transformer consta de dos capas y 5 cabezas de atención, dmodel 80 y 3 capas, además se incluye un dropout del 10% para evitar overfitting.

Se utiliza el encoder del transformer de modo que este recibe la observación del mercado, cada activo con sus variables explicativas, recibe también el cambio en el peso en cartera de cada activo fruto de su apreciación o depreciación en el mercado y por último recibe la matriz de correlación entre los activos de los últimos 60 días. Tras recibir este input el transformer codifica la información captando las características y dependencias globales de los activos devolviendo un vector de características de tamaño 256.

Este vector se sirve de entrada a una red LSTM de dos capas de 128 neuronas cada una y que consta con un dropout del 10%. Esta red dota de memoria temporal al modelo y captura las dependencias temporales entre las características que genera el transformer. De esta forma se pueden capturar patrones estacionales, procesar series no estacionarias y olvidar información obsoleta. Por ejemplo, antes de la revolución de la inteligencia artificial la cotización de Nvidia tenía un comportamiento distinto al que tiene actualmente. Otro ejemplo podría ser la evolución del comportamiento de una compañía a las variaciones del tipo de interés, en los momentos en los que se ha encontrado más endeudada su sensibilidad sería mayor.

La salida de la red LSTM se encuentra conectada a las redes de actor y critic del modelo PPO. La redes tienen capas de tamaño decreciente, esto es porque la salida de la red actor es igual a 62, el número de activos de la cartera, y el de la red critic es igual a 1, la función de valor. Aplicar una estructura decreciente ayuda a comprimir la información de manera más progresiva. Idealmente se hubiera aplicado un dropout del 10% a estas redes al igual que en las anteriores pero la librería de Python Stable Baselines 3 no permite esta opción.

Ilustración 8 Arquitectura Transformer+LSTM con tamaño de redes pequeñas

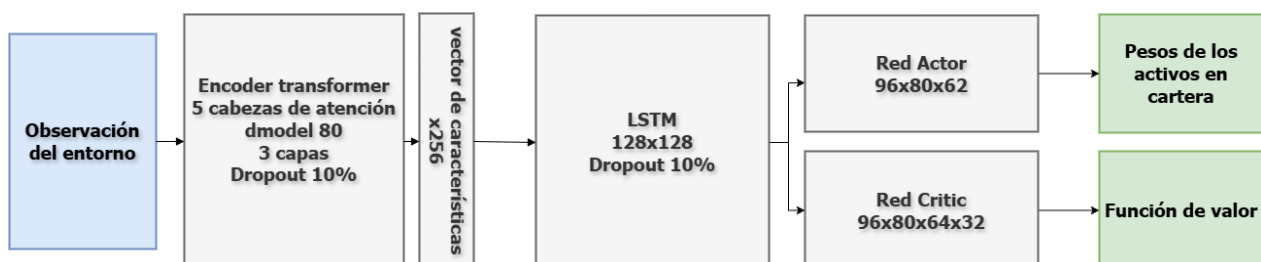


Tabla 17 Métricas ejecuciones TRFM+LSTM redes pequeñas

	Valor final	Sharpe	TE	IR	Max DD	Rentabilidad anual	Volatilidad anual
Prueba 1							
WF1	1,4349	0,6674	5,06%	1,3462	21,04%	15,03%	15,85%
WF2	1,5605	1,0018	4,26%	1,5773	29,06%	18,83%	14,35%
WF3	1,5439	0,486	8,65%	1,0882	42,70%	18,34%	28,58%
WF4	1,8703	1,2032	5,83%	1,3057	32,78%	27,47%	19,13%
Prueba 2							
WF1	1,4371	0,6714	5,06%	1,3586	21,04%	15,1%	15,86%
WF2	1,5604	1,0026	4,26%	1,5756	29,08%	18,83%	14,34%

WF3	1,5433	0,4854	8,70%	1,0803	42,76%	18,32%	28,58%
WF4	1,8679	1,2027	5,85%	1,2921	32,57%	27,41%	19,09%
Prueba 3							
WF1	1,4354	0,669	5,06%	1,3502	20,98%	15,0%	15,83%
WF2	1,5586	0,9985	4,27%	1,564	29,03%	18,78%	14,35%
WF3	1,5396	0,481	8,67%	1,0747	42,76%	18,21%	28,61%
WF4	1,8733	1,2098	5,86%	1,3091	32,69%	27,55%	19,10%
Prueba 4							
WF1	1,4379	0,6733	5,06%	1,362	20,97%	15,12%	15,85%
WF2	1,5584	0,9991	4,24%	1,5707	29,04%	18,77%	14,33%
WF3	1,5454	0,4881	8,61%	1,0963	42,62%	18,38%	28,55%
WF4	1,8658	1,1991	5,85%	1,2846	32,59%	27,35%	19,10%
Prueba 5							
WF1	1,4367	0,6706	5,07%	1,3546	21,02%	15,08%	15,85%
WF2	1,5605	1,0016	4,27%	1,5744	29,10%	18,83%	14,36%
WF3	1,5435	0,4855	8,68%	1,0834	42,73%	18,33%	28,59%
WF4	1,8697	1,2021	5,85%	1,2998	32,75%	27,46%	19,14%
MEDIA							
WF1	1,4364	0,6703	5,06%	1,3543	21,01%	15,01%	15,85%
WF2	1,5597	1,0007	4,26%	1,5724	29,06%	18,81%	14,35%
WF3	1,5431	0,4852	8,66%	1,0846	42,71%	18,32%	28,58%
WF4	1,8694	1,2034	5,85%	1,2983	32,68%	27,45%	19,11%

Fuente: Yahoo Finance, elaboración propia

La implementación de la red transformer y LSTM no han mejorado significativamente las métricas del modelo. Esto puede tener como causa un entorno poco complejo, cantidad de datos baja ya que se tratan de datos con frecuencia diaria. Si se tratara de datos con una frecuencia mayor podrían darse problemas de explosión y desvanecimiento de gradiente tomando mayor relevancia la red LSTM. En resumen, para el entorno actual en el que trabaja el modelo esta red presenta sobrecapacidad. Aun así, para confirmar que existe sobre capacidad se implementará la misma arquitectura pero con redes más grandes.

4.4.3. Transformer+LSTM tamaño de red grande

Esta arquitectura es exactamente igual a la anterior. Las únicas diferencias es que el transformer es una red de mayor tamaño con dmodel=160 y 10 cabezas de atención, del mismo modo el tamaño de la red LSTM también ha aumentado constando cada capa de 256 neuronas.

Esta modificación hace el modelo más pesado en cómputo con el objetivo de obtener una mejor representación e interpretación de la serie temporal.

Ilustración 9 Arquitectura Transformer+LSTM tamaño de redes grande

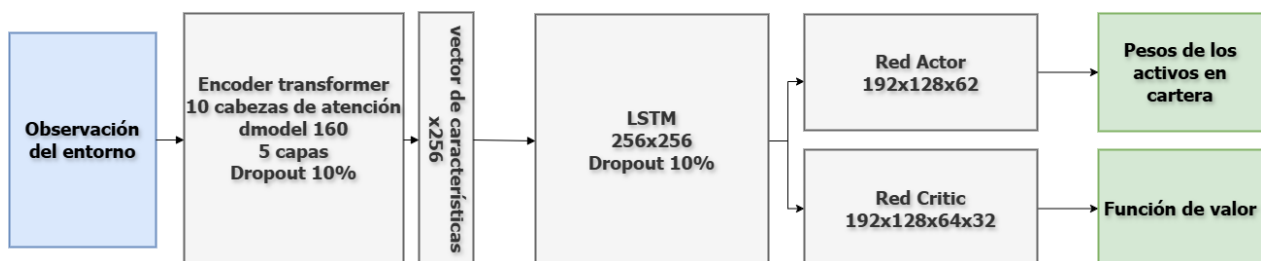


Tabla 18 Métricas ejecuciones TRFM+LSTM redes grandes

	Valor final	Sharpe	TE	IR	Max DD	Rentabilidad anual	Volatilidad anual
Prueba 1							
WF1	1,4347	0,6700	5,05%	1,35	20,95%	14,99%	15,86%

WF2	1,5607	1,0000	4,26%	1,58	29,05%	18,84%	14,34%
WF3	1,5426	0,4800	8,64%	1,09	42,64%	18,30%	28,57%
WF4	1,8716	1,2100	5,83%	1,31	32,69%	27,45%	19,11%
Prueba 2							
WF1	1,4366	0,6702	5,07%	1,3545	21,05%	15,08%	15,86%
WF2	1,5634	1,0000	4,29%	1,58	29,17%	18,80%	14,36%
WF3	1,5401	0,4821	8,66%	1,0752	42,76%	18,22%	28,57%
WF4	1,8695	1,2022	5,89%	1,2963	32,73%	27,45%	19,13%
Prueba 3							
WF1	1,434	0,6651	5,09%	1,3358	21,07%	15,00%	15,86%
WF2	1,5598	1,0005	4,26%	1,5747	29,10%	18,81%	14,35%
WF3	1,5436	0,486	8,64%	1,0877	42,62%	18,33%	28,56%
WF4	1,8722	1,2061	5,86%	1,3066	32,78%	27,52%	19,13%
Prueba 4							
WF1	1,4338	0,6655	5,06%	1,3423	20,97%	14,99%	15,84%
WF2	1,5599	1,0021	4,25%	1,5758	29,03%	18,81%	14,33%
WF3	1,5452	0,487	8,69%	1,088	42,73%	18,38%	28,60%
WF4	1,8703	1,2036	5,85%	1,30008	32,77%	27,47%	19,13%
Prueba 5							
WF1	1,433	0,6631	5,07%	1,3339	21,12%	14,97%	15,86%
WF2	1,5588	0,9983	4,27%	1,5635	29,07%	1,88%	14,35%
WF3	1,5439	0,4856	8,66%	1,0867	42,72%	18,34%	28,60%
WF4	1,8668	1,2009	5,85%	1,2877	32,65%	27,38%	19,10%
MEDIA							
WF1	1,4344	0,6668	5,07%	1,3433	21,03%	15,01%	15,86%
WF2	1,5605	1,0002	4,27%	1,5748	29,08%	15,43%	14,35%
WF3	1,5431	0,4841	8,66%	1,0855	42,69%	18,31%	28,58%
WF4	1,8701	1,2046	5,86%	1,3001	32,72%	27,45%	19,12%

Fuente: Yahoo Finance, elaboración propia.

Aumentar el tamaño de las redes transformer y LSTM no ha mejorado los parámetros del modelo. Las redes más grandes no mejoran la interpretación de los datos. Como se explicó anteriormente parece que existe sobrecapacidad en el modelo.

4.4.4. CNN+LSTM

Esta arquitectura es similar a las anteriores. Una red neuronal convolucional 2D procesa los activos y sus variables explicativas mediante sus filtros "kernel". La primera capa convolucional cuenta con kernels (8x8), la segunda con kernels (5x5) y la tercera con kernels (3x3). De esta forma se consiguen capturar dependencias que van de más globales a más locales logrando describir de una manera precisa el comportamiento de los activos y sus variables. La red cuenta un dropout del 10% para evitar overfitting. La primera capa da como salida 32 canales que sirven de input a la siguiente capa que como salida da 64 canales, estos canales sirven de input a la tercera capa que da como salida 128 canales.

Una vez los procesa los concatena con la matriz de correlaciones y los cambios de pesos en cartera debido a la depreciación o apreciación de los activos en el mercado.

La CNN devuelve un vector de características de 256 dimensiones que, al igual que en los modelos anteriores, es el input de una red LSTM de dos capas de 256 neuronas cada una.

En esta arquitectura se hace presente la limitación de hardware, el modelo consume mas memoria VRAM de la disponible por lo que se ha hecho necesario reducir el tamaño del primer kernel a (7x7) y el tamaño de batch a 512. Factores que pueden limitar el desempeño del modelo.

Ilustración 10 Arquitectura CNN+LSTM

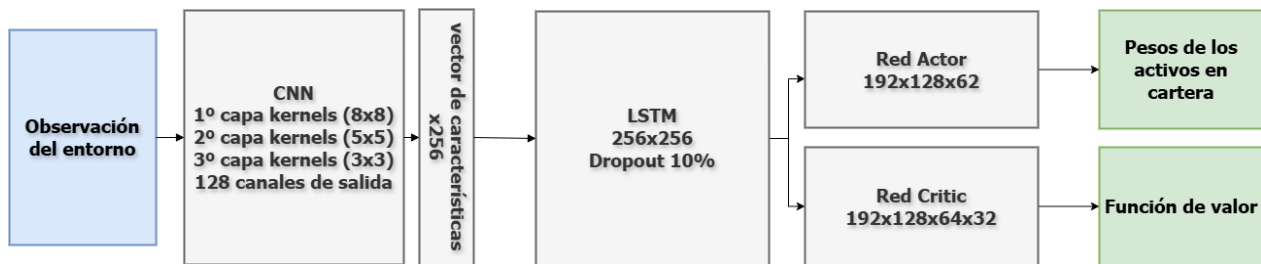


Tabla 19 Métricas ejecución CNN+LSTM

	Valor final	Sharpe	TE	IR	Max DD	Rentabilidad anual	Volatilidad anual
Prueba 1							
WF1	1,4349	0,6664	5,08%	1,3427	21,06%	15,03%	15,87%
WF2	1,5596	1,0008	4,26%	1,5719	29,10%	18,80%	14,34%
WF3	1,5435	0,4860	8,63%	1,0886	42,66%	18,33%	28,56%
WF4	1,8691	1,2031	5,85%	1,2671	32,69%	27,44%	19,11%
Prueba 2							
WF1	1,4353	0,6677	5,07%	1,3473	21,04%	15,04%	15,86%
WF2	1,5607	1,0030	4,26%	1,5781	29,04%	18,84%	14,34%
WF3	1,5464	0,4884	8,67%	1,0930	42,71%	18,41%	28,58%
WF4	1,8733	1,2087	5,86%	1,3091	32,76%	27,55%	19,11%
Prueba 3							
WF1	1,4356	0,6663	5,07%	1,3533	21,03%	15,02%	15,87%
WF2	1,5613	1,0009	4,26%	1,5917	29,04%	18,82%	14,35%
WF3	1,5459	0,4868	8,67%	1,0878	42,72%	18,37%	28,59%
WF4	1,8715	1,2037	5,86%	1,2791	32,77%	27,46%	19,11%
Prueba 4							
WF1	1,4346	0,6646	5,07%	1,3334	21,05%	14,99%	15,86%
WF2	1,5626	1,0036	4,26%	1,6226	29,03%	18,86%	14,35%
WF3	1,5465	0,4872	8,67%	1,0937	42,72%	18,38%	28,60%
WF4	1,8706	1,2022	5,86%	1,2624	32,76%	27,43%	19,12%
Prueba 5							
WF1	1,4350	0,6655	5,07%	1,3417	21,05%	15,00%	15,86%
WF2	1,5621	1,0030	4,26%	1,6118	29,04%	18,85%	14,35%
WF3	1,5462	0,4867	8,67%	1,0902	42,73%	18,38%	28,61%
WF4	1,8700	1,2008	5,86%	1,2540	32,75%	27,42%	19,13%
Media							
WF1	1,4351	0,6661	5,07%	1,3437	21,04%	15,02%	15,86%
WF2	1,5613	1,0022	4,26%	1,5952	29,05%	18,83%	14,35%
WF3	1,5457	0,4870	8,66%	1,0907	42,71%	18,37%	28,59%
WF4	1,8709	1,2037	5,86%	1,2743	32,75%	27,46%	19,11%

Fuente: Yahoo Finance, elaboración propia.

La red CNN tiene como desventaja respecto al transformer que únicamente modela patrones locales de la serie, los kernels, mientras que el transformer modela todos los datos en su conjunto. Se obtienen unos resultados con diferencias despreciables. De nuevo parece que implementar estas redes extractoras de características no han ayudado de manera significativa al modelo. Cabe añadir que este modelo es el menos eficiente pues es el que más memoria VRAM y tiempo de cómputo consume.

4.4.5. Selección de arquitectura

Como la cartera construida del modelo está compuesta en su mayoría por acciones del índice S&P500 esta presenta una alta correlación respecto a él. Por ello la métrica más valiosa para evaluar el comportamiento de la cartera en las diferentes ventanas de testeo es el ratio de información. Es cierto que presenta una alta variabilidad entre ventanas sin embargo se mantiene siempre por encima de uno lo que indica una gestión muy eficiente de la cartera, no hay ninguna ventana donde el rendimiento del modelo sea preocupante.

Para nuestro entorno y cantidad de datos parece que es suficiente con las redes actor y critic del modelo PPO, los extractores de características han aportado mejoras estadísticamente despreciables. Para la optimización de hiperparámetros se va a seleccionar el modelo transformer con redes pequeñas. La razón de esta decisión es que, aunque no aporte mejoras significativas al modelo PPO sencillo, abre las puertas para implementar mejoras en el futuro como añadir más complejidad al entorno o mayor cantidad de datos. Además, presenta un buen equilibrio entre consumo de recursos computacionales, resultados y capacidad de mejora en el futuro. Por último es relevante mencionar que esta combinación de redes neuronales es un enfoque innovador ya que la literatura sobre su aplicación a optimización de carteras es escasa.

4.5. Selección del algoritmo de deep reinforcement learning a emplear en el modelo final

Para cada uno de los algoritmos se empleará la arquitectura seleccionada anteriormente, aunque hay que recalcar que por problemas de compatibilidad los algoritmos DDPG y SAC no contarán con la red LSTM. La optimización de hiperparámetros se hará con la librería Optuna

4.5.1. Optimización de hiperparámetros de los diferentes algoritmos de Deep Reinforcement Learning

4.5.1.1. PPO

Para el modelo PPO los hiperparámetros óptimos encontrados son

Tasa de aprendizaje: $3.3896027707827866e^{-06}$, coeficiente de entropía: 1.5210089527581027e⁰⁵, tamaño de batch: 512, n_steps: 768, gamma: 0.8587760378271281, clip_range: 0.14188380242544762, gae_lambda: 0.9208035456282112, max_grad_norm: 0.9562075837093198, vf_coef: 0.8139391556754874, número de epochs: 20.

Tabla 20 Métricas ejecución Walk Forward PPO con hiperparámtros óptimos

	Valor final	Sharpe	TE	IR	Max DD	rentabilidad anual	volatilidad anual
Prueba 1							
WF1	1,4355	0,6687	5,070%	1,3488	21,01%	15,04%	15,84%
WF2	1,5596	1,0009	4,260%	1,5729	29,04%	18,80%	14,34%
WF3	1,5429	0,4851	8,660%	1,0843	42,68%	18,31%	28,57%
WF4	1,8687	1,2022	5,850%	1,2948	32,70%	27,43%	19,11%
Prueba 2							
WF1	1,4353	0,6677	5,070%	1,3458	21,05%	15,04%	15,86%
WF2	1,5599	1,0017	4,260%	1,575	29,06%	18,81%	14,34%
WF3	1,5445	0,4867	8,650%	1,0899	42,69%	18,36%	28,57%
WF4	1,868	1,2014	5,860%	1,291	32,69%	27,41%	19,11%
Prueba 3							
WF1	1,4356	0,6685	5,070%	1,3498	21,02%	15,05%	15,85%
WF2	1,5602	1,0024	4,260%	1,5772	29,06%	18,82%	14,34%
WF3	1,5437	0,4861	8,640%	1,088	42,66%	18,33%	28,56%
WF4	1,8695	1,2027	5,860%	1,2963	32,72%	27,45%	19,13%
Prueba 4							

WF1	1,4359	0,6692	5,070%	1,3501	21,03%	15,06%	15,85%
WF2	1,5603	1,0026	4,260%	1,578	29,05%	18,82%	14,34%
WF3	1,5442	0,4862	8,660%	1,0881	42,68%	18,35%	28,58%
WF4	1,8691	1,2026	5,860%	1,2945	32,71%	27,44%	19,12%
Prueba 5							
WF1	1,4361	0,6698	5,070%	1,3511	21,01%	15,06%	15,85%
WF2	1,5602	1,0018	4,260%	1,5742	29,08%	18,82%	14,35%
WF3	1,543	0,4856	8,640%	1,0863	42,65%	18,31%	28,55%
WF4	1,8684	1,2021	5,850%	1,294	32,70%	27,42%	19,11%
MEDIA							
WF1	1,4357	0,6688	5,07%	1,3491	21,02%	15,05%	15,85%
WF2	1,5600	1,0019	4,26%	1,5755	29,06%	18,81%	14,34%
WF3	1,5437	0,4859	8,65%	1,0873	42,67%	18,33%	28,57%
WF4	1,8687	1,2022	5,86%	1,2941	32,70%	27,43%	19,12%

Fuente: Yahoo Finance, elaboración propia.

Tras las pruebas walk forward, con estos nuevos hiperparámetros el modelo sigue siendo robusto en todas las ventanas temporales. Las métricas de esta validación son algo inferiores, esto es porque para estos hiperparámetros el modelo tarda más tiempo en converger, para la evaluación final se incorporará un mayor número de iteraciones en el entrenamiento.

Prueba de rendimiento en ventana final con un mayor número de iteraciones de entrenamiento

4.5.1.2. DDPG

Para DDPG los hiperparámetros óptimos encontrados fueron:

Tasa de aprendizaje: $4.192833539324774e^{-05}$, tamaño de buffer: 100000, inicios de aprendizaje: 242, tamaño de batch: 64, tau: 0.017185391874565033, gamma: 0.9752566985333868, frecuencia de entrenamiento: 32, 'gradient_steps': 2, ruido de acción: 0.273109382957806, ruido de acción theta: 0.16795991217321057

Para este algoritmo que requiere de una potencia de computo y consumo de VRAM sustancialmente mayor que el algoritmo PPO se ha tenido que establecer un máximo de 512 en el tamaño de batch.

Tabla 21 Métricas ejecución Walk Forward DDPG con hiperparámetros óptimos

	Valor final	Sharpe	TE	IR	Max DD	Rentabilidad anual	Volatilidad anual
Prueba 1							
WF1	1,4058	0,6114	5,20%	1,1558	21,29%	14,12%	15,81%
WF2	1,5989	1,1033	4,20%	1,815	28,86%	19,96%	14,05%
WF3	1,5891	0,5317	8,28%	1,2724	41,25%	19,67%	28,62%
WF4	1,9265	1,3167	5,51%	1,5734	32,83%	28,94%	18,60%
Prueba 2							
WF1	1,3876	0,5643	5,21%	1,0664	23,46%	13,54%	16,11%
WF2	1,5649	1,0163	4,26%	1,6004	28,51%	18,96%	14,28%
WF3	1,4959	0,4412	8,62%	0,9368	41,20%	16,90%	28,11%
WF4	1,9077	1,2582	5,73%	1,4621	33,08%	28,45%	19,08%
Prueba 3							
WF1	1,4101	0,6273	4,96%	1,2308	20,66%	14,25%	15,63%
WF2	1,5437	0,9596	4,41%	1,4326	30,15%	18,33%	14,47%
WF3	1,5673	0,5212	7,52%	1,3057	41,30%	19,03%	27,98%
WF4	1,8149	1,1276	6,00%	1,0736	32,41%	25,99%	19,11%
Prueba 4							
WF1	1,3815	0,5568	5,15%	1,0416	22,72%	13,35%	15,98%
WF2	1,5979	1,0671	4,50%	1,7033	30,37%	19,93%	14,50%

WF3	1,5169	0,4590	8,57%	1,0152	42,43%	17,53%	28,50%
WF4	1,8902	1,2565	5,56%	1,4303	33,45%	28,00%	18,74%
Prueba 5							
WF1	1,4185	0,6036	5,75%	1,1322	22,18%	14,52%	16,68%
WF2	1,5347	0,9487	4,47%	1,3587	29,18%	18,06%	14,35%
WF3	1,5276	0,4693	8,54%	1,0526	41,82%	17,85%	28,56%
WF4	1,8592	1,1654	6,16%	1,2097	33,22%	27,18%	19,50%
MEDIA							
WF1	1,4007	0,5927	5,25%	1,1254	22,06%	13,96%	16,04%
WF2	1,5680	1,0190	4,37%	1,5820	29,41%	19,05%	14,33%
WF3	1,5394	0,4845	8,31%	1,1165	41,60%	18,20%	28,35%
WF4	1,8797	1,2249	5,79%	1,3498	33,00%	27,71%	19,01%

Fuente: Yahoo Finance, elaboración propia.

La prueba de validación walk forward muestra que el modelo es robusto en todas las ventanas temporales. No obstante, destaca la mayor variabilidad en los resultados respecto al algoritmo PPO. El modelo sigue siendo robusto, aunque en menor medida que el algoritmo PPO. Para una optimización de cartera es preferible estabilidad. De igual modo, esta prueba tan solo tiene el objetivo de evaluar la robustez del modelo.

4.5.1.3. SAC

1. Para SAC los hiperparámetros óptimos encontrados fueron:

Tasa de aprendizaje: 0.0002050979072640205, tamaño de buffer: 20000, tamaño de batch: 256, gamma: 0.8070153089533664, tau: 0.011857156937757057, train_freq: 16, gradient_steps: 1, coeficiente de entropía: $5.872379480181151e^{-05}$, target_update_interval: 1, use_sde: True, sde_sample_freq 32.

Tabla 22 Métricas ejecución Walk Forward SAC con hiperparámetros óptimos

	Valor final	Sharpe	TE	IR	Max DD	rentabilidad anual	volatilidad anual
Prueba 1							
WF1	1,4053	0,618	5,07%	1,1767	20,87%	14,10%	15,61%
WF2	1,5299	0,9567	4,13%	1,4311	27,86%	17,92%	14,08%
WF3	1,5240	0,4735	8,41%	1,0421	41,88%	17,74%	28,08%
WF4	1,8631	1,218	6,11%	1,2108	32,00%	27,28%	18,75%
Prueba 2							
WF1	1,4298	0,659	5,15%	1,2964	22,14%	14,87%	15,81%
WF2	1,5584	0,995	4,33%	1,5431	29,27%	18,77%	14,39%
WF3	1,5564	0,5003	8,75%	1,1092	42,64%	18,71%	28,50%
WF4	1,8602	1,171	5,83%	1,2793	32,37%	27,21%	19,43%
Prueba 3							
WF1	1,4365	0,6646	5,06%	1,3612	20,92%	15,08%	15,99%
WF2	1,5757	1,0444	4,23%	1,6739	28,14%	19,28%	14,20%
WF3	1,5573	0,4892	9,30%	1,0683	44,16%	18,74%	29,20%
WF4	1,7831	1,0888	5,82%	0,9861	31,21%	25,14%	19,00%
Prueba 4							
WF1	1,4311	0,6754	5,00%	1,331	19,93%	14,91%	15,48%
WF2	1,5334	0,9578	4,16%	1,4461	28,80%	18,03%	14,17%
WF3	1,5524	0,4837	9,30%	1,0569	44,23%	18,59%	29,24%
WF4	1,803	1,103	5,63%	1,1034	32,29%	25,67%	19,24%
Prueba 5							
WF1	1,4197	0,6475	4,92%	1,2954	20,41%	14,55%	15,61%
WF2	1,5841	1,0891	4,07%	1,7777	27,32%	19,52%	13,84%
WF3	1,5321	0,4821	8,24%	1,0884	41,38%	17,99%	28,08%

WF4	1,8273	1,1777	6,09%	1,0852	31,44%	26,33%	18,58%
MEDIA							
WF1	1,4245	0,6529	5,04%	1,2921	20,85%	14,70%	15,70%
WF2	1,5563	1,0086	4,18%	1,5744	28,28%	18,70%	14,14%
WF3	1,5444	0,4858	8,80%	1,0730	42,86%	18,35%	28,62%
WF4	1,8273	1,1517	5,90%	1,1330	31,86%	26,33%	19,00%

Fuente: Yahoo Finance, elaboración propia.

El modelo es robusto en todas las ventanas de validación aunque presenta mayor variabilidad en los resultados que el modelo PPO. Para una optimización de carteras es preferible la estabilidad. Se puede decir que el algoritmo ha superado la prueba de validación.

4.5.2. Pruebas de rendimiento finales de cada uno de los algoritmos

Tras superar las validaciones walk forward se entrenará a cada uno de los algoritmos con un número mayor de iteraciones con el objetivo de asegurar la convergencia de los modelos y obtener los mejores resultados posibles. Esta evaluación se realizará en la última ventana, el periodo de test 30 de junio de 2022 a 16 de mayo de 2025.

Tabla 23 Resultados ejecuciones PPO en la última ventana con 100.000 timesteps

	1	2	3	4	5	6	7	8	9	10	Media
Valor final	1,8685	1,8656	1,8690	1,8695	1,8679	1,8715	1,8688	1,8686	1,8722	1,8682	1,8690
Sharpe	1,2000	1,2000	1,2000	1,2000	1,2000	1,2100	1,2000	1,2000	1,2100	1,2000	1,2020
TE	5,86%	5,86%	5,85%	5,86%	5,86%	5,85%	5,85%	5,86%	5,85%	5,85%	5,86%
IR	1,2900	1,2800	1,3000	1,3000	1,2900	1,3100	1,2900	1,2900	1,3100	1,2900	1,2950
Max DD	32,71%	32,73%	32,72%	32,78%	32,68%	32,78%	32,69%	32,71%	32,79%	32,72%	32,73%
Rentabilidad anual	27,53%	27,35%	27,44%	27,45%	27,41%	27,50%	27,43%	27,43%	27,52%	27,42%	27,45%
Volatilidad anual	19,13%	19,13%	19,12%	19,14%	19,13%	19,13%	19,11%	19,12%	19,14%	19,13%	19,13%

Fuente: Yahoo Finance elaboración propia

Tabla 24 Resultados ejecuciones DDPG en la última ventana con 100.000 timesteps

	1	2	3	4	5	6	7	8	9	10	Media
Valor final	1,8746	1,8118	1,8242	1,8531	1,7939	1,8664	1,7972	1,9061	1,9026	1,8456	1,8476
Sharpe	1,2089	1,1303	1,1091	1,1803	1,0828	1,2373	1,1036	1,2264	1,2203	1,2130	1,1712
TE	6,06%	5,94%	6,46%	5,87%	6,00%	5,63%	6,74%	6,33%	5,63%	5,57%	6,02%
IR	1,2715	1,0703	1,0443	1,2366	1,0068	1,3184	0,8993	1,3313	1,4845	1,2523	1,1915
Max DD	33,98%	32,21%	32,72%	32,61%	32,76%	31,35%	31,43%	34,18%	34,04%	30,71%	32,60%
Rentabilidad anual	27,59%	25,91%	26,24%	27,02%	25,43%	27,37%	25,52%	28,41%	28,32%	26,82%	26,86%
Volatilidad anual	19,14%	18,99%	19,65%	19,12%	19,37%	18,58%	19,09%	19,54%	19,56%	18,44%	19,15%

Fuente: Yahoo Finance elaboración propia

Tabla 25 Resultados ejecuciones SAC en la última ventana con 100.000 timesteps

	1	2	3	4	5	6	7	8	9	10	Media
Valor final	1,8572	1,8609	1,8504	1,8589	1,7837	1,8103	1,8551	1,8815	1,7826	1,8956	1,8436
Sharpe	1,2000	1,1884	1,1704	1,1976	1,1069	1,1324	1,1488	1,2418	1,0877	1,2450	1,1719
TE	6,21%	5,79%	5,88%	5,76%	5,83%	6,12%	6,10%	5,90%	5,80%	6,07%	5,95%
IR	1,1800	1,2835	1,2270	1,2759	0,9777	1,0314	1,2140	1,3196	0,9872	1,3360	1,1832
Max DD	32,20%	33,31%	33,45%	32,11%	30,25%	31,10%	33,73%	31,82%	30,77%	31,99%	32,07%
Rentabilidad anual	27,13%	27,22%	26,95%	27,17%	25,15%	25,87%	27,07%	27,77%	25,12%	28,14%	26,76%

Volatilidad anual	18,96%	19,16%	19,22%	18,97%	18,70%	18,92%	19,69%	18,78%	19,01%	19,03%	19,04%
-------------------	--------	--------	--------	--------	--------	--------	--------	--------	--------	--------	---------------

Fuente: Yahoo Finance elaboración propia

El algoritmo PPO ha mostrado, de media, un mejor comportamiento en las pruebas, además presenta resultados mas estables. DDPG y SAC oscilan significativamente en sus resultados y obtienen peores resultados. Es cierto que en algunas pruebas presentan resultados destacables, sería conveniente estudiar el por qué de estas oscilaciones para potenciar estos resultados, tarea muy demandante en tiempo que no cabe lugar en este trabajo.

Como decisión final, se empleará el algoritmo PPO tanto por su robustez en las distintas ventanas walk forward como por su robustez ante la mínima varianza en sus resultados y su buen desempeño.

5. Resultados

5.1.Resultados del modelo final

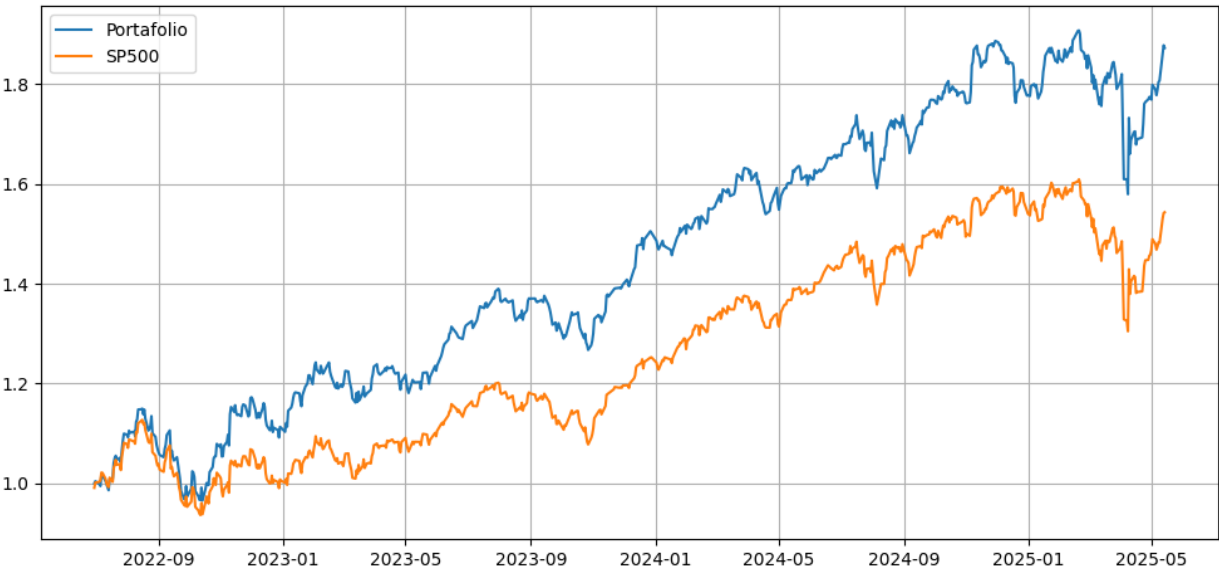
Para presentar los resultados se ha tomado la media de las ejecuciones del algoritmo PPO del apartado anterior excepto para los gráficos, estos corresponden a la ejecución número 7.

Tabla 26 Métricas de evaluación del modelo final vs S&P500

	Cartera	S&P500
Riqueza final	1,8690	1,5430
Sharpe	1,202	0,7586
TE	5,86%	
IR	1,295	
Max DD	32,73%	18,90%
Rentabilidad anual	27,45%	18,32%
Volatilidad anual	19,13%	18,32%

Fuente: Yahoo Finance, elaboración propia

Gráfico 6 Evolución de la rentabilidad acumulada de la cartera vs S&P500

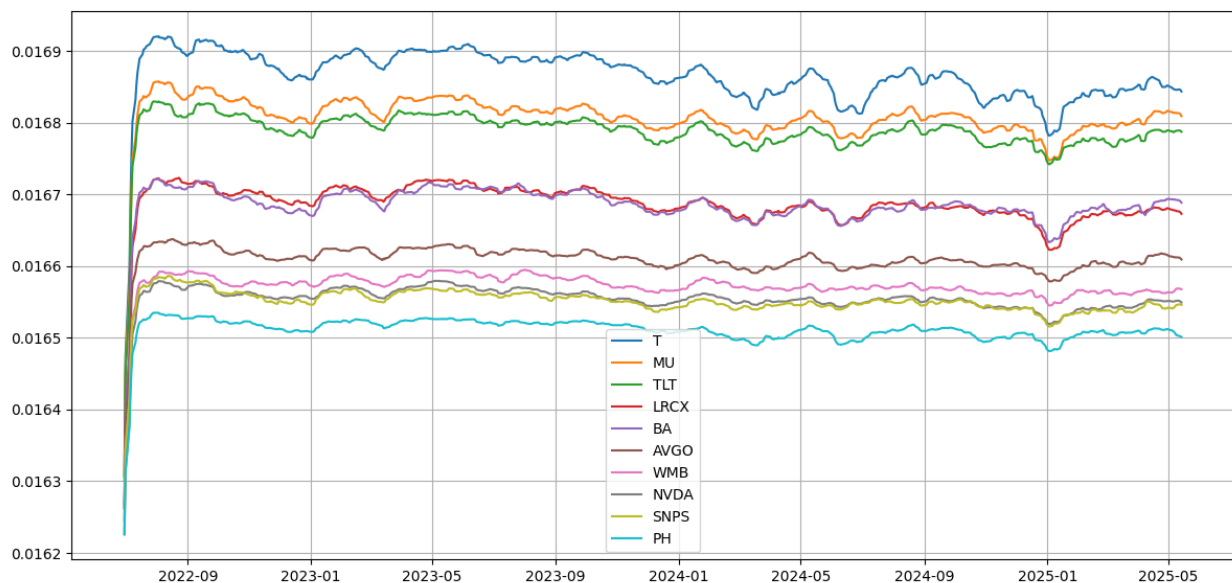


La cartera supera con creces el rendimiento del índice S&P500, además mejora la rentabilidad por riesgo medida por el ratio de Sharpe, por cada unidad de riesgo que se añade se obtiene 1,2 unidades de rentabilidad, una cifra que supera con creces al ratio de sharpe del índice, 0,7586.

Esto ocurre porque la volatilidad de la cartera apenas aumenta respecto al índice mientras que la rentabilidad se dispara.

De igual modo se obtiene una cifra excelente en el ratio de información. Este ratio nos indica que por cada unidad de riesgo asumida por encima del índice se generan 1,29 unidades de rentabilidad, cifras de este ratio por encima de la unidad indican una gestión muy eficiente de la cartera. Además esta métrica es especialmente útil ya que contextualiza la cartera, el universo de acción de la cartera son acciones del S&P500 por lo tanto la cartera estará muy correlacionada con el índice, por eso poner en relación la cartera directamente contra el S&P500 aporta verdadero valor comparativo.

Gráfico 7 Evolución de los pesos en cartera del top 10 activos en cartera al final del periodo



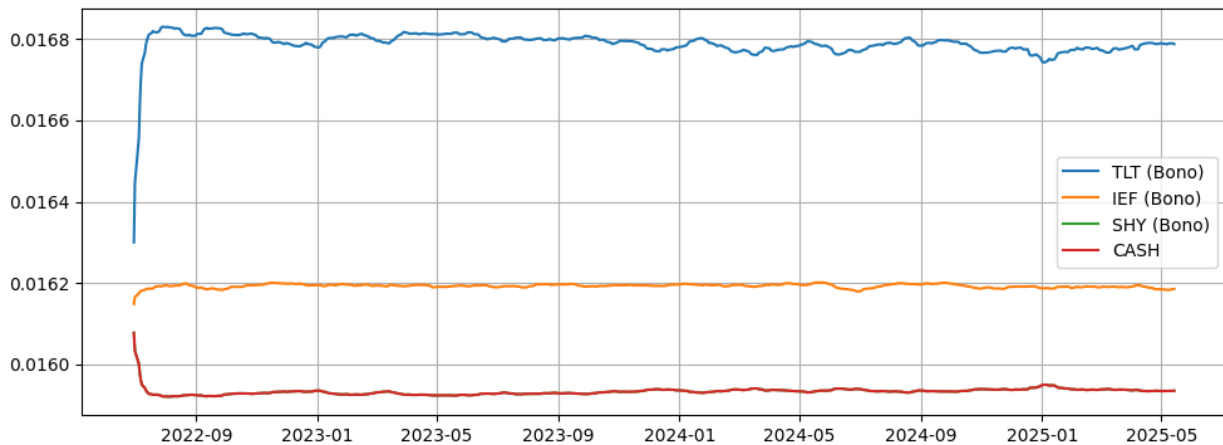
Fuente: elaboración propia

Para facilitar la visualización tan solo se muestran en el gráfico la evolución de los diez activos con mayor peso en la cartera al final del periodo, graficar la evolución de 62 activos hace la lectura imposible.

El modelo ha optado por mantener una asignación de pesos relativamente estable en el tiempo realizando pequeños ajustes a lo largo del periodo. Esta es la forma en la que el modelo, teniendo en cuenta los costes de transacción, ha maximizado la rentabilidad por riesgo de la cartera.

El brusco incremento de los pesos al inicio del periodo se debe a que para el inicio de cada simulación la asignación de pesos es estocástica y el modelo ha ajustado los pesos al empezar

Gráfico 8 Evolución del peso en cartera de los ETF de bonos del tesoro y efectivo



Fuente: elaboración propia

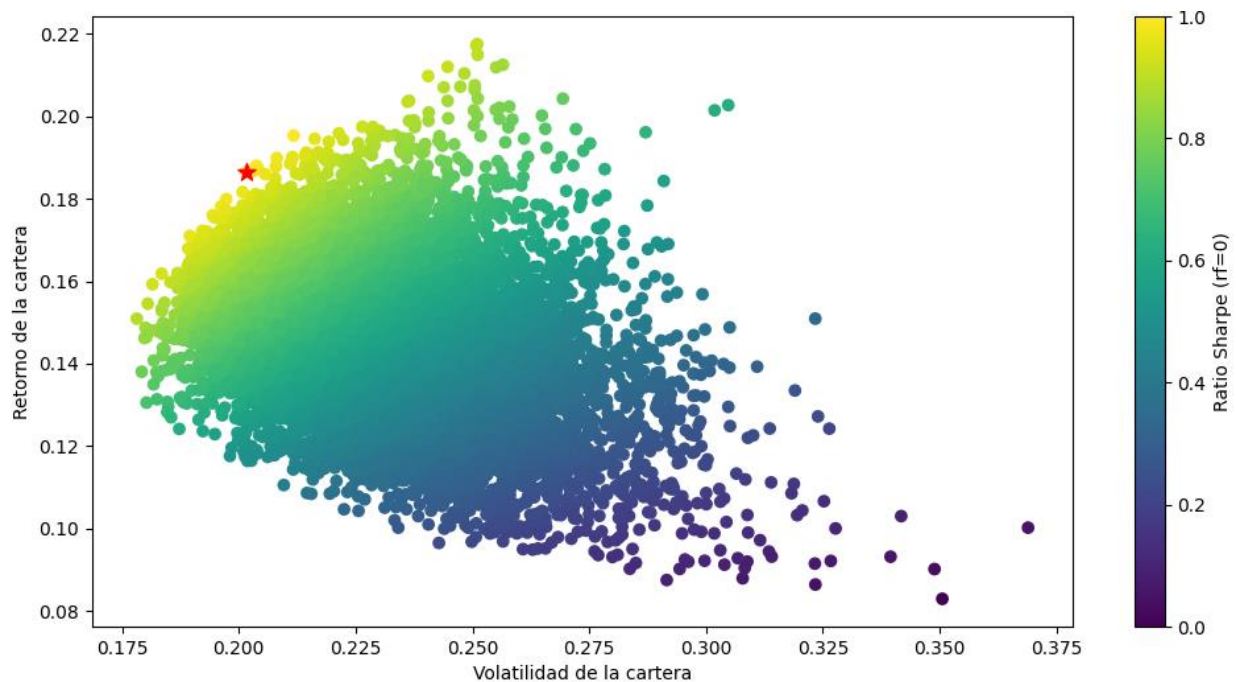
Analizar la evolución de estos tres últimos activos es interesante pues permite inferir el comportamiento del agente para cubrirse de volatilidad, por ejemplo, en previsión de una caída cubrirse vendiendo y acumulando efectivo o bonos. No obstante el modelo opta por seguir una política similar a las del resto de activos.

5.2. Comparación contra optimización por Markowitz

Para realizar esta comparación se optimizará una cartera empleando el método de Markowitz. Para emular la misma lógica que el modelo desarrollado en este trabajo, se construirá una cartera óptima empleando datos del periodo de entrenamiento de nuestro modelo, 3 de enero de 2012 a 30 de junio de 2022. Markowitz presenta como limitante el número de acciones a optimizar, un número elevado de activos imposibilita su ejecución. Para solucionar este problema se optimizará eligiendo el activo más representativo de cada clúster del clustering planteado en la sección cuatro de este trabajo.

Se optimizará el ratio de Sharpe ya que la función de recompensa de nuestro modelo maximiza la rentabilidad teniendo en cuenta la desviación típica, por lo que este método es el que más se asemeja.

Gráfico 9 Optimización ratio de Sharpe por Markowitz



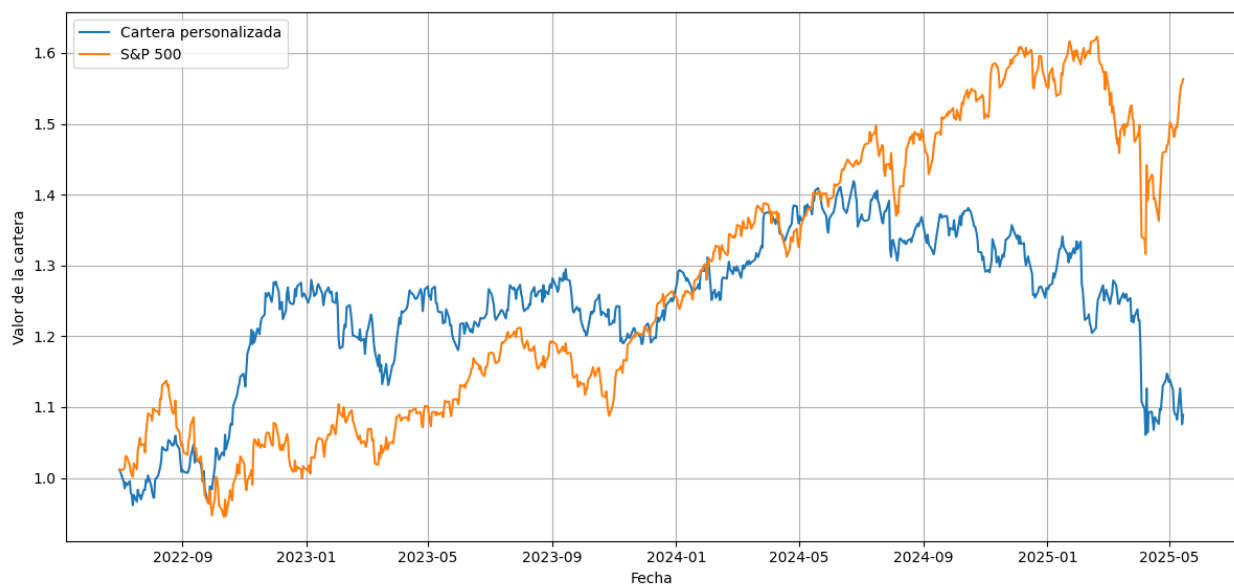
Fuente: Yahoo Finance, elaboración propia, código Sara Diaz.

Composición de la cartera óptima:

APD (0.0%), FTNT (24.03%), MRK, (25.34%), RCL (0.0%), AAPL, (50.629%), EOG (0.0%)

Una vez tenemos la cartera óptima procedemos a evaluarla en el periodo de test, 30 de junio de 2022 a 16 de mayo de 2025.

Gráfico 10 Evolución cartera Sharpe óptimo Markowitz



Fuente: Yahoo Finance, elaboración propia.

La cartera óptima construida por Markowitz obtiene una rentabilidad acumulada del 8,92%, una volatilidad del 17,89% y un ratio de Sharpe de 0,0089 durante el periodo de test. Lo cual es inferior a lo obtenido por el modelo de deep reinforcement learning. Se hace presente la

superioridad del modelo al poder adaptar pesos en la cartera según varían las condiciones del mercado.

Se puede argumentar que existe la posibilidad de hacer rebalanceos a lo largo del periodo de test. Se ha construido una cartera que, tras optimizar por Markowitz, rebalancea los pesos seis veces a lo largo del periodo. Además, se recortan los datos históricos que se utilizan para realizar la optimización de modo que los cambios recientes en el mercado tengan un mayor impacto. Ahora la optimización se realiza desde enero de 2020 a junio de 2022.

Gráfico 11 Evolución cartera sharpe óptimo Markowitz con 6 rebalanceos



Fuente: Yahoo Finance, elaboración propia

Tabla 27 Métricas evaluación cartera sharpe óptimo Markowitz con 6 rebalanceos vs cartera del modelo

	Markowitz con rebalanceo	Cartera modelo
Valor final	1,0276	1,8690
Sharpe	-0,1845	1,2020
TE	19,10%	5,86%
IR	-0.7523	1,2950
Máximo drawdown	33.86%	32,73%
Rentabilidad anual	0.9523%	27,45%
Volatilidad anual	18,87%	19,13%

Fuente: Yahoo Finance, elaboración propia.

Tabla 28 Asignación de pesos en cartera sharpe optimo Markowitz con rebalanceos

	APD	FTNT	MRK	RCL	AAPL	EOG
Rebalanceo 0	0%	24,03%	25,34%	0%	50,63%	0%
Rebalanceo 1	0%	31,9%	0,53%	0%	53,64%	13,93%
Rebalanceo 2	0%	38,89%	13,49%	0%	47,62%	0%
Rebalanceo 3	0%	22,24%	0%	0%	75,82%	1,94%
Rebalanceo 4	0%	23,64%	29,80%	0%	46,55%	0%
Rebalanceo 5	0%	4,13%	17,38%	0%	77,09%	1,41%
Rebalanceo 6	0%	29,56%	0%	0%	70,44%	0%

Fuente: Yahoo Finance, elaboración propia

Incluso sin tener en cuenta costes de transacción la cartera construida por Markowitz incluyendo rebalanceos tiene un desempeño pobre y muy lejano al modelo planteado en este trabajo, además no mejora a la cartera construida sin rebalanceos.

Las ventajas de la capacidad de rebalanceo diario y modelaje de series temporales de nuestro modelo se hacen evidentes en estos resultados. Markowitz presenta limitaciones fuertes ante el dinamismo del mercado mientras que el modelo desarrollado en este trabajo consigue superar este reto.

5.3. Comparación contra fondos de inversión con objetivo y benchmark similar

Se han escogido fondos de inversión similares al modelo planteado en este trabajo, con mismo benchmark y/o objetivo y activos en cartera similares. En el anexo se recogen las estrategias y objetivos de cada fondo obtenidos del fact sheet de cada uno de ellos.

Tabla 29 Métricas de rendimiento de fondos de inversión

	Rentabilidad anual últimos 3 años	Volatilidad	Beta	Sharpe	TE	IR
Alger focus Equity Fund	12,31%	21,66%*	1,04*	0,8*	-	-0,13*
Polen Capital Focus US Growth Fund	6,65%	-	1,04	0,69	-	-0,06
BlackRock Advantage US Equity Fund	10,17%	15,52%	0,99	0,36	-	-
BlackRock BGS Flexible Equity Fund	5,08%	15,68%	0,88	0,03	-	-
BlackRock BGS Basic Value Fund	4,35%	14,48%	0,83	-0,01	-	-
Fidelity American Growth Fund	0,50%	13,27%	0,7	-0,15	8,31%	-1,01
Fidelity Fund A-ACC-Euro	-0,30%	15,35%	0,8	-0,19	8,98%	-1,03
JP Morgan large cap growth fund	11,37%	-	0,94	0,46	-	-
JP Morgan US equity fund	8,04%	-	0,98	0,32	-	-
Goldman Sachs US Enhanced Equity	12,29%	16,66%	1,02	0,47	1,98%	0,32
Goldman Sachs US Core Equity Portfolio	7,75%	15,52%	0,9	0,20	5,62%	-0,70
Goldman Sachs US Core Equity Portfolio	11,01%	16,19%	0,99	0,40	2,14%	-0,30
Vanguard Growth and Income Fund Admiral Shares	11,96%	-	-	-	-	-
Vanguard Growth and Income Fund Investor Shares	11,85%	-	-	-	-	-
Cartera del modelo	27,45%	19,13%	-	1,2020	5,86%	1,2950

* Últimos 5 años

Fuente: Alger Focus, Polen Capital, BlackRock, Fidelity, JP Morgan, Goldman Sachs y Vanguard.

Los colores indican el benchmark de cada fondo, naranja S&P500, gris MCSI USA, azul Russel1000 index/growth/value. Las rentabilidades de la tabla son netas de comisiones corrientes, estas oscilan en torno a 0,4 y 1% anual.

Los desempeños de los fondos son inferiores y lejanos al rendimiento de la cartera construida por el modelo. La eficiencia en la gestión del modelo supera con creces a los fondos de inversión con objetivos similares.

6. Conclusiones y posibles vías de mejora del modelo

El objetivo de este trabajo era el de construir una cartera óptima prescindiendo de métodos tradicionales para emplear una tecnología disruptiva como lo es la inteligencia artificial, en concreto el deep reinforcement learning. Los métodos tradicionales, la gran mayoría contruidos

sobre la base establecida por Harry Markowitz en 1952, sufren de limitaciones importantes a la hora de optimizar carteras.

El supuesto de linealidad y los modelos estáticos comprometen el desempeño de las carteras, en este trabajo se ha tratado de atajar estos problemas. El deep reinforcement learning ha demostrado una elevada eficiencia a la hora de gestionar carteras, ha superado el supuesto de linealidad y ha mostrado una gran capacidad de adaptación a los cambios en un entorno dinámico, el mercado. Se ha demostrado una fuerte robustez del modelo en las distintas ventanas de validación walk forward. En contraparte, la cartera compuesta por optimización de Markowitz ha hecho evidente sus carencias, imposibilidad de incluir gran variedad de activos, incapacidad de adaptación al entorno y por ende resultados muy pobres, lejos de los obtenidos por nuestro modelo. Por otro lado, los fondos de inversión, algunos gestionados mediante análisis fundamental, otros cuantitativo o combinación de ambos, han demostrado desempeños superiores que la cartera construida por optimización de Markowitz pero de nuevo significativamente inferiores al modelo planteado en este trabajo. Sin duda la implementación de inteligencia artificial ha aportado mucho valor a la hora de gestionar carteras.

Del mismo modo es importante señalar las limitaciones de hardware que se han sufrido en el desarrollo del trabajo. El consumo de memoria VRAM y el coste computacional del entrenamiento de ciertas arquitecturas y algoritmos de aprendizaje por refuerzo ha hecho necesario el recorte de estos. La demanda de recursos computacionales es un factor limitante en el desarrollo de estos modelos, aspecto que se ha evidenciado en este trabajo.

A pesar de obtener buenos resultados es posible seguir mejorando el modelo. Enlazando con el párrafo anterior, una de las posibles vías de mejora del modelo es, contando con un hardware más capaz, diseñar un entorno más complejo, con más variables explicativas o incorporación de activos de distintos mercados. Además, disponer de datos con una frecuencia mayor a la diaria, por ejemplo, datos horarios o a minutos, puede potenciar el desempeño del modelo y aprovechar las arquitecturas más potentes.

Otra vía de mejora es realizar una selección más exhaustiva de variables explicativas, esta tarea es muy intensiva en tiempo en el contexto en el que nos encontramos, evaluar individualmente el efecto marginal de añadir variables es fundamental para potenciar el rendimiento del modelo. Como añadidura existen métodos como SHAP o LIME que ayudan a interpretar las decisiones que toman las redes neuronales por lo que pueden ser herramientas que aporten un alto valor añadido en la selección de variables explicativas.

Otras ideas que no se han implementado en el modelo y siguiendo el enfoque de la inteligencia artificial, es desarrollar variables explicativas a través de redes neuronales. Incorporar análisis de sentimiento mediante modelos de procesamiento de lenguaje natural puede aportar señales valiosas aprovechables por el modelo. De igual modo, podría estudiarse la incorporación a las variables explicativas de una predicción de precios de cada activo para el periodo siguiente empleando redes neuronales. Sumado a lo anterior, pese a que no es un método de inteligencia artificial, extender el análisis de cointegración a las variables explicativas de cada activo puede tener efectos enriquecedores en el modelo aún más sabiendo que el impacto del análisis de cointegración fue muy positivo.

En paralelo a lo anterior se puede sustituir la asignación estocástica inicial de los pesos de los activos, por ejemplo, antes de poner en marcha el modelo se puede realizar un análisis fundamental y establecer unos pesos de partida basados en este análisis.

En último lugar merece la pena destacar la aplicabilidad de estos modelos a la práctica profesional, los fondos de inversión, bancos y aseguradoras pueden aprovechar las ventajas de la gestión dinámica de estos modelos especialmente considerando que su limitación computacional es mucho menor a la de un ordenador particular como el que se ha utilizado para el desarrollo de

este trabajo. Del mismo modo que esta tecnología se aplica a la gestión de carteras de valores puede tener aplicabilidad en otros ámbitos de las finanzas como puede ser el control de gestión empresarial.

7. Referencias

- Haarnoja, T., Zhou, A., Abbeel, P. & Levine, S., 2018. *Soft Actor-Critic:: Off-policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*, s.l.: arXiv preprint arXiv:1801.01290v2.
- Krokhmal, P., Palmquist, J. & Uryasev, S., 2002. Portfolio Optimization with Conditional Value-at-Risk Objective and Constraints. *Journal of Risk*, Volumen 4, pp. 43-68.
- Abdi, H. & Williams, L. J., 2010. Principal component analysis. *WIREs Computational Statistics*, 2(4), pp. 433-459.
- Akiba, T. y otros, 2019. *Optuna: A Next-generation Hyperparameter Optimization Framework*, s.l.: arXiv preprint arXiv:1907.10902v1.
- Allen, D. E. & McAleer, M., 2020. A nonlinear autoregressive distributed lag (NARDL) analysis of west texas intermediate oil prices and the DOW JONES index. *Energies*, 13(15).
- Alpaydin, E., 2014. *Introduction to Machine Learning*. tercera ed. Londres: The MIT Press.
- Anghel, G. D. I., 2015. Stock Market Efficiency and the MACD. Evidence from Countries around the World. *Procedia Economics and Finance*, Volumen 32, pp. 1414-1431.
- Bank, D., Koenigstein, N. & Giryas, R., 2020. *Autoencoders*, s.l.: arXiv preprint arXiv:2003.05991v2.
- Baur, D. G. & McDermott, T. K., 2010. Is gold a safe haven? International evidence. *Journal of Banking & Finance*, 34(8), pp. 1886-1898.
- Black, F. & Litterman, R., 1992. Global Portfolio Optimization. *Financial Analysts Journal*, 48(5), pp. 28-43.
- Cai, Q., Pan, L. & Tang, P., 2018. *Deterministic Policy Gradients With General State Transitions*, s.l.: arXiv arXiv:1807.03708v3.
- Caruana, R., Lawrence, S. & Giles, L., 2000. *Overfitting in Neural Nets: Backpropagation, Conjugate Gradient, and Early Stopping*. s.l., s.n.
- Dwyer, G. P., 2015. *The Johansen Tests for Cointegration*, s.l.: s.n.
- Fujimoto, S., van Hoof, H. & Menger, D., 2018. *Addressing Function Approximation Error in Actor-Critic Methods*, s.l.: arXiv preprint arXiv:1802.09477v3.
- Gaivoronski, A. A. & Pflug, G., 2005. Value-at-Risk in Portfolio Optimization: Properties and Computational Approach. *Journal of Risk*, 7(2), pp. 1-31.
- Ghobadi, M., 2014. "Profitability of Technical Analysis Indicators to Earn Abnormal Returns in International Exchange Markets. *Journal of Economics, Finance & Accounting (JEFA)*, 1(4), pp. 334-346.
- Goodfellow, I., Bengio, Y. & Courville, A., 2016. *Deep Learning*. Cambridge: MIT Press.
- Gujarati, D. N. & Porter, D. C., 2013. *Econometría*. 5 ed. s.l.:Mc Graw Hill.
- Gunjan, A. & Bhattacharyya, S., 2023. A brief review of portfolio optimization techniques. *Artificial Intelligence Review*, 56(5), pp. 3847-3886.
- Hieu, L. T., 2020. Deep Reinforcement Learning for Stock Portfolio Optimization. *International Journal of Modeling and Optimization*, Volumen 10, pp. 139-144.

- Huang, S.-H., Miao, Y.-H. & Hsiao, Y.-T., 2021. Novel Deep Reinforcement Algorithm With Adaptive Sampling Strategy for continuous Portfolio Optimization. *IEEE Access*, Volumen 9, pp. 77371-77385.
- Konno, H. & Hiroaki, Y., 1991. Mean-Absolute Deviation Portfolio Optimization Model and Its Applications to Tokyo Stock Market. *Management Science*, mayo, 37(5), pp. 519-531.
- Liang, . Z. y otros, 2018. *Adversarial Deep Reinforcement Learning in Portfolio Managment*, s.l.: arXiv preprint arXiv:1808.09940v3.
- Markowitz, H., 1952. Portfolio Selection. *The Journal of Finance*, Marzo, 7(1), pp. 77-91.
- Michelucci, U., 2022. *An Introduction to Autoencoders*, s.l.: arXiv:2201.03898v1.
- Mnih, V. y otros, 2013. *Playing Atari with Deep Reinforcement Learning*, s.l.: arXiv preprint arXiv:1312.5602v1.
- O'Shea, K. & Nash, R., 2015. *An Introduction to Convolutional Neural Networks*, s.l.: arXiv preprint arXiv:1511.08458v2.
- Obeidat, S., Shapiro, D. & Lemay, M., 2018. Adaptive Portfolio Asset Allocation Optimization with Deep Learning. *International Journal on Advances in Intelligent Systems*, 11(1), pp. 25-34.
- Olorunnimbe, K. & Viktor, H., 2023. Deep learning in the stock market—a systematic survey of practice, backtesting and applications. *Artificial Intelligence Review*, Volumen 56, pp. 2057-2109.
- Palomar, D. P., 2025. *Portfolio Optimization Theory and Application*. Cambridge: Cambridge University Press.
- Potters, C., 2024. *Investopedia*. [En línea] Available at: <https://www.investopedia.com/articles/trading/07/adx-trend-indicator.asp> [Último acceso: 24 mayo 2025].
- Qiang Cai, X., Lay Teo, K., Yang, X. & Zhou, X., 2004. Minimax portfolio optimization: Empirical numerical study. *Journal of the Operational Research Society*, 55(1), pp. 65-72.
- Rummery, G. & Niranjan, M., 1994. *On-line Q-Learning using connectionist systems*. Cambridge: University of Cambridge.
- Schulman, J. y otros, 2017. *Proximal Policy Optimization Algorithms*, s.l.: arXiv preprint arXiv:1707.06347v2 .
- Sharpe, W. F., 1964. Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk. *The Journal of Finance*, 19(3), pp. 425-442.
- Silver, D. y otros, 2014. *Deterministic Policy Gradient Algorithms*, Londres: s.n.
- Silver, D. y otros, 2016. *Continuous Control with Deep Reinforcement Learning*. Beijing, s.n.
- Soleymani, F. & Paquet, E., 2020. Financial Portfolio Optimization with Online Deep Reinforcement learning and Restricted Stacked Autoencoder-DeepBreath. *Expert Systems with Applications*, Volumen 156.
- Song, R., Xiao, Y., Zhang, S. & Wang, J., 2020. A Review on Graph Neural Network Methods in Financial Applications. *Journal of Data Science*, pp. 1-32.
- Srivastava, N. y otros, 2014. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research 15*, Volumen 15, pp. 1929-1958.

Staudemeyer, R. C. & Rothstein Morris, E., 2019. *Understanding LSTM a tutorial into Long Short-Term Memory Recurrent Neural Networks*, s.l.: arXiv preprint arXiv:1909.09586v1.

Sutton, R. S. & Barto, A. G., 2018. *Reinforcement Learning: An introduction*. segunda ed. Londres: The MIT Press.

Vaswani, A. y otros, 2023. *Attention Is All You Need*, s.l.: arXiv preprint arXiv:1706.03762v7.

Watkins, C. J. & Dayan, P., 1992. Q-Learning. *Machine Learning*, Volumen 8, pp. 279-292.

Williams, R. J., 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, Volumen 8, pp. 229-256.

Yañez, C., Kristjanpoller, W. & Minutolo, M. C., 2024. Stock market index prediction using transformer neural network models and frequency decomposition. *Neural Computing and Applications*, 36(25), pp. 15777-15797.

Zhang, Z., Zohren, S. & Roberts, S., 2020. *Deep Learning for Portfolio Optimization*, s.l.: s.n.

8. Anexo

8.1. Distribuciones rentabilidades diarias valores S&P500

Tabla 30 P-valor contraste de Jarque Bera para los retornos diarios de las 100 acciones de mayor capitalización del S&P500 a marzo de 2025. 2012-2025

	p-valor	Normalidad
AAPL	0	Distribución no normal
ABBV	0	Distribución no normal
ABT	1.53E-146	Distribución no normal
ACN	7.33E-138	Distribución no normal
ADBE	0	Distribución no normal
ADP	0	Distribución no normal
AMAT	4.93E-34	Distribución no normal
AMD	3.09E-150	Distribución no normal
AMGN	0	Distribución no normal
AMT	7.77E-68	Distribución no normal
AMZN	8.54E-256	Distribución no normal
AVGO	0	Distribución no normal
AXP	0	Distribución no normal
BA	4.61E-139	Distribución no normal
BAC	0	Distribución no normal
BKNG	0	Distribución no normal
BLK	0	Distribución no normal
BMJ	0	Distribución no normal
BRK-B	2.56E-144	Distribución no normal
BSX	3.93E-166	Distribución no normal
BX	1.42E-113	Distribución no normal
C	0	Distribución no normal
CAT	7.11E-112	Distribución no normal
CB	3.92E-70	Distribución no normal
CMCSA	0	Distribución no normal
CME	1.24E-128	Distribución no normal
COP	2.52E-113	Distribución no normal
COST	0	Distribución no normal
CRM	0	Distribución no normal

CSCO	0	Distribución no normal
CVX	1.32E-150	Distribución no normal
DE	1.19E-263	Distribución no normal
DHR	7.67E-185	Distribución no normal
DIS	0	Distribución no normal
DUK	6.53E-09	Distribución no normal
ELV	3.99E-299	Distribución no normal
ETN	0	Distribución no normal
FI	1.50E-238	Distribución no normal
GE	4.90E-109	Distribución no normal
GILD	0	Distribución no normal
GOOG	7.17E-105	Distribución no normal
GOOGL	8.06E-98	Distribución no normal
GS	0	Distribución no normal
HD	1.06E-102	Distribución no normal
HON	2.42E-211	Distribución no normal
IBM	0	Distribución no normal
INTU	2.22E-65	Distribución no normal
ISRG	0	Distribución no normal
JNJ	6.66E-222	Distribución no normal
JPM	0	Distribución no normal
KO	2.32E-218	Distribución no normal
LIN	6.72E-117	Distribución no normal
LLY	0	Distribución no normal
LMT	0	Distribución no normal
LOW	4.61E-120	Distribución no normal
MA	1.17E-197	Distribución no normal
MCD	6.35E-69	Distribución no normal
MDT	4.14E-143	Distribución no normal
META	0	Distribución no normal
MMC	1.60E-88	Distribución no normal
MO	0	Distribución no normal
MRK	0	Distribución no normal
MS	3.49E-170	Distribución no normal
MSFT	4.65E-81	Distribución no normal
NEE	8.30E-151	Distribución no normal
NFLX	0	Distribución no normal
NOW	1.97E-147	Distribución no normal
NVDA	2.98E-201	Distribución no normal
ORCL	0	Distribución no normal
PANW	0	Distribución no normal
PEP	8.96E-78	Distribución no normal
PFE	2.03E-123	Distribución no normal
PG	7.88E-142	Distribución no normal
PGR	0	Distribución no normal
PLD	8.07E-119	Distribución no normal
PLTR	0	Distribución no normal
PM	0	Distribución no normal
QCOM	5.88E-144	Distribución no normal
RTX	0	Distribución no normal
SBUX	0	Distribución no normal
SCHW	1.13E-266	Distribución no normal
SO	3.16E-13	Distribución no normal
SPGI	7.08E-194	Distribución no normal
SYK	1.13E-195	Distribución no normal

T	0	Distribución no normal
TJX	0	Distribución no normal
TMO	2.78E-80	Distribución no normal
TMUS	3.28E-256	Distribución no normal
TSLA	3.03E-102	Distribución no normal
TXN	0	Distribución no normal
UBER	1.48E-115	Distribución no normal
UNH	1.81E-248	Distribución no normal
UNP	0	Distribución no normal
V	0	Distribución no normal
VRTX	0	Distribución no normal
VZ	0	Distribución no normal
WELL	0	Distribución no normal
WFC	6.17E-137	Distribución no normal
WMT	0	Distribución no normal
XOM	9.50E-74	Distribución no normal

Fuente: Yahoo Finance, elaboración propia.

8.2. VECM tipos de interés, tipos de cambio, S&P500, oro y barril WTI.

Sistema VECM, orden del retardo 2

Estimaciones de Máxima Verosimilitud, observaciones 2012-01-05-2022-06-30 (T = 2736)

Rango de cointegración = 4

Caso 3: Constante no restringida

beta (Vectores cointegrantes, Desviaciones típicas entre paréntesis)

dgs3mo	1.0000	0.0000	0.0000	0.0000
	(0.0000)	(0.0000)	(0.0000)	(0.0000)
dgs1	0.0000	1.0000	0.0000	0.0000
	(0.0000)	(0.0000)	(0.0000)	(0.0000)
dgs2	0.0000	0.0000	1.0000	0.0000
	(0.0000)	(0.0000)	(0.0000)	(0.0000)
dgs5	0.0000	0.0000	0.0000	1.0000
	(0.0000)	(0.0000)	(0.0000)	(0.0000)
dgs10	-3.7795	-3.8374	-3.3413	-1.8030
	(0.39076)	(0.34190)	(0.23051)	(0.099781)
dbaa	-0.65886	-0.69426	-0.72633	-0.43878
	(0.41210)	(0.36057)	(0.24310)	(0.10523)
daaa	2.0993	2.1631	2.3683	1.8005
	(0.64750)	(0.56653)	(0.38196)	(0.16534)
wti	0.029978	0.031719	0.020393	-0.0042134
	(0.0076916)	(0.0067297)	(0.0045373)	(0.0019640)
CNYUSDx	14.314	15.894	16.982	14.007
	(16.542)	(14.473)	(9.7581)	(4.2240)
eurusd	1.8335	2.1094	2.0012	0.68903
	(1.5493)	(1.3556)	(0.91395)	(0.39562)
gold	-0.0040236	-0.0041828	-0.0026372	0.00081149
	(0.00080895)	(0.00070779)	(0.00047720)	(0.00020657)
S&P500	0.00052400	0.00058242	0.00049115	0.00011559
	(0.00021161)	(0.00018514)	(0.00012483)	(5.4034e-05)

alpha (vectores de ajuste)

dgs3mo	-0.022388	0.027922	-0.0094330	-0.0035052
--------	-----------	----------	------------	------------

dgs1	0.0021609	-0.043209	0.059762	-0.026000
dgs2	-0.021706	0.028476	-0.0027628	-0.0010423
dgs5	-0.018275	0.015873	0.016884	-0.013938
dgs10	-0.022765	0.027213	0.0080256	-0.014978
dbaa	-0.0069472	-0.015735	0.046260	-0.020068
daaa	-0.026011	0.0088103	0.035643	-0.047043
wti	0.23592	-0.50689	0.29872	0.045458
CNYUSDX	0.00011185	-0.00029968	0.00025890	-0.00015919
eurusd	-0.0020396	0.0057326	-0.0054024	0.0013799
gold	-5.4213	9.1597	-3.4498	-4.3849
S&P500	0.60477	20.749	-30.355	15.788

Log-verosimilitud = 45643.361

Determinante de la matriz de covarianzas = 5.2487916e-30

AIC = -33.1457

BIC = -32.4972

HQC = -32.9114

Ecuación 1: d_dgs3mo

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	0.0356162	0.0156219	2.280	0.0227	**
d_dgs3mo_1	0.0377348	0.0213330	1.769	0.0770	*
d_dgs1_1	0.104674	0.0257478	4.065	<0.0001	***
d_dgs2_1	0.0737712	0.0278269	2.651	0.0081	***
d_dgs5_1	-0.0251645	0.0343360	-0.7329	0.4637	
d_dgs10_1	-0.0118804	0.0337102	-0.3524	0.7245	
d_dbaa_1	-0.00226675	0.0225051	-0.1007	0.9198	
d_daaa_1	-0.0219786	0.0169865	-1.294	0.1958	
d_wti_1	0.000452416	0.000203679	2.221	0.0264	**
d_CNYUSDX_1	-0.281183	1.16144	-0.2421	0.8087	
d_eurusd_1	-0.0162370	0.0697629	-0.2327	0.8160	
d_gold_1	1.26851e-05	2.84226e-05	0.4463	0.6554	
d_S&P500_1	-5.11247e-05	1.44437e-05	-3.540	0.0004	***
EC1	-0.0223877	0.00413182	-5.418	<0.0001	***
EC2	0.0279223	0.0107993	2.586	0.0098	***
EC3	-0.00943302	0.0104705	-0.9009	0.3677	
EC4	-0.00350517	0.00542988	-0.6455	0.5186	

Media de la vble. dep.	0.000621	D.T. de la vble. dep.	0.021630
Suma de cuad. residuos	1.171156	D.T. de la regresión	0.020762
R-cuadrado	0.084708	R-cuadrado corregido	0.078644
rho	0.005286	Durbin-Watson	1.984196

Ecuación 2: d_dgs1

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-0.0151843	0.0175867	-0.8634	0.3880	
d_dgs3mo_1	0.258785	0.0240161	10.78	<0.0001	***
d_dgs1_1	-0.0617390	0.0289862	-2.130	0.0333	**
d_dgs2_1	0.0345492	0.0313268	1.103	0.2702	

d_dgs5_1	0.0430557	0.0386545	1.114	0.2654	
d_dgs10_1	-0.0186844	0.0379501	-0.4923	0.6225	
d_dbaa_1	-0.0332483	0.0253357	-1.312	0.1895	
d_daaa_1	0.00109595	0.0191229	0.05731	0.9543	
d_wti_1	0.000333346	0.000229297	1.454	0.1461	
d_CNYUSDX_1	0.359308	1.30752	0.2748	0.7835	
d_eurusd_1	-0.0300362	0.0785372	-0.3824	0.7022	
d_gold_1	5.17667e-05	3.19975e-05	1.618	0.1058	
d_S&P500_1	-6.36447e-05	1.62603e-05	-3.914	<0.0001	***
EC1	0.00216085	0.00465150	0.4646	0.6423	
EC2	-0.0432093	0.0121575	-3.554	0.0004	***
EC3	0.0597624	0.0117874	5.070	<0.0001	***
EC4	-0.0259997	0.00611281	-4.253	<0.0001	***
Media de la vble. dep.	0.000980	D.T. de la vble. dep.	0.024487		
Suma de cuad. residuos	1.484283	D.T. de la regresión	0.023373		
R-cuadrado	0.094936	R-cuadrado corregido	0.088940		
rho	-0.009417	Durbin-Watson	2.014898		

Ecuación 3: d_dgs2

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-1.34917e-05	0.0246550	-0.0005472	0.9996	
d_dgs3mo_1	0.105257	0.0336684	3.126	0.0018	***
d_dgs1_1	0.0908186	0.0406360	2.235	0.0255	**
d_dgs2_1	-0.203702	0.0439174	-4.638	<0.0001	***
d_dgs5_1	0.135044	0.0541902	2.492	0.0128	**
d_dgs10_1	0.00512137	0.0532026	0.09626	0.9233	
d_dbaa_1	-0.0811111	0.0355183	-2.284	0.0225	**
d_daaa_1	-0.00915424	0.0268086	-0.3415	0.7328	
d_wti_1	0.000646889	0.000321454	2.012	0.0443	**
d_CNYUSDX_1	0.958367	1.83302	0.5228	0.6011	
d_eurusd_1	0.0843718	0.110102	0.7663	0.4436	
d_gold_1	9.59440e-05	4.48576e-05	2.139	0.0325	**
d_S&P500_1	-4.12735e-05	2.27955e-05	-1.811	0.0703	*
EC1	-0.0217062	0.00652098	-3.329	0.0009	***
EC2	0.0284758	0.0170438	1.671	0.0949	*
EC3	-0.00276275	0.0165249	-0.1672	0.8672	
EC4	-0.00104227	0.00856962	-0.1216	0.9032	
Media de la vble. dep.	0.000976	D.T. de la vble. dep.	0.033287		
Suma de cuad. residuos	2.917140	D.T. de la regresión	0.032767		
R-cuadrado	0.037405	R-cuadrado corregido	0.031028		
rho	-0.004768	Durbin-Watson	2.001340		

Ecuación 4: d_dgs5

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>
const	0.0242719	0.0323252	0.7509	0.4528

d_dgs3mo_1	0.0840023	0.0441428	1.903	0.0572	*
d_dgs1_1	0.0573915	0.0532780	1.077	0.2815	
d_dgs2_1	-0.00525031	0.0575802	-0.09118	0.9274	
d_dgs5_1	-0.0930897	0.0710489	-1.310	0.1902	
d_dgs10_1	0.150546	0.0697541	2.158	0.0310	**
d_dbaa_1	-0.0815580	0.0465682	-1.751	0.0800	*
d_daaa_1	-0.0366316	0.0351488	-1.042	0.2974	
d_wti_1	0.000804643	0.000421459	1.909	0.0563	*
d_CNYUSDX_1	1.51405	2.40328	0.6300	0.5288	
d_eurusd_1	0.291178	0.144355	2.017	0.0438	**
d_gold_1	0.000112555	5.88129e-05	1.914	0.0558	*
d_S&P500_1	-9.52582e-05	2.98872e-05	-3.187	0.0015	***
EC1	-0.0182747	0.00854968	-2.137	0.0326	**
EC2	0.0158728	0.0223461	0.7103	0.4776	
EC3	0.0168845	0.0216659	0.7793	0.4359	
EC4	-0.0139382	0.0112357	-1.241	0.2149	
Media de la vble. dep.	0.000775	D.T. de la vble. dep.		0.043424	
Suma de cuad. residuos	5.014536	D.T. de la regresión		0.042961	
R-cuadrado	0.027655	R-cuadrado corregido		0.021213	
rho	-0.001179	Durbin-Watson		1.998167	

Ecuación 5: d_dgs10

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	0.0545572	0.0339223	1.608	0.1079	
d_dgs3mo_1	0.0968959	0.0463237	2.092	0.0366	**
d_dgs1_1	0.0615723	0.0559103	1.101	0.2709	
d_dgs2_1	-0.0468562	0.0604251	-0.7754	0.4381	
d_dgs5_1	-0.0241379	0.0745592	-0.3237	0.7462	
d_dgs10_1	0.0903894	0.0732004	1.235	0.2170	
d_dbaa_1	-0.0380383	0.0488690	-0.7784	0.4364	
d_daaa_1	-0.0232817	0.0368854	-0.6312	0.5280	
d_wti_1	0.000958751	0.000442282	2.168	0.0303	**
d_CNYUSDX_1	2.38764	2.52202	0.9467	0.3439	
d_eurusd_1	0.368145	0.151487	2.430	0.0152	**
d_gold_1	0.000127643	6.17187e-05	2.068	0.0387	**
d_S&P500_1	-0.00013897	3.13639e-05	-4.431	<0.0001	***
EC1	-0.0227650	0.00897209	-2.537	0.0112	**
EC2	0.0272126	0.0234502	1.160	0.2460	
EC3	0.00802559	0.0227363	0.3530	0.7241	
EC4	-0.0149778	0.0117908	-1.270	0.2041	
Media de la vble. dep.	0.000358	D.T. de la vble. dep.		0.045582	
Suma de cuad. residuos	5.522285	D.T. de la regresión		0.045083	
R-cuadrado	0.028220	R-cuadrado corregido		0.021782	
rho	-0.000306	Durbin-Watson		1.998007	

Ecuación 6: d_dbaa

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-0.0192617	0.0332368	-0.5795	0.5623	
d_dgs3mo_1	-0.0239686	0.0453877	-0.5281	0.5975	
d_dgs1_1	0.0818931	0.0547806	1.495	0.1350	
d_dgs2_1	0.0263628	0.0592041	0.4453	0.6561	
d_dgs5_1	-0.170043	0.0730527	-2.328	0.0200	**
d_dgs10_1	0.0765118	0.0717213	1.067	0.2862	
d_dbaa_1	0.0476305	0.0478815	0.9948	0.3199	
d_daaa_1	0.0241665	0.0361401	0.6687	0.5038	
d_wti_1	0.000389721	0.000433345	0.8993	0.3686	
d_CNYUSDX_1	1.47405	2.47106	0.5965	0.5509	
d_eurusd_1	0.165944	0.148426	1.118	0.2637	
d_gold_1	8.45046e-05	6.04716e-05	1.397	0.1624	
d_S&P500_1	-0.00023462	3.07301e-05	-7.635	<0.0001	***
4					
EC1	-0.00694718	0.00879080	-0.7903	0.4294	
EC2	-0.0157352	0.0229763	-0.6848	0.4935	
EC3	0.0462598	0.0222769	2.077	0.0379	**
EC4	-0.0200676	0.0115525	-1.737	0.0825	*
Media de la vble. dep.					
	-3.65e-06	D.T. de la vble. dep.		0.045126	
Suma de cuad. residuos					
	5.301370	D.T. de la regresión		0.044172	
R-cuadrado					
	0.048143	R-cuadrado corregido		0.041837	
rho					
	-0.010985	Durbin-Watson		2.021472	

Ecuación 7: d_daaa

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	0.172669	0.0362883	4.758	<0.0001	***
d_dgs3mo_1	0.0153825	0.0495547	0.3104	0.7563	
d_dgs1_1	0.0353018	0.0598100	0.5902	0.5551	
d_dgs2_1	0.0693977	0.0646396	1.074	0.2831	
d_dgs5_1	-0.327996	0.0797596	-4.112	<0.0001	***
d_dgs10_1	0.254166	0.0783060	3.246	0.0012	***
d_dbaa_1	0.234559	0.0522775	4.487	<0.0001	***
d_daaa_1	-0.221650	0.0394581	-5.617	<0.0001	***
d_wti_1	0.000572704	0.000473130	1.210	0.2262	
d_CNYUSDX_1	3.66631	2.69793	1.359	0.1743	
d_eurusd_1	0.291802	0.162053	1.801	0.0719	*
d_gold_1	7.57112e-05	6.60234e-05	1.147	0.2516	
d_S&P500_1	-0.00021031	3.35515e-05	-6.268	<0.0001	***
5					
EC1	-0.0260113	0.00959788	-2.710	0.0068	***
EC2	0.00881034	0.0250858	0.3512	0.7255	
EC3	0.0356427	0.0243222	1.465	0.1429	
EC4	-0.0470425	0.0126132	-3.730	0.0002	***
Media de la vble. dep.					
	0.000110	D.T. de la vble. dep.		0.049238	
Suma de cuad. residuos					
	6.319488	D.T. de la regresión		0.048228	
R-cuadrado					
	0.046945	R-cuadrado corregido		0.040631	
rho					
	-0.001168	Durbin-Watson		2.002266	

Ecuación 8: d_wti

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-1.03606	1.47205	-0.7038	0.4816	
d_dgs3mo_1	1.92259	2.01021	0.9564	0.3389	
d_dgs1_1	-0.696550	2.42622	-0.2871	0.7741	
d_dgs2_1	1.05961	2.62213	0.4041	0.6862	
d_dgs5_1	-2.07942	3.23548	-0.6427	0.5205	
d_dgs10_1	2.44910	3.17652	0.7710	0.4408	
d_dbaa_1	0.252665	2.12066	0.1191	0.9052	
d_daaa_1	-2.79664	1.60064	-1.747	0.0807	*
d_wti_1	-0.236644	0.0191927	-12.33	<0.0001	***
d_CNYUSDX_1	88.1722	109.443	0.8056	0.4205	
d_eurusd_1	0.849310	6.57376	0.1292	0.8972	
d_gold_1	0.00388653	0.00267827	1.451	0.1469	
d_S&P500_1	0.000474698	0.00136103	0.3488	0.7273	
EC1	0.235922	0.389342	0.6060	0.5446	
EC2	-0.506889	1.01762	-0.4981	0.6184	
EC3	0.298717	0.986639	0.3028	0.7621	
EC4	0.0454577	0.511658	0.08884	0.9292	
Media de la vble. dep.	0.000928	D.T. de la vble. dep.		2.011426	
Suma de cuad. residuos	10399.06	D.T. de la regresión		1.956376	
R-cuadrado	0.060215	R-cuadrado corregido		0.053989	
rho	-0.019377	Durbin-Watson		2.036544	

Ecuación 9: d_CNYUSDX

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	0.000300311	0.000253973	1.182	0.2371	
d_dgs3mo_1	-0.00103183	0.000346821	-2.975	0.0030	***
d_dgs1_1	0.000183367	0.000418595	0.4381	0.6614	
d_dgs2_1	-0.00010663	0.000452396	-0.2357	0.8137	
	6				
d_dgs5_1	-0.00026712	0.000558217	-0.4785	0.6323	
	3				
d_dgs10_1	0.00139529	0.000548044	2.546	0.0110	**
d_dbaa_1	-0.00080683	0.000365877	-2.205	0.0275	**
	3				
d_daaa_1	-0.00026812	0.000276157	-0.9709	0.3317	
	4				
d_wti_1	3.32437e-06	3.31132e-06	1.004	0.3155	
d_CNYUSDX_1	-0.111466	0.0188821	-5.903	<0.0001	***
d_eurusd_1	0.00493673	0.00113417	4.353	<0.0001	***
d_gold_1	4.94282e-06	4.62081e-07	10.70	<0.0001	***
d_S&P500_1	1.01102e-06	2.34818e-07	4.306	<0.0001	***
EC1	0.000111853	6.71731e-05	1.665	0.0960	*
EC2	-0.00029967	0.000175569	-1.707	0.0880	*
	8				
EC3	0.000258902	0.000170225	1.521	0.1284	
EC4	-0.00015918	8.82763e-05	-1.803	0.0715	*
	5				

Media de la vble. dep.	-3.49e-06	D.T. de la vble. dep.	0.000351
Suma de cuad. residuos	0.000310	D.T. de la regresión	0.000338
R-cuadrado	0.083801	R-cuadrado corregido	0.077731
rho	-0.014412	Durbin-Watson	2.028435

Ecuación 10: d_eurusd

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	0.00622526	0.00416929	1.493	0.1355	
d_dgs3mo_1	-0.00595870	0.00569351	-1.047	0.2954	
d_dgs1_1	0.00204276	0.00687177	0.2973	0.7663	
d_dgs2_1	-0.0113041	0.00742667	-1.522	0.1281	
d_dgs5_1	-0.0403407	0.00916385	-4.402	<0.0001	***
d_dgs10_1	0.0354609	0.00899685	3.941	<0.0001	***
d_dbaa_1	-0.0131664	0.00600634	-2.192	0.0285	**
d_daaa_1	0.0119038	0.00453348	2.626	0.0087	***
d_wti_1	7.22085e-05	5.43596e-05	1.328	0.1842	
d_CNYUSDX_1	0.0614112	0.309974	0.1981	0.8430	
d_eurusd_1	-0.0658317	0.0186189	-3.536	0.0004	***
d_gold_1	9.39113e-05	7.58566e-06	12.38	<0.0001	***
d_S&P500_1	1.40621e-05	3.85484e-06	3.648	0.0003	***
EC1	-0.00203961	0.00110273	-1.850	0.0645	*
EC2	0.00573260	0.00288219	1.989	0.0468	**
EC3	-0.00540236	0.00279446	-1.933	0.0533	*
EC4	0.00137991	0.00144917	0.9522	0.3411	

Media de la vble. dep.	-0.000095	D.T. de la vble. dep.	0.005844
Suma de cuad. residuos	0.083420	D.T. de la regresión	0.005541
R-cuadrado	0.106862	R-cuadrado corregido	0.100945
rho	-0.016319	Durbin-Watson	2.028984

Ecuación 11: d_gold

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	34.7932	10.8591	3.204	0.0014	***
d_dgs3mo_1	-6.53009	14.8291	-0.4404	0.6597	
d_dgs1_1	13.2700	17.8979	0.7414	0.4585	
d_dgs2_1	2.04645	19.3431	0.1058	0.9158	
d_dgs5_1	-15.1503	23.8677	-0.6348	0.5256	
d_dgs10_1	-65.2362	23.4328	-2.784	0.0054	***
d_dbaa_1	30.2665	15.6438	1.935	0.0531	*
d_daaa_1	1.57852	11.8077	0.1337	0.8937	
d_wti_1	0.492751	0.141582	3.480	0.0005	***
d_CNYUSDX_1	-349.710	807.343	-0.4332	0.6649	
d_eurusd_1	16.4066	48.4938	0.3383	0.7351	
d_gold_1	-0.0862315	0.0197572	-4.365	<0.0001	***
d_S&P500_1	0.0255997	0.0100401	2.550	0.0108	**
EC1	-5.42134	2.87213	-1.888	0.0592	*
EC2	9.15974	7.50682	1.220	0.2225	
EC3	-3.44983	7.27831	-0.4740	0.6355	
EC4	-4.38490	3.77444	-1.162	0.2454	

Media de la vble. dep.	0.070249	D.T. de la vble. dep.	14.62887
Suma de cuad. residuos	565898.2	D.T. de la regresión	14.43193
R-cuadrado	0.033149	R-cuadrado corregido	0.026744
rho	-0.007164	Durbin-Watson	2.013862

Ecuación 12: d_S&P500

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-6.06152	22.1398	-0.2738	0.7843	
d_dgs3mo_1	0.847392	30.2337	0.02803	0.9776	
d_dgs1_1	-42.2924	36.4905	-1.159	0.2466	
d_dgs2_1	-0.867996	39.4371	-0.02201	0.9824	
d_dgs5_1	-65.4063	48.6619	-1.344	0.1790	
d_dgs10_1	154.943	47.7751	3.243	0.0012	***
d_dbaa_1	-99.4632	31.8949	-3.118	0.0018	***
d_daaa_1	-22.8863	24.0737	-0.9507	0.3419	
d_wti_1	0.574481	0.288660	1.990	0.0467	**
d_CNYUSDX_1	2992.30	1646.02	1.818	0.0692	*
d_eurusd_1	116.483	98.8699	1.178	0.2388	
d_gold_1	-0.0140821	0.0402814	-0.3496	0.7267	
d_S&P500_1	-0.145169	0.0204700	-7.092	<0.0001	***
EC1	0.604766	5.85574	0.1033	0.9178	
EC2	20.7493	15.3050	1.356	0.1753	
EC3	-30.3548	14.8391	-2.046	0.0409	**
EC4	15.7883	7.69538	2.052	0.0403	**

Media de la vble. dep.	0.916696	D.T. de la vble. dep.	29.85780
Suma de cuad. residuos	2352308	D.T. de la regresión	29.42404
R-cuadrado	0.035235	R-cuadrado corregido	0.028844
rho	0.003611	Durbin-Watson	1.992309

Matriz de covarianzas cruzadas entre ecuaciones:

	dgs3mo	dgs1	dgs2	dgs5	dgs10	dbaa	daaa	wti	CNYUSDX
			eurusd	gold	S&P500				
dgs3mo	0.00042805	0.00022713	0.00022239	0.00021707	0.00020017				
	9.3412e-05	8.6456e-05	0.0021282	-2.5310e-08	-3.9863e-07	-0.019251			
			0.043917						
dgs1	0.00022713	0.00054250	0.00055391	0.00058556	0.00050792				
	0.00031208	0.00031144	0.0036581	-2.1382e-07	1.2840e-07	-0.044502			
			0.090553						
dgs2	0.00022239	0.00055391	0.0010662	0.0011794	0.0010372				
	0.00068434	0.00069894	0.0062191	-1.0327e-07	-1.6560e-06	-0.096081			
			0.17741						
dgs5	0.00021707	0.00058556	0.0011794	0.0018328	0.0017751				
	0.0013259	0.0013211	0.010387	3.3003e-08	-4.2210e-07	-0.15717			
			0.26911						
dgs10	0.00020017	0.00050792	0.0010372	0.0017751	0.0020184				
	0.0016514	0.0016522	0.012721	1.1818e-07	2.1519e-06	-0.15616			
			0.34545						
dbaa	9.3412e-05	0.00031208	0.00068434	0.0013259	0.0016514				
	0.0019376	0.0018758	0.0078437	-4.6477e-08	-1.2799e-06	-0.13045			
			0.14351						

```

daaa      8.6456e-05  0.00031144  0.00069894  0.0013211  0.0016522
          0.0018758  0.0023098  0.0084229 -1.3897e-07 -5.5505e-06 -0.15013
                                0.096366
wti        0.0021282  0.0036581  0.0062191  0.010387  0.012721
          0.0078437  0.0084229  3.80081.8375e-06 -0.00014931 3.1273 8.9238
CNYUSDx    -2.5310e-08 -2.1382e-07 -1.0327e-07 3.3003e-08 1.1818e-07 -
          4.6477e-08 -1.3897e-07 1.8375e-06 1.1314e-07 3.1594e-07 0.00012280 -
                                0.00016502
eurusd     -3.9863e-07 1.2840e-07 -1.6560e-06 -4.2210e-07 2.1519e-06 -
          1.2799e-06 -5.5505e-06 -0.00014931 3.1594e-07 3.0490e-05 0.0039532
                                0.0018868
gold       -0.019251  -0.044502  -0.096081  -0.15717  -0.15616  -0.13045
          -0.15013  3.1273 0.00012280  0.0039532  206.83 2.2161
S&P500      0.043917  0.090553  0.17741  0.26911  0.34545
          0.14351  0.096366  8.9238 -0.00016502 0.0018868 2.2161 859.76

```

Determinante = 5.24879e-30

8.3. VECM tipos de interés tipos de cambios

Sistema VECM, orden del retardo 2

Estimaciones de Máxima Verosimilitud, observaciones 2012-01-05-2022-06-30 (T = 2735)

Rango de cointegración = 4

Caso 3: Constante no restringida

beta (Vectores cointegrantes, Desviaciones típicas entre paréntesis)

```

DGS3MO      1.0000      0.0000      0.0000      0.0000
            (0.0000)    (0.0000)    (0.0000)    (0.0000)
DGS1  0.0000      1.0000      0.0000      0.0000
            (0.0000)    (0.0000)    (0.0000)    (0.0000)
DGS2  0.0000      0.0000      1.0000      0.0000
            (0.0000)    (0.0000)    (0.0000)    (0.0000)
DGS5  0.0000      0.0000      0.0000      1.0000
            (0.0000)    (0.0000)    (0.0000)    (0.0000)
DGS7  17.205      20.004      15.178      1.2849
            (2.1961)    (2.8083)    (2.2871)    (0.40702)
DGS10 -26.236     -30.041     -23.275     -3.5703
            (2.8190)    (3.6049)    (2.9358)    (0.52247)
DBAA  -3.3179     -3.7039     -2.9709     -0.55335
            (0.68006)    (0.86965)    (0.70824)    (0.12604)
DAAA  13.704      15.520      12.314      2.2438
            (1.2841)    (1.6421)    (1.3373)    (0.23800)
cadusd 34.819      41.929      33.328      5.7395
            (4.9827)    (6.3718)    (5.1891)    (0.92349)
cnyusd 188.82      220.90      176.36      31.553
            (31.374)    (40.120)    (32.673)    (5.8148)
eurusd 4.2153      4.8114      4.0065      0.77622
            (3.3490)    (4.2827)    (3.4878)    (0.62070)
mxnusd -279.39     -337.45     -273.16     -50.807
            (40.268)    (51.494)    (41.936)    (7.4632)

```

alpha (vectores de ajuste)

DGS3MO	-0.020617	0.027731	-0.020011	0.038063
DGS1	0.0071706	-0.049456	0.055904	-0.0047898
DGS2	-0.0097481	0.017654	-0.025480	0.084275
DGS5	0.0048387	-0.015617	0.0092943	0.036825
DGS7	0.0021613	-0.0049938	-0.014286	0.10898
DGS10	0.0021605	-0.0055074	-0.0023202	0.045379
DBAA	0.017410	-0.053496	0.048791	0.010571
DAAA	-0.0014491	-0.026226	0.029246	0.021173
cadusd	-0.00040472	0.0015945	-0.0026640	0.0056418
cnyusd	5.0976e-05	-0.00011916	3.5106e-05	0.00027773
eurusd	-0.0024032	0.0071615	-0.0076284	0.0059410
mxnusd	-7.9911e-05	0.00018077	-0.00039092	0.0013649

Log-verosimilitud = 114373.96

Determinante de la matriz de covarianzas = 7.7109255e-52

AIC = -83.4179

BIC = -82.7692

HQC = -83.1835

Ecuación 1: d_DGS3MO

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	0.0297376	0.0157277	1.891	0.0588	*
d_DGS3MO_1	0.0366763	0.0213908	1.715	0.0865	*
d_DGS1_1	0.107006	0.0258174	4.145	<0.0001	***
d_DGS2_1	0.0721086	0.0280547	2.570	0.0102	**
d_DGS5_1	-0.0329519	0.0480250	-0.6861	0.4927	
d_DGS7_1	0.0438055	0.0595578	0.7355	0.4621	
d_DGS10_1	-0.0683595	0.0446241	-1.532	0.1257	
d_DBAA_1	0.00854871	0.0227129	0.3764	0.7067	
d_DAAA_1	-0.0175456	0.0169363	-1.036	0.3003	
d_cadusd_1	-0.0340274	0.120381	-0.2827	0.7775	
d_cnyusd_1	-0.291258	1.17258	-0.2484	0.8039	
d_eurusd_1	-0.0158354	0.0755209	-0.2097	0.8339	
d_mxnusd_1	1.82829	0.954611	1.915	0.0556	*
EC1	-0.0206166	0.00370732	-5.561	<0.0001	***
EC2	0.0277313	0.0108833	2.548	0.0109	**
EC3	-0.0200110	0.0122154	-1.638	0.1015	
EC4	0.0380631	0.0223477	1.703	0.0886	*

Media de la vble. dep.	0.000622	D.T. de la vble. dep.	0.021635
Suma de cuad. residuos	1.176562	D.T. de la regresión	0.020813
R-cuadrado	0.080627	R-cuadrado corregido	0.074534
rho	0.002742	Durbin-Watson	1.989232

Ecuación 2: d_DGS1

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-0.0343580	0.0176853	-1.943	0.0522	*
d_DGS3MO_1	0.251916	0.0240533	10.47	<0.0001	***
d_DGS1_1	-0.0542068	0.0290309	-1.867	0.0620	*
d_DGS2_1	0.0202294	0.0315466	0.6413	0.5214	

d_DGS5_1	0.127888	0.0540026	2.368	0.0179	**
d_DGS7_1	-0.108768	0.0669710	-1.624	0.1045	
d_DGS10_1	-0.0123971	0.0501784	-0.2471	0.8049	
d_DBAA_1	-0.0174479	0.0255400	-0.6832	0.4946	
d_DAAA_1	0.00693997	0.0190444	0.3644	0.7156	
d_cadusd_1	0.0145771	0.135364	0.1077	0.9143	
d_cnyusd_1	0.159507	1.31853	0.1210	0.9037	
d_eurusd_1	-0.0334585	0.0849210	-0.3940	0.6936	
d_mxnusd_1	2.47353	1.07343	2.304	0.0213	**
EC1	0.00717062	0.00416877	1.720	0.0855	*
EC2	-0.0494560	0.0122380	-4.041	<0.0001	***
EC3	0.0559037	0.0137359	4.070	<0.0001	***
EC4	-0.00478984	0.0251293	-0.1906	0.8488	
Media de la vble. dep.	0.000980	D.T. de la vble. dep.		0.024484	
Suma de cuad. residuos	1.487684	D.T. de la regresión		0.023404	
R-cuadrado	0.092308	R-cuadrado corregido		0.086292	
rho	-0.008821	Durbin-Watson		2.013887	

Ecuación 3: d_DGS2

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-0.0547144	0.0248309	-2.203	0.0276	**
d_DGS3MO_1	0.0913323	0.0337717	2.704	0.0069	***
d_DGS1_1	0.103280	0.0407605	2.534	0.0113	**
d_DGS2_1	-0.211429	0.0442927	-4.773	<0.0001	***
d_DGS5_1	0.189040	0.0758217	2.493	0.0127	**
d_DGS7_1	-0.0903193	0.0940298	-0.9605	0.3369	
d_DGS10_1	0.0236205	0.0704524	0.3353	0.7374	
d_DBAA_1	-0.0683476	0.0358591	-1.906	0.0568	*
d_DAAA_1	-0.00839722	0.0267391	-0.3140	0.7535	
d_cadusd_1	0.129203	0.190057	0.6798	0.4967	
d_cnyusd_1	0.390411	1.85127	0.2109	0.8330	
d_eurusd_1	0.0584560	0.119232	0.4903	0.6240	
d_mxnusd_1	2.98107	1.50714	1.978	0.0480	**
EC1	-0.00974808	0.00585311	-1.665	0.0959	*
EC2	0.0176539	0.0171826	1.027	0.3043	
EC3	-0.0254797	0.0192857	-1.321	0.1866	
EC4	0.0842751	0.0352825	2.389	0.0170	**
Media de la vble. dep.	0.000976	D.T. de la vble. dep.		0.033330	
Suma de cuad. residuos	2.932705	D.T. de la regresión		0.032860	
R-cuadrado	0.034371	R-cuadrado corregido		0.027972	
rho	-0.003194	Durbin-Watson		1.998410	

Ecuación 4: d_DGS5

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-0.0698694	0.0327196	-2.135	0.0328	**
d_DGS3MO_1	0.0670253	0.0445009	1.506	0.1321	
d_DGS1_1	0.0812015	0.0537099	1.512	0.1307	
d_DGS2_1	-0.0189804	0.0583643	-0.3252	0.7451	

d_DGS5_1	-0.0531406	0.0999101	-0.5319	0.5948	
d_DGS7_1	-0.0436515	0.123903	-0.3523	0.7246	
d_DGS10_1	0.119034	0.0928350	1.282	0.1999	
d_DBAA_1	-0.0699861	0.0472514	-1.481	0.1387	
d_DAAA_1	-0.0289370	0.0352340	-0.8213	0.4116	
d_cadusd_1	0.182528	0.250437	0.7288	0.4662	
d_cnyusd_1	1.23079	2.43941	0.5045	0.6139	
d_eurusd_1	0.258527	0.157112	1.645	0.1000	*
d_mxnusd_1	1.19559	1.98595	0.6020	0.5472	
EC1	0.00483868	0.00771263	0.6274	0.5305	
EC2	-0.0156169	0.0226415	-0.6897	0.4904	
EC3	0.00929428	0.0254127	0.3657	0.7146	
EC4	0.0368251	0.0464916	0.7921	0.4284	
Media de la vble. dep.	0.000775	D.T. de la vble. dep.		0.043459	
Suma de cuad. residuos	5.092130	D.T. de la regresión		0.043300	
R-cuadrado	0.013833	R-cuadrado corregido		0.007297	
rho	-0.000275	Durbin-Watson		1.996447	

Ecuación 5: d_DGS7

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-0.0443708	0.0349264	-1.270	0.2040	
d_DGS3MO_1	0.0591788	0.0475023	1.246	0.2129	
d_DGS1_1	0.0870942	0.0573324	1.519	0.1289	
d_DGS2_1	-0.0632254	0.0623007	-1.015	0.3103	
d_DGS5_1	0.125730	0.106649	1.179	0.2385	
d_DGS7_1	-0.284417	0.132259	-2.150	0.0316	**
d_DGS10_1	0.199453	0.0990963	2.013	0.0442	**
d_DBAA_1	-0.0414253	0.0504383	-0.8213	0.4115	
d_DAAA_1	-0.0187817	0.0376104	-0.4994	0.6176	
d_cadusd_1	0.135117	0.267328	0.5054	0.6133	
d_cnyusd_1	1.97769	2.60394	0.7595	0.4476	
d_eurusd_1	0.325404	0.167709	1.940	0.0524	*
d_mxnusd_1	1.22699	2.11990	0.5788	0.5628	
EC1	0.00216132	0.00823281	0.2625	0.7929	
EC2	-0.00499380	0.0241685	-0.2066	0.8363	
EC3	-0.0142858	0.0271267	-0.5266	0.5985	
EC4	0.108978	0.0496273	2.196	0.0282	**
Media de la vble. dep.	0.000589	D.T. de la vble. dep.		0.046400	
Suma de cuad. residuos	5.802177	D.T. de la regresión		0.046220	
R-cuadrado	0.014267	R-cuadrado corregido		0.007734	
rho	-0.000576	Durbin-Watson		1.998282	

Ecuación 6: d_DGS10

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-0.0547256	0.0344170	-1.590	0.1119	
d_DGS3MO_1	0.0811450	0.0468096	1.734	0.0831	*
d_DGS1_1	0.0876244	0.0564964	1.551	0.1210	
d_DGS2_1	-0.0616006	0.0613922	-1.003	0.3158	

d_DGS5_1	-0.0138523	0.105093	-0.1318	0.8951	
d_DGS7_1	0.0286580	0.130331	0.2199	0.8260	
d_DGS10_1	-9.01328e-05	0.0976512	-0.0009230	0.9993	
d_DBAA_1	-0.0241504	0.0497028	-0.4859	0.6271	
d_DAAA_1	-0.0116104	0.0370619	-0.3133	0.7541	
d_cadusd_1	0.141749	0.263430	0.5381	0.5906	
d_cnyusd_1	2.31561	2.56597	0.9024	0.3669	
d_eurusd_1	0.343197	0.165263	2.077	0.0379	**
d_mxnusd_1	1.36308	2.08898	0.6525	0.5141	
EC1	0.00216053	0.00811275	0.2663	0.7900	
EC2	-0.00550744	0.0238161	-0.2312	0.8171	
EC3	-0.00232024	0.0267311	-0.08680	0.9308	
EC4	0.0453791	0.0489036	0.9279	0.3535	
Media de la vble. dep.	0.000358	D.T. de la vble. dep.		0.045616	
Suma de cuad. residuos	5.634189	D.T. de la regresión		0.045546	
R-cuadrado	0.009643	R-cuadrado corregido		0.003080	
rho	-0.000515	Durbin-Watson		1.998380	

Ecuación 7: d_DBAA

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	-0.112669	0.0338004	-3.333	0.0009	***
d_DGS3MO_1	-0.0348557	0.0459708	-0.7582	0.4484	
d_DGS1_1	0.112795	0.0554841	2.033	0.0422	**
d_DGS2_1	0.00856544	0.0602922	0.1421	0.8870	
d_DGS5_1	-0.195984	0.103210	-1.899	0.0577	*
d_DGS7_1	0.105203	0.127996	0.8219	0.4112	
d_DGS10_1	-0.0819602	0.0959015	-0.8546	0.3928	
d_DBAA_1	0.0529272	0.0488122	1.084	0.2783	
d_DAAA_1	0.0547568	0.0363978	1.504	0.1326	
d_cadusd_1	-0.385335	0.258710	-1.489	0.1365	
d_cnyusd_1	2.82309	2.51999	1.120	0.2627	
d_eurusd_1	0.286233	0.162302	1.764	0.0779	*
d_mxnusd_1	-4.39341	2.05155	-2.142	0.0323	**
EC1	0.0174101	0.00796739	2.185	0.0290	**
EC2	-0.0534960	0.0233894	-2.287	0.0223	**
EC3	0.0487915	0.0262522	1.859	0.0632	*
EC4	0.0105709	0.0480273	0.2201	0.8258	
Media de la vble. dep.	-3.66e-06	D.T. de la vble. dep.		0.045139	
Suma de cuad. residuos	5.434094	D.T. de la regresión		0.044730	
R-cuadrado	0.024522	R-cuadrado corregido		0.018057	
rho	-0.005644	Durbin-Watson		2.010777	

Ecuación 8: d_DAAA

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	0.0817573	0.0368568	2.218	0.0266	**
d_DGS3MO_1	0.00634523	0.0501278	0.1266	0.8993	
d_DGS1_1	0.0611587	0.0605013	1.011	0.3122	

d_DGS2_1	0.0633431	0.0657442	0.9635	0.3354	
d_DGS5_1	-0.435604	0.112543	-3.871	0.0001	***
d_DGS7_1	0.240283	0.139570	1.722	0.0853	*
d_DGS10_1	0.0444959	0.104574	0.4255	0.6705	
d_DBAA_1	0.234332	0.0532262	4.403	<0.0001	***
d_DAAA_1	-0.197405	0.0396892	-4.974	<0.0001	***
d_cadusd_1	-0.0687307	0.282104	-0.2436	0.8075	
d_cnyusd_1	4.86927	2.74787	1.772	0.0765	*
d_eurusd_1	0.328125	0.176978	1.854	0.0638	*
d_mxnusd_1	-4.70697	2.23707	-2.104	0.0355	**
EC1	-0.00144910	0.00868786	-0.1668	0.8675	
EC2	-0.0262265	0.0255044	-1.028	0.3039	
EC3	0.0292457	0.0286261	1.022	0.3070	
EC4	0.0211726	0.0523703	0.4043	0.6860	
Media de la vble. dep.	0.000110	D.T. de la vble. dep.	0.049253		
Suma de cuad. residuos	6.461306	D.T. de la regresión	0.048775		
R-cuadrado	0.025792	R-cuadrado corregido	0.019335		
rho	0.002214	Durbin-Watson	1.995424		

Ecuación 9: d_cadusd

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	0.00575253	0.00256776	2.240	0.0252	**
d_DGS3MO_1	-0.00045905	0.00349233	-0.1314	0.8954	
1					
d_DGS1_1	0.00109955	0.00421503	0.2609	0.7942	
d_DGS2_1	-0.00214565	0.00458030	-0.4685	0.6395	
d_DGS5_1	-0.0347878	0.00784072	-4.437	<0.0001	***
d_DGS7_1	0.00799642	0.00972361	0.8224	0.4109	
d_DGS10_1	0.0317950	0.00728548	4.364	<0.0001	***
d_DBAA_1	0.00448233	0.00370818	1.209	0.2269	
d_DAAA_1	-0.00846340	0.00276508	-3.061	0.0022	***
d_cadusd_1	-0.0556741	0.0196537	-2.833	0.0046	***
d_cnyusd_1	-0.308563	0.191440	-1.612	0.1071	
d_eurusd_1	-0.0186474	0.0123298	-1.512	0.1306	
d_mxnusd_1	2.95749	0.155853	18.98	<0.0001	***
EC1	-0.00040472	0.000605269	-0.6687	0.5038	
3					
EC2	0.00159454	0.00177685	0.8974	0.3696	
EC3	-0.00266404	0.00199433	-1.336	0.1817	
EC4	0.00564178	0.00364855	1.546	0.1221	
Media de la vble. dep.	-0.000078	D.T. de la vble. dep.	0.003693		
Suma de cuad. residuos	0.031361	D.T. de la regresión	0.003398		
R-cuadrado	0.159012	R-cuadrado corregido	0.153439		
rho	-0.004504	Durbin-Watson	2.008860		

Ecuación 10: d_cnyusd

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	0.000616989	0.000260865	2.365	0.0181	**

d_DGS3MO_1	-0.00101768	0.000354795	-2.868	0.0042	***
d_DGS1_1	6.46357e-05	0.000428216	0.1509	0.8800	
d_DGS2_1	0.000170450	0.000465325	0.3663	0.7142	
d_DGS5_1	-0.00206855	0.000796559	-2.597	0.0095	***
d_DGS7_1	0.00224369	0.000987847	2.271	0.0232	**
d_DGS10_1	0.000661028	0.000740151	0.8931	0.3719	
d_DBAA_1	-0.00066374	0.000376724	-1.762	0.0782	*
	5				
d_DAAA_1	-0.00062386	0.000280912	-2.221	0.0264	**
	7				
d_cadusd_1	0.00498300	0.00199668	2.496	0.0126	**
d_cnyusd_1	-0.120044	0.0194489	-6.172	<0.0001	***
d_eurusd_1	0.00407526	0.00125262	3.253	0.0012	***
d_mxnusd_1	0.0404428	0.0158335	2.554	0.0107	**
EC1	5.09755e-05	6.14909e-05	0.8290	0.4072	
EC2	-0.00011916	0.000180515	-0.6601	0.5092	
	5				
EC3	3.51059e-05	0.000202609	0.1733	0.8625	
EC4	0.000277726	0.000370666	0.7493	0.4538	
Media de la vble. dep.	-3.49e-06	D.T. de la vble. dep.		0.000352	
Suma de cuad. residuos	0.000324	D.T. de la regresión		0.000345	
R-cuadrado	0.041959	R-cuadrado corregido		0.035609	
rho	-0.008597	Durbin-Watson		2.017002	

Ecuación 11: d_eurusd

	<i>Coficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>	
const	0.00888018	0.00422397	2.102	0.0356	**
d_DGS3MO_1	-0.00717397	0.00574490	-1.249	0.2119	
d_DGS1_1	0.00196913	0.00693375	0.2840	0.7764	
d_DGS2_1	-0.0105188	0.00753461	-1.396	0.1628	
d_DGS5_1	-0.0466444	0.0128980	-3.616	0.0003	***
d_DGS7_1	0.0130609	0.0159954	0.8165	0.4143	
d_DGS10_1	0.0221617	0.0119846	1.849	0.0645	*
d_DBAA_1	-0.00407050	0.00609998	-0.6673	0.5046	
d_DAAA_1	0.00436256	0.00454857	0.9591	0.3376	
d_cadusd_1	-0.00156902	0.0323305	-0.04853	0.9613	
d_cnyusd_1	-0.239898	0.314919	-0.7618	0.4463	
d_eurusd_1	-0.0659296	0.0202826	-3.251	0.0012	***
d_mxnusd_1	2.87906	0.256379	11.23	<0.0001	***
EC1	-0.00240318	0.000995671	-2.414	0.0159	**
EC2	0.00716153	0.00292293	2.450	0.0143	**
EC3	-0.00762843	0.00328068	-2.325	0.0201	**
EC4	0.00594103	0.00600189	0.9899	0.3223	
Media de la vble. dep.	-0.000095	D.T. de la vble. dep.		0.005845	
Suma de cuad. residuos	0.084865	D.T. de la regresión		0.005590	
R-cuadrado	0.091400	R-cuadrado corregido		0.085378	
rho	0.006626	Durbin-Watson		1.983969	

Ecuación 12: d_mxnusd

	<i>Coefficiente</i>	<i>Desv. Típica</i>	<i>Estadístico t</i>	<i>valor p</i>								
const	0.000554390	0.000325421	1.704	0.0886	*							
d_DGS3MO_1	0.000140056	0.000442595	0.3164	0.7517								
d_DGS1_1	-0.00069188	0.000534186	-1.295	0.1954								
8												
d_DGS2_1	0.000426353	0.000580478	0.7345	0.4627								
d_DGS5_1	-0.00240662	0.000993682	-2.422	0.0155	**							
d_DGS7_1	0.00132732	0.00123231	1.077	0.2815								
d_DGS10_1	0.00240107	0.000923315	2.600	0.0094	***							
d_DBAA_1	-0.00081089	0.000469951	-1.725	0.0846	*							
1												
d_DAAA_1	-0.00077886	0.000350429	-2.223	0.0263	**							
1												
d_cadusd_1	8.60346e-05	0.00249079	0.03454	0.9724								
d_cnyusd_1	-0.0398155	0.0242618	-1.641	0.1009								
d_eurusd_1	0.00172733	0.00156260	1.105	0.2691								
d_mxnusd_1	-0.0263750	0.0197518	-1.335	0.1819								
EC1	-7.99114e-	7.67080e-05	-1.042	0.2976								
05												
EC2	0.000180766	0.000225187	0.8027	0.4222								
EC3	-0.00039092	0.000252749	-1.547	0.1221								
3												
EC4	0.00136486	0.000462394	2.952	0.0032	***							
Media de la vble. dep.												
	-8.54e-06	D.T. de la vble. dep.		0.000433								
Suma de cuad. residuos												
	0.000504	D.T. de la regresión		0.000431								
R-cuadrado												
	0.016254	R-cuadrado corregido		0.009734								
rho												
	-0.001869	Durbin-Watson		2.003557								
Matriz de covarianzas cruzadas entre ecuaciones:												
	DGS3MO	DGS1	DGS2	DGS5	DGS7	DGS10	DBAA	DAAA	cadusd	cnyusd	eurusd	mxnusd
DGS3MO	0.00043019	0.00022956	0.00022499	0.00022386	0.00021545							
	0.00020915	0.00010656	9.9117e-05	4.1856e-08	-6.4847e-08	-1.4930e-06						
			1.4727e-07									
DGS1	0.00022956	0.00054394	0.00055569	0.00059310	0.00057280							
	0.00051839	0.00032822	0.00032837	1.2225e-06	-2.0655e-07	-6.0631e-07						
			1.7760e-07									
DGS2	0.00022499	0.00055569	0.0010723	0.0011952	0.0011670							
	0.0010559	0.00070491	0.00072235	2.1998e-06	-4.5075e-08	-1.3150e-06						
			2.3968e-07									
DGS5	0.00022386	0.00059310	0.0011952	0.0018618	0.0019262							
	0.0018096	0.0013621	0.0013599	8.4595e-07	6.5037e-08	5.3138e-07						
			5.9839e-07									
DGS7	0.00021545	0.00057280	0.0011670	0.0019262	0.0021215							
	0.0020340	0.0016195	0.0016251	1.0738e-06	4.1388e-08	1.7765e-06						
			2.6855e-07									
DGS10	0.00020915	0.00051839	0.0010559	0.0018096	0.0020340							
	0.0020600	0.0016964	0.0017007	1.8492e-06	1.2425e-07	2.7979e-06						
			2.8811e-07									

DBAA	0.00010656	0.00032822	0.00070491	0.0013621	0.0016195	
0.0016964	0.0019869	0.0019260	-7.0044e-06	-1.0964e-07	1.8698e-07	-
		9.3990e-07				
DAAA	9.9117e-05	0.00032837	0.00072235	0.0013599	0.0016251	
0.0017007	0.0019260	0.0023625	-8.3410e-06	-2.3284e-07	-4.4627e-06	-
		4.0486e-07				
cadusd	4.1856e-08	1.2225e-06	2.1998e-06	8.4595e-07	1.0738e-06	1.8492e-
06	-7.0044e-06	-8.3410e-06	1.1467e-05	1.9409e-07	6.5253e-06	2.3693e-07
cnyusd	-6.4847e-08	-2.0655e-07	-4.5075e-08	6.5037e-08	4.1388e-08	1.2425e-
07	-1.0964e-07	-2.3284e-07	1.9409e-07	1.1835e-07	3.9987e-07	1.3006e-08
eurusd	-1.4930e-06	-6.0631e-07	-1.3150e-06	5.3138e-07	1.7765e-06	2.7979e-
06	1.8698e-07	-4.4627e-06	6.5253e-06	3.9987e-07	3.1029e-05	1.8616e-07
mxnusd	1.4727e-07	-1.7760e-07	-2.3968e-07	-5.9839e-07	-2.6855e-07	
	2.8811e-07	-9.3990e-07	-4.0486e-07	2.3693e-07	1.3006e-08	1.8616e-07
			1.8417e-07			

Determinante = 7.71093e-52

8.4. Objetivos y estrategias de fondos de inversión referenciados.

Alger focus Equity fund: *Invests in a focused portfolio of approximately 50 holdings consisting of companies of any capitalization identified through our fundamental research as demonstrating promising growth potential. Seeks long term capital appreciation.*

Polen Capital Focus US Growth Fund: *Our Focus U.S. Growth strategy seeks to achieve long-term growth by building a concentrated portfolio of outstanding businesses with competitive advantages and the potential for sustained growth*

BlackRock Advantage US Equity Fund: *The Fund aims to achieve long-term capital growth on your investment. The Fund seeks to gain at least 70% of its investment exposure to equity securities (e.g. shares) of companies domiciled in, listed in, or the main business of which is in, the United States of America. This is achieved by investing in equity securities, other equity-related securities and, when determined appropriate fixed income (FI) securities (such as bonds), money market instruments (MMIs) (i.e. debt securities with short-term maturities), deposits and cash.*

BlackRock BGS Flexible Equity Fund: *The US Flexible Equity Fund seeks to maximise total return. The Fund invests at least 70% of its total assets in the equity securities of companies domiciled in, or exercising the predominant part of their economic activity in, the US. The Fund normally invests in securities that, in the opinion of the Investment Adviser, exhibit either growth or value investment characteristics, placing an emphasis as the market outlook warrants*

BlackRock BGS Basic Value Fund: *The US Basic Value Fund seeks to maximise total return. The Fund invests at least 70% of its total assets in the equity securities of companies domiciled in, or exercising the predominant part of their economic activity in, the US. The Fund places particular emphasis on companies that are, in the opinion of the Investment Adviser, undervalued and therefore represent basic investment value*

Fidelity American Growth Fund: *Los gestores tratan de invertir en negocios de crecimiento secular a largo plazo. El enfoque flexible en todos los segmentos de capitalización del Subfondo se centra en contar con posiciones en empresas de crecimiento cuya valoración cuenta con un respaldo y que presentan características sostenibles, para incrementar la probabilidad de superar al mercado a largo plazo. Al diseccionar la cadena de valor de un tema a largo plazo, el Subfondo puede invertir en ideas en todo el espectro de capitalización bursátil. Las consideraciones sobre aspectos medioambientales, sociales y de gobernanza están integradas en el proceso de inversión*

Fidelity Fund A-ACC-Euro: Los gestores tratan de invertir en negocios de crecimiento secular a largo plazo. El enfoque flexible en todos los segmentos de capitalización del Subfondo se centra en contar con posiciones en empresas de crecimiento cuya valoración cuenta con un respaldo y que presentan características sostenibles, para incrementar la probabilidad de superar al mercado a largo plazo. Al diseccionar la cadena de valor de un tema a largo plazo, el Subfondo puede invertir en ideas en todo el espectro de capitalización bursátil. Las consideraciones sobre aspectos medioambientales, sociales y de gobernanza están integradas en el proceso de inversión.

JP Morgan large cap growth fund: Designed to provide long-term capital appreciation primarily through a diversified portfolio of high-growth U.S. equity securities. Approach. Typically invests in a diversified portfolio of large cap companies with above-average growth prospects. Invests primarily in large, well established companies. Looks for companies with attractive fundamentals, potential to exceed market expectations and positive price momentum

JP Morgan US equity fund: Designed to provide high total return primarily through a portfolio of U.S. large cap stocks. Approach. Employs fundamental analysis to identify companies with growth potential and construct a high-conviction portfolio of best ideas. Research-driven approach that focuses on identifying the most attractively valued stocks in each sector. Maintains sector weightings similar to the S&P 500 Index, seeking to outperform with similar volatility

Goldman Sachs US Enhanced Equity: El fondo invierte principalmente en empresas que están establecidas, cotizan o se negocian en los Estados Unidos de América. Nuestro objetivo es diseñar una cartera que maximice la rentabilidad y controle el riesgo. El fondo emplea la gestión activa mediante una combinación de estrategias basadas en modelos y el análisis fundamental de inversiones para determinar si una inversión es atractiva, manteniendo los límites de desviación con respecto al valor de referencia por valor y sector. Por consiguiente, su composición a nivel de títulos concretos puede ser sustancialmente diferente de la del valor de referencia. Pretendemos superar la rentabilidad del valor de referencia, el S&P 500 (Net), calculada en un periodo de varios años

Goldman Sachs US Core Equity Portfolio: El Fondo trata de aumentar el capital a largo plazo y pretende lograr su objetivo invirtiendo en acciones o en instrumentos similares relacionados con empresas estadounidenses. Estas empresas tienen su sede en EE. UU. u obtienen la mayor parte de sus ingresos o beneficios en dicha región. El Fondo también puede invertir en empresas con sede en cualquier parte del mundo.

Vanguard Growth and Income Fund Admiral Shares: This fund seeks to outperform the Standard & Poor's 500 Index. The fund's quantitative and fundamental investment approaches result in risk and sector profiles that are similar to those of its benchmark index. The fund has a total return goal, meaning that it seeks both capital appreciation and dividend income. The key risk for the fund is the volatility that comes with its full exposure to the stock market. The fund provides broad exposure to a diverse group of large U.S. companies and may be considered a primary equity holding in a portfolio.

Vanguard Growth and Income Fund Investor Shares: This fund seeks to outperform the Standard & Poor's 500 Index. The fund's quantitative and fundamental investment approaches result in risk and sector profiles that are similar to those of its benchmark index. The fund has a total return goal, meaning that it seeks both capital appreciation and dividend income. The key risk for the fund is the volatility that comes with its full exposure to the stock market. The fund provides broad exposure to a diverse group of large U.S. companies and may be considered a primary equity holding in a portfolio.

8.5. Código íntegro de python

Siguiendo las tendencias actuales en el universo de la programación, el código se ha generado poco a poco y por segmentos mediante consultas a ChatGPT incluyendo correcciones y modificaciones cuando se ha hecho necesario.

```
import yfinance as yf

from curl_cffi import requests

import pandas as pd

import numpy as np

from ta.momentum import RSIIndicator, StochasticOscillator

from ta.trend import MACD, ADXIndicator

from sklearn.preprocessing import StandardScaler

import gym

from gym import spaces

import matplotlib.pyplot as plt

from stable_baselines3.common.vec_env import DummyVecEnv

import torch


HIST_GLOBAL = []


session = requests.Session(impersonate="chrome")

# -----
# 1. PARÁMETROS INICIALES
# -----

tickers = [
    'APD', 'UNP', 'AFL', 'CSCO', 'FAST', 'AMGN', 'PLD', 'PCAR', 'RTX', 'BK',
    'FTNT', 'NXPI', 'MPC', 'AMAT', 'KLAC', 'LRCX', 'VLO', 'DFS', 'URI', 'AVGO',
    'MRK', 'T', 'PSA', 'BDX', 'SO', 'PEG', 'XEL', 'PM', 'MDLZ', 'MO',
    'RCL', 'AXON', 'MU', 'NFLX', 'AMD', 'NVDA', 'FCX', 'TSLA',
    'AAPL', 'MSCI', 'INTU', 'SNPS', 'BAC', 'ODFL', 'PH', 'MCO', 'BKNG', 'ADBE',
    'EOG', 'BKR', 'COP', 'BA', 'WMB', 'F', 'INTC', 'HES', 'OKE', 'SPG',
    "SHY", "TLT", "IEF"
]

start_date = "2012-01-03"
```

```
end_date = "2025-05-16"
```

```
# -----
```

```
# 2. FUNCIÓN PARA CONSTRUIR FEATURES
```

```
# -----
```

```
def build_features(df):
```

```
    df = df.copy()
```

```
    # 1) Aplanar MultiIndex si existe
```

```
    if isinstance(df.columns, pd.MultiIndex):
```

```
        df.columns = [" ".join(map(str, col)).strip().lower() for col in df.columns]
```

```
    else:
```

```
        df.columns = [c.lower().strip() for c in df.columns]
```

```
    # 2) Buscar columna de precios close (prefiere adj close si existe)
```

```
    close_cols = [c for c in df.columns if "close" in c]
```

```
    if not close_cols:
```

```
        raise KeyError("No se encontró columna con 'close'")
```

```
    # si hay alguna que contenga 'adj', esa tiene prioridad
```

```
    adj = [c for c in close_cols if "adj" in c]
```

```
    if adj:
```

```
        close_col = adj[0]
```

```
    else:
```

```
        # buscar exactamente 'close'
```

```
        exact = [c for c in close_cols if c == "close"]
```

```
        close_col = exact[0] if exact else close_cols[0]
```

```
    df["close"] = df[close_col]
```

```
    # 3) Buscar high / low
```

```
    high_cols = [c for c in df.columns if "high" in c]
```

```
    low_cols = [c for c in df.columns if "low" in c]
```

```
    if not high_cols or not low_cols:
```



```
raise KeyError("Falta columna con 'high' o 'low'")
df["high"] = df[high_cols[0]]
df["low"] = df[low_cols[0]]

# 4) Indicadores técnicos
df["rsi"] = RSIIndicator(df["close"], window=14).rsi()
st = StochasticOscillator(df["high"], df["low"], df["close"], window=14)
df["stoch_k"] = st.stoch()
df["stoch_d"] = st.stoch_signal()
macd_ind = MACD(df["close"], window_slow=26, window_fast=12, window_sign=9)
df["macd"] = macd_ind.macd()
df["macd_signal"] = macd_ind.macd_signal()
adx_ind = ADXIndicator(df["high"], df["low"], df["close"], window=14)
df["adx"] = adx_ind.adx()

# 5) Retorno con winsorización
df["return"] = df["close"].pct_change().clip(-1.0, 1.5)

# 6) Limpiar NaNs
return df.dropna()
```

```
from datetime import timedelta
```

```
def marcar_ventana_resultados(df, earnings_dates, pre=2, post=2):
```

```
    """
```

Añade una columna 'earnings_window' que vale 1 en los días cercanos a resultados, 0 si no.

- df: DataFrame con índice de fechas

- earnings_dates: lista de fechas (datetime.date)

- pre/post: días antes/después de la publicación que marcamos

```
    """
```

```
df['earnings_window'] = 0
fechas = set(df.index.date)
for fecha in earnings_dates:
    for delta in range(-pre, post + 1):
        dia = fecha + timedelta(days=delta)
        if dia in fechas:
            df.loc[df.index.date == dia, 'earnings_window'] = 1
return df

# -----
# 3. DESCARGA Y MERGE CON TIPOS DE INTERÉS (CON FILTRADO Y LISTADO DE
# DESCARTADOS)
# -----
datasets = {}
descartados = [] # lista para guardar los símbolos descartados

for t in tickers:
    print(f"Descargando {t}...")
    data = yf.download(t, start=start_date, end=end_date, session=session, progress=False)

    # 1) Sin datos: descartamos
    if data.empty:
        print(f" → Sin datos para {t}, se descarta.")
        descartados.append(t)
        continue

    # 2) Fecha de primer cotización después del inicio: descartamos
    primera_fecha = data.index.min()
    if primera_fecha > pd.to_datetime(start_date):
        print(f" → {t} empezó a cotizar el {primera_fecha.date()}, después de {start_date}. Se
        descarta.")
        descartados.append(t)
        continue
```

```
# 3) Sólo construimos features de los válidos
feats = build_features(data)

try:
    ticker_obj = yf.Ticker(t)
    earnings_dates = ticker_obj.get_earnings_dates(limit=60) # Puedes aumentar el límite si
    quieres
    result_dates = earnings_dates.index.date.tolist()
    feats = marcar_ventana_resultados(feats, result_dates, pre=2, post=2)
except Exception as e:
    print(f"Error obteniendo earnings para {t}: {e}")
    feats['earnings_window'] = 0 # Si hay error, todo a cero

# -----

if not feats.empty:
    datasets[t] = feats

# Al terminar, imprimimos el resumen de descartados
if descartados:
    print("\nTickers descartados por comenzar a cotizar despues de 2012:")
    print(", ".join(descartados))
else:
    print("\nNo se han descartado símbolos; todos cotizaban desde el inicio.")

# -----
# 4. DESCARGA Y PROCESAMIENTO DE RATES + VIX + CITI economic surprise
# -----
# Descargamos los ETF de tipos de interés
from pandas_datareader import data as web

rate_tickers = [
    'DGS3MO', 'DGS1', 'DGS2', 'DGS5', 'DGS7', 'DGS10', # yields soberanos
```

```
'DBAA', 'DAAA',          # Moody's Baa y Aaa Corporate Bond Yields
'BAA10Y', 'AAA10Y'      # Spreads directos
]
rates = web.DataReader(rate_tickers, 'fred', start_date, end_date)
rates.columns = [
    'rate_3m', 'rate_1y', 'rate_2y', 'rate_5y', 'rate_7y', 'rate_10y',
    'moody_baa', 'moody_aaa', 'spread_baa_10y', 'spread_aaa_10y'
]
rates = rates.ffill()

rates['spread_10y_2y'] = rates['rate_10y'] - rates['rate_2y']
rates['spread_5y_3m'] = rates['rate_5y'] - rates['rate_3m']

# --- NUEVO: descarga VIX ---
vix_raw = yf.download("^VIX", start=start_date, end=end_date, session=session,
progress=False)
# aplanar columnas
if isinstance(vix_raw.columns, pd.MultiIndex):
    vix_raw.columns = [" ".join(map(str,c)).lower() for c in vix_raw.columns]
else:
    vix_raw.columns = [c.lower() for c in vix_raw.columns]
# elegir adj close > close
vix_cols = [c for c in vix_raw.columns if "close" in c]
vix_adj = [c for c in vix_cols if "adj" in c]
col_vix = vix_adj[0] if vix_adj else vix_cols[0]

# añadimos serie VIX
rates["vix"] = vix_raw[col_vix].ffill()

# -----
# 4.b. AÑADIR VARIABLES MACROECONÓMICAS DIARIAS
# -----
```

(insertar justo después de haber construido el DataFrame `rates` con rate_10y, rate_3m y vix)

```
import pandas_datareader.data as web
```

1) Descarga y merge de Commodities: Oro y Petróleo WTI

```
commodities = yf.download(['GC=F', 'CL=F'], start=start_date, end=end_date, session=session,  
progress=False)[['Close']]
```

Renombrar columnas a minúsculas

```
commodities.columns = ['gold', 'wti']
```

Forward-fill para evitar NaNs

```
commodities = commodities.ffill()
```

Merge con rates

```
rates = rates.merge(commodities, left_index=True, right_index=True)
```

3) Descarga e incorporación del CBOE Skew Index

```
skew_raw = yf.download('^SKEW', start=start_date, end=end_date, session=session,  
progress=False)[['Close']]
```

```
skew_raw.columns = ['skew']
```

```
skew_raw = skew_raw.ffill()
```

Merge con rates

```
rates = rates.merge(skew_raw, left_index=True, right_index=True)
```

4) Descarga e incorporación de tipos de cambio: EUR/USD, CNY/USD, MXN/USD y CAD/USD

```
fx_raw = yf.download(  
    ['EURUSD=X', 'CNYUSD=X', 'MXNUSD=X', 'CADUSD=X'],  
    start=start_date,  
    end=end_date,  
    session=session,  
    progress=False  
)[['Close']]
```

Renombrar columnas a minúsculas

```
fx_raw.columns = ['eur_usd', 'cny_usd', 'mxn_usd', 'cad_usd']
```

Forward-fill para evitar NaNs

```
fx_raw = fx_raw.ffill()
```

Merge con rates

```
rates = rates.merge(fx_raw, left_index=True, right_index=True)
```

```
from statsmodels.tsa.vector_ar.vecm import VECM
```

```
vecm_df = pd.concat([
    rates[['rate_3m','rate_1y','rate_2y','rate_5y','rate_7y','rate_10y','moody_baa','moody_aaa']],
    fx_raw[['eur_usd','cny_usd','mxn_usd','cad_usd']]
], axis=1).dropna()
```

```
fecha_corte = "2022-06-30" # (el último índice de train)
```

```
fecha_corte = pd.to_datetime(fecha_corte)
```

```
start_test = fecha_corte + timedelta(days=1)
```

```
vecm_train = VECM(vecm_df.loc[:fecha_corte], k_ar_diff=2, coint_rank=4,
deterministic="n").fit()
```

3) Cálculo de los error-correction terms (ECT)

```
beta = vecm_train.beta # matriz (n_vars × 4)
```

```
ects = vecm_df.values.dot(beta) # resultado (T × 4)
```

```
ects_df = pd.DataFrame(
    ects,
    index=vecm_df.index,
    columns=[f"ect_{i+1}" for i in range(beta.shape[1])]
)
```

Escalado opcional para comparabilidad

```
scaler_ect = StandardScaler().fit(ects_df.loc[:fecha_corte])
```

```
ects_df.loc[:fecha_corte] = scaler_ect.transform(ects_df.loc[:fecha_corte])
```

```
ects_df.loc[start_test:] = scaler_ect.transform(ects_df.loc[start_test:])
```

4) Merge de los ECT en cada DataFrame de activo (antes de reindex/fill global)

for ticker in tickers:

```
datasets[ticker] = datasets[ticker].merge(
```

```
ects_df, left_index=True, right_index=True
)

sp500_raw = yf.download("^GSPC", start=start_date, end=end_date,
                        session=session, progress=False)[['Close']]
sp500_raw.columns = ['sp500']      # renombrar
sp500 = sp500_raw.ffill()         # evitar NaN
# 1) Construimos el DataFrame de las nuevas variables
# - Aprovechamos rates (DGS3MO, DGS1, DGS2, DGS5, DGS10, DBAA, DAAA)
# - Usamos sp500_df para '^GSPC'
# - Usamos commodities y fx_raw que ya tienes
vecm2_df = pd.concat([
    rates[['rate_3m', 'rate_1y', 'rate_2y', 'rate_5y', 'rate_10y', 'moody_baa', 'moody_aaa']],
    sp500,      # '^GSPC'
    commodities['gold'],      # 'GC=F'
    commodities['wti'],      # 'CL=F'
    fx_raw[['eur_usd', 'cny_usd']]      # 'EURUSD=X', 'CNYUSD=X'
], axis=1).dropna()

# 2) Entrenamos el VECM sobre train (mismo fecha_corte y start_test que antes)
vecm2_train = VECM(
    vecm2_df.loc[:fecha_corte],
    k_ar_diff=2,      # lag order = 2
    coint_rank=4,      # rango cointegración = 4
    deterministic="n"
).fit()

# 3) Calculamos los ECTs y los escalamos
beta2 = vecm2_train.beta      # (n_vars × 4)
ects2 = vecm2_df.values.dot(beta2)      # (T × 4)
ects2_df = pd.DataFrame(
    ects2,
    index=vecm2_df.index,
```

```
        columns=[f'ect2_{i+1}' for i in range(beta2.shape[1])]
    )

    # Escalado con los datos de train solamente
    scaler2 = StandardScaler().fit(ects2_df.loc[:fecha_corte])
    ects2_df.loc[:fecha_corte] = scaler2.transform(ects2_df.loc[:fecha_corte])
    ects2_df.loc[start_test:] = scaler2.transform(ects2_df.loc[start_test:])

    # 4) Merge de los nuevos ECTs en cada DataFrame de activo
    for ticker in tickers:
        datasets[ticker] = datasets[ticker].merge(
            ects2_df,
            left_index=True, right_index=True
        )

    # -----
    # 5. ÍNDICE UNIÓN + REINDEX + RELLENO + MÁSCARAS DE VALIDEZ
    # -----
    # 5.1. Para cada ticker, guardamos su primera fecha real (sin fill)
    first_valid_idx = {t: df.index.min() for t, df in datasets.items()}

    # 5.2. Construimos la unión de todos los índices junto con rates
    union_idx = sorted(
        set(rates.index)
        .union(*[df.index for df in datasets.values()])
    )

    # 5.3. Reindexamos y rellenamos (forward then backward) cada DataFrame
    for t in list(datasets):
        df = datasets[t].reindex(union_idx).ffill().bfill()
        # merge de rates (incluye rate_10y, rate_3m, vix)
        datasets[t] = df.merge(rates.reindex(union_idx).ffill(),
```



```
left_index=True, right_index=True)
```

```
# 5.4. Dividir train/test por fecha
```

```
n = len(union_idx)
```

```
cut = int(n * 0.8)
```

```
train_idx = union_idx[:cut]
```

```
test_idx = union_idx[cut:]
```

```
# Descarga SP500 completo una sola vez (antes del env)
```

```
sp500_df = yf.download("^GSPC", start=start_date, end=end_date, session=session,  
progress=False)
```

```
# Aplana MultiIndex / minúsculas
```

```
if isinstance(sp500_df.columns, pd.MultiIndex):
```

```
    sp500_df.columns = [" ".join(map(str,c)).strip().lower()
```

```
        for c in sp500_df.columns]
```

```
else:
```

```
    sp500_df.columns = [c.lower().strip() for c in sp500_df.columns]
```

```
# Elegir adj close > close
```

```
close_cols = [c for c in sp500_df.columns if "close" in c]
```

```
adj_cols = [c for c in close_cols if "adj" in c]
```

```
col_sp = adj_cols[0] if adj_cols else close_cols[0]
```

```
sp = sp500_df[col_sp].ffill().reindex(union_idx).ffill()
```

```
sp_ret_full = sp.pct_change().fillna(0).values
```

```
sp = sp500_df[col_sp].ffill().reindex(union_idx).ffill()
```

```
sp_ret_full = sp.pct_change().fillna(0).values # array alineado a union_idx
```

```
#separa en train/test:
```

```
sp_ret_train = sp_ret_full[:cut]
```

```
sp_ret_test = sp_ret_full[cut:]
```

```
# Creamos una Serie de retornos de SP500 alineada
```

```
sp_ret_series = pd.Series(sp_ret_full, index=union_idx)

# — 5.4.1. CÁLCULO DE FEATURES DE CLUSTERING EN VENTANAS DE 50 DÍAS —

for t, df in datasets.items():
    ret = df['close'].pct_change()

    clustering_feats = [
        'mean_return_cluster', 'volatility_cluster', 'cvar95_cluster',
        'skewness_cluster', 'beta_cluster', 'corr_cluster',
        'sharpe_cluster', 'tracking_error_cluster', 'ir_cluster'
    ]
    # inicializar
    for c in clustering_feats:
        df[c] = np.nan

    n = len(df)
    for start in range(50, n, 50):
        win = slice(start-50, start)
        block = slice(start, min(start + 50, n))

        win_ret = ret.iloc[win]
        sp_win = sp_ret_series.iloc[win]

        # 1) Retorno medio y volatilidad
        m_ret = win_ret.mean()
        vol = win_ret.std()

        # 2) CVaR95
        var95 = win_ret.quantile(0.05)
        cvar = win_ret[win_ret <= var95].mean()

        # 3) Skewness
        skew = win_ret.skew()
```

```
# 4) Beta y correlación
```

```
cov = win_ret.cov(sp_win)
```

```
var_sp = sp_win.var()
```

```
beta = cov / var_sp if (var_sp and not np.isnan(var_sp)) else 0
```

```
std_ret = win_ret.std()
```

```
std_sp = sp_win.std()
```

```
corr = (
```

```
    win_ret.corr(sp_win)
```

```
    if (std_ret and std_sp and not np.isnan(std_ret) and not np.isnan(std_sp))
```

```
        else 0
```

```
)
```

```
# 5) Sharpe
```

```
sharpe = m_ret / vol if (vol and not np.isnan(vol)) else 0
```

```
# 6) Tracking Error e Information Ratio
```

```
active = win_ret - sp_win
```

```
te = active.std()
```

```
ir = (active.mean() / te) if (te and not np.isnan(te)) else 0
```

```
# asignar toda la ventana "block"
```

```
idx_block = df.index[block]
```

```
df.loc[idx_block, 'mean_return_cluster'] = m_ret
```

```
df.loc[idx_block, 'volatility_cluster'] = vol
```

```
df.loc[idx_block, 'cvar95_cluster'] = cvar
```

```
df.loc[idx_block, 'skewness_cluster'] = skew
```

```
df.loc[idx_block, 'beta_cluster'] = beta
```

```
df.loc[idx_block, 'corr_cluster'] = corr
```

```
df.loc[idx_block, 'sharpe_cluster'] = sharpe
```

```
df.loc[idx_block, 'tracking_error_cluster'] = te
```

```
df.loc[idx_block, 'ir_cluster'] = ir
```

```
# rellenar primeros 50 días con primer valor válido
```

```
for c in clustering_feats:  
    df[c] = df[c].bfill()
```

```
from sklearn.cluster import KMeans  
#hacer clustering para detectar reimenos de mercado y añadir esta informacion  
numero_de_clusters = 5  
for t, df in datasets.items():  
    features = df[clustering_feats].dropna()  
    if len(features) < numero_de_clusters:  
        # Si hay pocos datos, etiqueta todo como cluster 0  
        df['regimen_mercado'] = 0  
    else:  
        kmeans = KMeans(n_clusters=numero_de_clusters, random_state=42)  
        cluster_labels = pd.Series(  
            kmeans.fit_predict(features),  
            index=features.index  
        )  
        # Asigna cluster, rellenando días vacíos  
        df['regimen_mercado'] = cluster_labels.reindex(df.index).ffill().astype(int)  
    datasets[t] = df
```

```
for t, df in datasets.items():  
    dummies = pd.get_dummies(df['regimen_mercado'], prefix='regime')  
    df = pd.concat([df, dummies], axis=1)  
    datasets[t] = df  
  
#dependencia de la cola, como se comporta los activos en la cola de distribución cuando el sp500  
tambien lo está  
  
windowcola = 252 # ventana rolling de 1 año (si los datos son diarios)  
alphacola = 0.05 # cuantil para la cola  
#cola inferior  
for t, df in datasets.items():  
    # Asegúrate de tener retornos diarios del activo y del SP500  
    ret_asset = df['return']
```

```
ret_sp = sp_ret_series

tail_dep = np.full(len(df), np.nan)
for i in range(windowcola, len(df)):
    r1 = ret_asset.iloc[i-windowcola:i]
    r2 = ret_sp.iloc[i-windowcola:i]
    q1 = np.quantile(r1, alphacola)
    q2 = np.quantile(r2, alphacola)
    # Calcula la frecuencia conjunta de estar ambos en su cola inferior
    freq = np.mean((r1 <= q1) & (r2 <= q2))
    tail_dep[i] = freq

df['tail_dep_5pct'] = tail_dep
datasets[t] = df

#cola superior
for t, df in datasets.items():

    ret_asset = df['return']
    ret_sp = sp_ret_series

    tail_dep = np.full(len(df), np.nan)
    for i in range(windowcola, len(df)):
        r1 = ret_asset.iloc[i-windowcola:i]
        r2 = ret_sp.iloc[i-windowcola:i]
        q1 = np.quantile(r1, 1-alphacola)
        q2 = np.quantile(r2, 1-alphacola)
        # Calcula la frecuencia conjunta de estar ambos en su cola inferior
        freq = np.mean((r1 >= q1) & (r2 >= q2))
        tail_dep[i] = freq

    df['tail_dep_95pct'] = tail_dep
    datasets[t] = df
```

5.5. Construir máscaras de validez para cada ticker (antes, sin CASH)

```
tickers_list = list(datasets.keys()) # length = m, sin CASH
```

```
valid_mask = np.zeros((n, len(tickers_list)), dtype=bool)
```

```
for j, t in enumerate(tickers_list):
```

```
    fv = first_valid_idx[t]
```

```
    valid_mask[:, j] = np.array(union_idx) >= fv
```

——— NUEVO: añadir columna CASH siempre válida ———

ahora tenemos m tickers + 1 CASH = n_activos

```
valid_mask = np.hstack([
```

```
    valid_mask,
```

```
    np.ones((n, 1), dtype=bool)
```

```
])
```

actualizamos lista de activos

```
activos_finales = tickers_list + ["CASH"]
```

5.6. Partir train/test incluyendo CASH

```
valid_mask_train = valid_mask[:cut]
```

```
valid_mask_test = valid_mask[cut:]
```

5.6. Obtener train_dfs y test_dfs con los índices seleccionados

```
train_dfs = {t: df.loc[train_idx] for t, df in datasets.items()}
```

```
test_dfs = {t: df.loc[test_idx] for t, df in datasets.items()}
```

5.7. Escalado y creación de tensores como antes

```
feat_names = train_dfs[next(iter(train_dfs))].columns.tolist()
```

```
scaled_tr, scaled_te = [], []
```

```
for t in tickers_list:
```

```
    tr = train_dfs[t].replace([np.inf, -np.inf], np.nan).fillna(0).values
```

```
    te = test_dfs[t].replace([np.inf, -np.inf], np.nan).fillna(0).values
```

```
sc = StandardScaler().fit(tr)
scaled_tr.append(sc.transform(tr))
scaled_te.append(sc.transform(te))

# añadir CASH como columna neutra
num_feats = len(feats_names)
scaled_tr.append(np.zeros((cut, num_feats)))
scaled_te.append(np.zeros((n-cut, num_feats)))

# apilar para mercado
market_train = np.stack(scaled_tr, axis=1) # shape (cut, n_assets+1, n_feats)
market_test = np.stack(scaled_te, axis=1)

# tensores de precios
prices_train = np.column_stack(
    [train_dfs[t]['close'].values for t in tickers_list] + [np.ones(cut)]
)
prices_test = np.column_stack(
    [test_dfs[t]['close'].values for t in tickers_list] + [np.ones(n-cut)]
)

# actualizamos lista final de activos (incluye CASH)
activos_finales = tickers_list + ["CASH"]

# -----
# Funciones auxiliares (deben ir antes de PortfolioEnv)
# -----
import numpy as np

def normalize_action(a, cash_index=-1, max_cash=0.2):
```

```
# 1) Softmax (numéricamente estable)
e = np.exp(a - np.max(a))
w = e / e.sum()

# 2) Si excede el cash cap, lo recortamos y redistribuimos el resto
if w[cash_index] > max_cash:
    # fijamos cash al máximo
    w[cash_index] = max_cash

    # redistribuimos 1-max_cash entre los otros activos
    mask_otros = np.arange(len(w)) != cash_index
    suma_otros = w[mask_otros].sum()
    if suma_otros > 0:
        w[mask_otros] = w[mask_otros] / suma_otros * (1 - max_cash)
    else:
        # en caso extremo, repartir uniformemente entre no-cash
        n_otros = mask_otros.sum()
        w[mask_otros] = (1 - max_cash) / n_otros

return w
```

```
def compute_reward(net_return, sp_return, past_net, past_port, past_sp, current_wealth,
max_wealth, penalty_coef=0.5, cvar_coef=1, cvar_window=50, alpha=5, minimoperiodos=10):
    # Recompensa base: logaritmo del retorno neto de costes
    eps = 1e-8

    # 1) Normalización del retorno neto
    periodos= np.array(past_net)
    if len(periodos)< minimoperiodos:
        rolling_std = 1.0
    else:
        raw_std = np.std(past_net) if len(past_net)>1 else 1.0
        rolling_std = max(raw_std, 1e-3) # nunca por debajo de 0.001
```



```
norm_r = net_return / (rolling_std+eps)
norm_r = np.maximum(norm_r, -0.999)
base = np.log1p(norm_r)

# Penalización por underperformance: si el SP500 sube más que tu cartera
# tras rellenar los buffers:
cum_port = np.prod([1 + x for x in past_port]) - 1
cum_sp = np.prod([1 + x for x in past_sp]) - 1
diferencial = cum_sp - cum_port
diferencial_norm = diferencial / (rolling_std + eps)
penalty = penalty_coef * diferencial_norm
# Penalización extra por drawdown severo
drawdown = (current_wealth - max_wealth) / max_wealth
drawdown = drawdown / (rolling_std + eps)
dd_pen = drawdown * 100 if drawdown < -0.2 else 0.0
# CVAR penalización cola 5% peores rendimientos
histcvar = np.array(past_net)
if len(histcvar) >= cvar_window:
    # recortamos a la ventana más reciente
    tail = np.percentile(histcvar[-cvar_window:], alpha)
    cvar = histcvar[-cvar_window:][histcvar[-cvar_window:] <= tail].mean()
else:
    cvar = 0.0

# 2) Como cvar es negativo (pérdidas), lo convertimos en penalización positiva
cvar_pen = (cvar_coef * (-cvar)) / (rolling_std + eps)

return base - penalty - cvar_pen + dd_pen

def compute_corr_matrix(prices: np.ndarray, n_assets: int) -> np.ndarray:
    """
    Dada una matriz de precios shape (T, n_assets),
    devuelve la matriz de correlaciones shape (n_assets, n_assets),

```

limpiando NaNs e infinitos.

"""

1) retornos día a día (shape (T-1, n_assets))

Evitamos división por cero con un pequeño epsilon:

eps = 1e-8

rets = (prices[1:] - prices[:-1]) / (prices[:-1] + eps)

2) correlación cruzada (shape (n_assets, n_assets))

corr = np.corrcoef(rets.T)

3) limpieza de NaN/Inf y casting a float32

corr = np.nan_to_num(corr, nan=0.0, posinf=0.0, neginf=0.0)

return corr.astype(np.float32)

6. ENTORNO GYM CON MÁSCARA

import collections

class PortfolioEnv(gym.Env):

def __init__(self, market, prices, valid_mask, sp_returns, penalty_coef=0.5, init_wealth=1.0, tc_coef=0.001, norm_window=50, perf_window=30, window=60):

super().__init__()

self.market = market

self.prices = prices

self.valid_mask = valid_mask

self.sp_ret = sp_returns

self.penalty_coef = penalty_coef

self.n_steps, self.n_assets, self.n_feats = market.shape

self.init_w = init_wealth

self.tc = tc_coef

self.past_net = collections.deque(maxlen=50)

self.window = window

self.observation_space = spaces.Dict({

'market': spaces.Box(-np.inf, np.inf,

```
        shape=(self.n_assets, self.n_feats),
        dtype=np.float32),
    'drift': spaces.Box(0.0, 1.0,
        shape=(self.n_assets,),
        dtype=np.float32),
    'corr': spaces.Box(0.0, 1.0,
        shape=(self.n_assets, self.n_assets),
        dtype=np.float32),
    'time_idx': spaces.Box(low=0, high=self.n_steps-1, shape=(1,), dtype=np.int32),
})
self.action_space = spaces.Box(0, 1, shape=(self.n_assets,), dtype=np.float32)
self.past_net = collections.deque(maxlen=norm_window)
self.past_port = collections.deque(maxlen=perf_window)
self.past_sp = collections.deque(maxlen=perf_window)
self.hist = []
global HIST_GLOBAL
HIST_GLOBAL.clear()
# ahora sí podemos llamar a reset()
self.reset()

def reset(self):
    self.i = 0
    self.wealth = self.init_w
    self.max_w = self.init_w
    # pesos iniciales proporcionados sólo a assets válidos
    valid0 = self.valid_mask[0].astype(float)
    self.prev_w = valid0 / valid0.sum()
    self.past_net.clear()
    self.past_port.clear()
    self.past_sp.clear()
    self.hist = []
    # 1) calcula la corr inicial sobre ventana de precios
    corr = compute_corr_matrix(self.prices[:self.window], self.n_assets)
```

```
# 2) arma la obs incluyendo corr
obs = {
    'market': self.market[0],
    'drift': self.prev_w.copy(),
    'corr': corr,
    'time_idx': np.array([self.i], dtype=np.int32),
}

# 3) limpia NaN/Inf y cast a float32
obs['market'] = np.nan_to_num(obs['market']).astype(np.float32)
obs['drift'] = np.nan_to_num(obs['drift']).astype(np.float32)
obs['corr'] = np.nan_to_num(obs['corr']).astype(np.float32)
return obs

def step(self, action_logits):
    # 1) Filtrar acciones no válidas
    mask = self.valid_mask[self.i].astype(float)
    action = action_logits * mask

    # 2) Calcular drift de pesos por los retornos antes de reequilibrar
    if self.i > 0:
        p0 = self.prices[self.i - 1]
        p1 = self.prices[self.i]
        r = (p1 - p0) / p0
    else:
        r = np.zeros(self.n_assets)
    g = 1 + r
    expo = self.prev_w * g
    S = expo.sum()
    w_drift = expo / S # pesos derivados por precios

    # 3) Normalizar la nueva acción respecto al cash cap
    w_new = normalize_action(action, cash_index=-1, max_cash=0.2)
```

```
# 4) Calcular coste de transacción frente al drift
tc = self.tc * np.abs(w_new - w_drift).sum()

# 5) Rebalancear: actualizar prev_w
self.prev_w = w_new.copy()

# 6) Cálculo de retornos de cartera
port_r = np.dot(w_new, r)
net_r = port_r - tc
sp_r = self.sp_ret[self.i]

# 7) Actualizar buffers históricos
self.past_net.append(net_r)
self.past_port.append(port_r)
self.past_sp.append(sp_r)

# 8) Calcular recompensa
rew = compute_reward(net_return=net_r, sp_return=sp_r, past_net=self.past_net,
past_port=self.past_port, past_sp=self.past_sp, current_wealth=self.wealth,
max_wealth=self.max_w, penalty_coef=self.penalty_coef, cvar_coef=1,
cvar_window=50, alpha=5)

# 9) Actualizar riqueza y máximo histórico
self.wealth *= (1 + net_r)
self.max_w = max(self.max_w, self.wealth)

# 10) Guardar historial (ahora con drift)
self.hist.append({'return': net_r, 'wealth': self.wealth, 'weights_drift': w_drift, 'weights':
w_new})

global HIST_GLOBAL
HIST_GLOBAL.append({'return': net_r, 'wealth': self.wealth, 'weights_drift': w_drift,
'weights': w_new})

# 11) Avanzar el contador y devolver siguiente obs
```

```
self.i += 1
done = self.i >= self.n_steps - 1
if done:
    empty_obs = {
        'market': np.zeros((self.n_assets, self.n_feats), dtype=np.float32),
        'drift': np.zeros(self.n_assets, dtype=np.float32),
        'corr': np.zeros((self.n_assets, self.n_assets), dtype=np.float32),
        'time_idx': np.array([self.i], dtype=np.int32),
    }
    return empty_obs, rew, done, {}
corr = compute_corr_matrix(
    self.prices[max(0, self.i-self.window):self.i+1],
    self.n_assets
)
obs = {
    'market': self.market[self.i],
    'drift': w_drift,
    'corr': corr,
    'time_idx': np.array([self.i], dtype=np.int32),
}
# limpia antes de devolver
obs['market'] = np.nan_to_num(obs['market']).astype(np.float32)
obs['drift'] = np.nan_to_num(obs['drift']).astype(np.float32)
obs['corr'] = np.nan_to_num(obs['corr']).astype(np.float32)
return obs, rew, done, {}

# -----
# 7. ENTRENAMIENTO / EVALUACIÓN
# -----

from stable_baselines3.common.torch_layers import BaseFeaturesExtractor
import torch as th
```

```
import torch.nn as nn
from sb3_contrib import RecurrentPPO
from stable_baselines3.common.distributions import SquashedDiagGaussianDistribution
from sb3_contrib.ppo_recurrent.ppo_recurrent import RecurrentActorCriticPolicy
from gym import spaces

# 1) Feature extractor: del state (n_assets, n_feats) al vector flatten + encod.
# — 1) Subclase de la política LSTM para usar SquashedDiagGaussian —————
import torch.nn as nn
from sb3_contrib.ppo_recurrent.ppo_recurrent import RecurrentActorCriticPolicy

class SquashedGaussianLSTMWithDropoutPolicy(RecurrentActorCriticPolicy):
    """
    Igual que RecurrentActorCriticPolicy, pero:
    - fuerza SquashedDiagGaussianDistribution
    - añade dropout entre capas de la LSTM
    """
    def __init__(self, *args, lstm_dropout: float = 0.1, **kwargs):
        # Guardamos el dropout deseado antes de llamar al init base
        self.lstm_dropout = lstm_dropout
        super().__init__(*args, **kwargs)
        # Forzamos la distribución de salida
        self.dist_class = SquashedDiagGaussianDistribution

    def _build_lstm(self, input_dim: int, lstm_hidden_size: int, n_lstm_layers: int):
        """
        Este método es llamado internamente por __init__ tras crear
        el extractor de características y la MLP. Aquí definimos
        la LSTM con dropout entre capas (solo si n_lstm_layers>1).
        """
        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=lstm_hidden_size,
            num_layers=n_lstm_layers,
```

```
        batch_first=True,
        dropout=self.lstm_dropout if n_lstm_layers > 1 else 0.0,
    )

def forward_actor(self, features, lstm_states, episode_start, **kwargs):
    # Paso 1: Pasa la salida del Transformer por la LSTM del actor
    lstm_output, lstm_states = self.lstm_actor(features, lstm_states, episode_start)
    # Paso 2: Pasa la salida de la LSTM por el MLP de la policy
    latent_pi = self.mlp_extractor.policy_net(lstm_output)
    return latent_pi, lstm_states

def forward_critic(self, features, lstm_states, episode_start, **kwargs):
    # Igual para el crítico
    lstm_output, lstm_states = self.lstm_critic(features, lstm_states, episode_start)
    latent_vf = self.mlp_extractor.value_net(lstm_output)
    return latent_vf, lstm_states

def extract_features(self, obs):

    return super().extract_features(obs)

import math

def sinusoidal_time_embedding(time_idx, d_model):
    """
    Devuelve el embedding sinusoidal (batch, d_model) para los índices de tiempo dados.
    """
    device = time_idx.device
    # (batch,)
    position = time_idx.float().unsqueeze(1) # (batch, 1)
    div_term = torch.exp(torch.arange(0, d_model, 2, device=device).float() * (-np.log(10000.0)
    / d_model))
    pe = torch.zeros((time_idx.shape[0], d_model), device=device)
    pe[:, 0::2] = torch.sin(position * div_term)
    pe[:, 1::2] = torch.cos(position * div_term)
```



```
return pe
```

```
def contar_numero_dias(index_fechas):
```

```
    """
```

```
    Recibe una lista, array o pandas Index de fechas (datetime, pd.Timestamp, strings...).
```

```
    Devuelve el número de días únicos.
```

```
    """
```

```
    import pandas as pd
```

```
    fechas = pd.to_datetime(index_fechas)      # Asegura tipo fecha
```

```
    dias_unicos = fechas.normalize().unique()   # Normaliza a día y elimina duplicados
```

```
    return len(dias_unicos)
```

```
num_dias = contar_numero_dias(union_idx)
```

```
class TransformerExtractor(BaseFeaturesExtractor):
```

```
    """
```

```
    Extrae dependencias globales con Self-Attention,
```

```
    incluye la matriz de correlaciones como input adicional y
```

```
    embeddings temporales aprendibles que indican el instante temporal (día).
```

```
    """
```

```
def __init__(self, observation_space, features_dim=256,
```

```
            d_model=80, nhead=5, num_layers=3, dropout=0, max_time_steps=num_dias+1):
```

```
    # Extrae la info de assets/features desde el observation_space
```

```
    if hasattr(observation_space, 'spaces'):
```

```
        market_space = observation_space.spaces['market']
```

```
        corr_space = observation_space.spaces['corr']
```

```
    else:
```

```
        raise ValueError("Se esperaba Dict con 'market' y 'corr'")
```

```
    n_assets, n_feats = market_space.shape
```

```
    # ==> PRIMERO inicializar la clase base <==
```

```
    super().__init__(market_space, features_dim)
```

```
self.max_time_steps = max_time_steps
self.temporal_embedding_dim = d_model

self.temb = nn.Embedding(self.max_time_steps, self.temporal_embedding_dim)

# Embedding posicional (uno por activo)
self.pos_emb = nn.Parameter(th.zeros(1, n_assets, d_model))
# Proyección de input: features + drift => d_model
self.input_proj = nn.Linear(n_feats + 1, d_model)
# Proyección de la matriz de correlaciones: (n_assets) => d_model
self.corr_proj = nn.Linear(n_assets, d_model)
self.weight_proj = nn.Linear(d_model, 1) # Capa para los pesos

# Encoder Transformer completo
encoder_layer = nn.TransformerEncoderLayer(
    d_model=d_model, nhead=nhead,
    dim_feedforward=128, dropout=dropout,
    activation='gelu', batch_first=True
)
self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

# Normalización + salida
self.ln = nn.LayerNorm(d_model)
self.output_proj = nn.Sequential(
    nn.Linear(d_model, features_dim),
    nn.ReLU()
)

def forward(self, observations):
    ...
    # 1) Extrae market y drift y concatena
    x = observations['market'] # (batch, n_assets, n_feats)
```

```

drift = observations['drift'].unsqueeze(-1) # (batch, n_assets, 1)
x     = th.cat([x, drift], dim=-1)        # (+1 en feats)

# 2) Proyección lineal + positional + correlación
x_proj = self.input_proj(x)               # (batch, n_assets, d_model)
corr_mat = observations['corr']           # (batch, n_assets, n_assets)
corr_feat= self.corr_proj(corr_mat)       # (batch, n_assets, d_model)
x       = x_proj + self.pos_emb + corr_feat

# 3) **Positional encoding sinusoidal**
t_idx = observations['time_idx']
if t_idx.dim() == 2 and t_idx.shape[1] == 1:
    t_idx = t_idx.squeeze(1)
t_idx = t_idx.to(th.long)
t_emb = sinusoidal_time_embedding(t_idx, x.shape[-1]) # (batch, d_model)
t_emb = t_emb.unsqueeze(1)                         # (batch, 1, d_model)
x = x + t_emb                                       # (batch, n_assets, d_model)

# 4) Transformer encoder
x = self.transformer(x)                           # (batch, n_assets, d_model)

# 5) Weighted mean aggregation en vez de mean(dim=1)
# Proyecta a scores por activo
weights_logits = nn.functional.relu(self.weight_proj(x)) # (batch, n_assets, 1)
weights = torch.softmax(weights_logits, dim=1)           # (batch, n_assets, 1)
# Weighted sum
x_weighted = (x * weights).sum(dim=1)                   # (batch, d_model)
x_weighted = self.ln(x_weighted)
return self.output_proj(x_weighted)

```

#	—	3)	policy_kwargs	sin	dist_class
---	---	----	---------------	-----	------------

```
policy_kwargs = dict(
    features_extractor_class=TransformerExtractor,
    features_extractor_kwargs=dict(features_dim=256, d_model=80, nhead=5, num_layers=3,
max_time_steps=num_dias+1),
    net_arch=dict(pi=[192, 128,62], vf=[192, 128, 64, 32]),
    lstm_hidden_size=256,
    n_lstm_layers=2,
    lstm_dropout=0
)

from stable_baselines3.common.vec_env import VecNormalize

# Pasamos también la máscara al crear el env
env_tr = DummyVecEnv([lambda: PortfolioEnv(market_train, prices_train, valid_mask_train,
sp_returns=sp_ret_train, penalty_coef=0.5)])

#normalizacion para entrenamiento estable
env_tr = VecNormalize(
    env_tr,
    norm_obs=False,      # normaliza observaciones
    norm_reward=True,    # normaliza recompensas
) # recorta observaciones normalizadas a  $\pm 10\sigma$ 

model = RecurrentPPO(SquashedGaussianLSTMWithDropoutPolicy, env_tr,
policy_kwargs=policy_kwargs, verbose=1, ent_coef=1.5210089527581027e-05,
learning_rate=3.3896027707827866e-06,
    batch_size=512, n_steps=768, n_epochs=20, gamma=0.8587760378271281,
clip_range=0.14188380242544762, clip_range_vf= 0.1, gae_lambda=0.9208035456282112,
max_grad_norm=0.9562075837093198, vf_coef=0.8139391556754874, device="cuda")

model.learn(total_timesteps=100000)
model.save("ppo_portfolio_model_TRFMLSTM3")

# 1. Construimos el entorno de evaluación (NO training para la normalización)
base_env_ev = PortfolioEnv(market_test, prices_test, valid_mask_test, sp_returns=sp_ret_test,
penalty_coef=0.5)
env_ev = DummyVecEnv([lambda: base_env_ev])
```

2. Normalización, compartiendo estadísticas con el entorno de entrenamiento

```
env_ev = VecNormalize(  
    env_ev,  
    norm_obs=False,  
    norm_reward=True,  
    training=False # Muy importante: en test, no actualizar stats  
)  
env_ev.ret_rms = env_tr.ret_rms # Copiamos estadísticas de recompensas  
env_ev.gamma = env_tr.gamma  
  
print(f"market_test.shape: {market_test.shape}") # ¿Cuántos pasos deberías tener en test?  
  
obs = env_ev.reset()  
state = None  
episode_start = [True]  
done = [False] # <-- ¡ojo! debe ser lista para VecEnv  
total_reward = 0  
step_count = 0  
  
while not done[0]:  
    act, state = model.predict(obs, state=state, episode_start=episode_start, deterministic=True)  
    obs, reward, done, _ = env_ev.step(act)  
    total_reward += reward[0]  
    episode_start = [done[0]]  
    step_count += 1  
  
print(f"Número de pasos recorridos en test: {step_count}")  
  
portfolio_env_real = env_ev.venv.envs[0]  
hist = pd.DataFrame(HIST_GLOBAL)  
  
if hist.empty:
```

```
print("¡OJO! El historial sigue vacío tras la evaluación. Debug info:")
print("¿Quizá el entorno termina en el primer paso?")
print("Posibles causas: longitud de test, error en lógica de step/reset, máscara de activos sin válidos, etc.")
# Salir o lanzar excepción aquí si lo deseas
import sys; sys.exit(1)
else:
    hist.index = test_idx[:len(hist)]
    print("Columnas en hist:", hist.columns.tolist())
    print(hist.head())
# -----
# BENCHMARK SP500 y RETORNOS (sin volver a descargar)
# -----
# convierto el array de retornos de SP500 en la etapa de test en Series
sp_series = pd.Series(sp_ret_test, index=test_idx)
print("Columnas en hist:", hist.columns.tolist())
print("Número de filas en hist:", len(hist))

# calculo los retornos de tu cartera y alineo fechas
port_ret = hist['return']
sp_aligned = sp_series.loc[port_ret.index]

# exceso y métricas
excess = port_ret.values - sp_aligned.values
mean_exc_ann = excess.mean() * 252
te_ann = excess.std() * np.sqrt(252)
IR = mean_exc_ann / te_ann

ret_ann = (np.prod(1+port_ret)**(252/len(port_ret)) -1)
vol_ann = port_ret.std()*np.sqrt(252)
rf_10y = rates['rate_10y'].iloc[-1]/100
SR = (ret_ann - rf_10y)/vol_ann
max_dd = (hist['wealth'].cummax()-hist['wealth']).max()
```

```
print(f"Wealth final: {hist['wealth'].iloc[-1]:.4f}")
print(f"Sharpe ratio anual: {SR:.2f}")
print(f"Tracking error anual: {te_ann:.4f}")
print(f"Information Ratio: {IR:.2f}")
print(f"Máximo drawdown: {max_dd:.2%}")
print(f"rentabilidad anual: {ret_ann:.4f}')
print(f"volatilidad anual: {vol_ann:.4f}')
# -----
# 6. GRAFICAS
# -----
# 6.1 Valor vs SP500 (alineado y acumulado)
sp_cum = (1 + sp_aligned).cumprod()

plt.figure(figsize=(10,5))
plt.plot(hist.index, hist['wealth'], label="Portafolio")
plt.plot(sp_cum.index, sp_cum.values, label="SP500")
plt.title("Valor del portafolio VS SP500")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

# 6.2 Evolución de pesos (Top10)
weights = np.vstack(hist['weights'].values)
wdf = pd.DataFrame(weights, columns=activos_finales, index=hist.index)
top10 = wdf.mean().sort_values(ascending=False).head(10).index

plt.figure(figsize=(12,6))
for t in top10:
    plt.plot(wdf.index, wdf[t], label=t)
plt.title("Pesos asignados (Top 10)")
plt.legend(); plt.grid(); plt.tight_layout(); plt.show()
```

```
# 6.3 Bonos y efectivo
plt.figure(figsize=(10,4))
for b in ["TLT","IEF","SHY"]:
    if b in wdf:
        plt.plot(wdf.index, wdf[b], label=f"{b} (Bono)")
plt.plot(wdf.index, wdf['CASH'], label="CASH")
plt.title("Exposición a bonos y efectivo")
plt.legend(); plt.grid(); plt.tight_layout(); plt.show()

# Guardar pesos
wdf.to_csv("pesos_portafolio_TRFMLSTM3.csv")
```